
A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)

Affiliation

Address

email

1 **Revision note.** All text, tables and figures added in this revision are typeset in red (new figures carry
2 a red frame); everything in black is unchanged from the submitted version.

Abstract

3 When solving PDEs, classical numerical solvers are often computationally ex-
4 pensive, while machine learning methods can suffer from spectral bias, failing to
5 capture high-frequency components. Designing an optimal hybrid iterative solver—
6 where, at each iteration, a solver is selected from an ensemble of solvers to leverage
7 their complementary strengths—poses a challenging combinatorial problem. While
8 greedy selection is desirable for its constant-factor approximation guarantee to the
9 optimal solution under Lipschitz assumptions, it requires knowledge of the true
10 error at each step, which is unavailable in practice. We address this by proposing
11 an approximate greedy router that efficiently mimics a greedy approach to solver
12 selection. Empirical results on the Poisson and convection–diffusion equations
13 show that our method consistently reduces final error and area-under-the-curve
14 (AUC) of the error trajectory relative to single-solver baselines and existing hybrid
15 approaches such as HINTS. In particular, our method reaches comparable error
16 levels in substantially fewer iterations while exhibiting more stable error decay.
17 With a cost-aware instantiation of the greedy rule, a lightweight router, and a band-
18 limited scale-equivariant corrector, these gains carry over to end-to-end wall-clock
19 time on 128^2 to 512^2 grids: in every one of the 15 solver–equation pairings at 128^2
20 (five classical solvers; Poisson, convection–diffusion and anisotropic diffusion)
21 the learned router reaches truncation-level accuracy $238\times$ – $1983\times$ faster than the
22 classical solver alone (which never reaches it on the anisotropic equation) and
23 $2.68\times$ – $11.8\times$ faster than HINTS, it beats HINTS on every grid even when the
24 latter’s period is tuned per setting, and it is faster than geometric multigrid and
25 multigrid-preconditioned Krylov methods on the isotropic equations; routing over
26 ensembles of classical solvers is never worse than the best pair and strictly better
27 where the error passes through regimes that favour different solvers.

28 1 Introduction

29 Natural phenomena and engineered systems are often governed by ordinary and partial differential
30 equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the
31 system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay,
32 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing
33 heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010],
34 and tackling inverse problems based on external measurements (e.g., reconstructing brain activity
35 from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which discretize the spatial and/or temporal domain and approximate the solution by solving a system of equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006, Brenner, 2008] construct a system of equations based on local basis functions, and then rely on numerical linear algebra techniques—such as the Jacobi and Gauss-Seidel method [Saad, 2003]—to compute the solution. These iterative methods, however, lack generalization across different initial conditions, boundary conditions, or forcing functions, as even minor changes to any of these parameters require re-solving the entire system of equations from scratch.

These challenges have motivated the development of neural operators [Kovachki et al., 2023]—a class of machine learning models that aim to learn the solution operator directly, enabling fast inference across a range of parameters. Neural operators lift the classical linear layer in neural networks to an operator—typically a kernel integral operator—between function spaces. Despite strong approximation guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning low-frequency components of the solution function and often struggle to capture high-frequency features that iterative solvers excel at.

Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that interleaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While HINTS reports significant empirical gains in convergence and accuracy over the standalone neural operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction may increase the error and undo recent progress.

Although state-dependent solver schedules are desirable, this naturally suggests a reinforcement learning formulation for learning routing policies. However, in our setting, the structure of iterative PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when the final error satisfies (weak) supermodularity—so that applying a beneficial solver earlier is at least as valuable as applying it later. To support this approach, we make the following contributions:

- We present a general hybrid PDE solver in which a routing rule chooses, at each iteration, from a set of classical solvers and neural operators. With oracle access to the true error at every step, we show that a greedy routing rule achieves a constant-factor approximation to the optimal strategy for linear PDEs, provided the updates are error-reducing (Lipschitz with constant < 1) and zero-preserving—conditions under which weak supermodularity holds.
- Given that the true error is not observed at test time, a convex surrogate loss is introduced, which, when minimized, enables the learned model to imitate the greedy router without access to true error information. This alignment is guaranteed through Bayes consistency.
- We empirically demonstrate that our approximate greedy solver achieves faster convergence than HINTS and individual solvers, often reaching comparable error levels in fewer iterations on the Poisson and convection-diffusion equations.

2 Background

Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \quad (1)$$

where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the coefficient function a , f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and b is the boundary term. Such PDEs can be solved numerically using various discretization strategies, such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd, 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with difference quotients (e.x., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D . For a function $g : D \rightarrow \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$

86 is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \quad (2)$$

87 with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which
 88 retains function space formulations for discretization invariance, we follow the conventions of classical
 89 numerical analysis and represent functions and operators as vectors and matrices. Discretization
 90 invariance is not essential here, since we focus on fixed-grid problems.

91 2.1 Iterative PDE Solvers

92 Direct methods like Gauss Elimination and Thomas Algorithm can be computationally expensive in
 93 high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods like
 94 Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually
 95 converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C \left(f_h - \mathcal{L}_h^a u^{(t)} \right), \quad (3)$$

96 where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$,
 97 where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict
 98 lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

99 2.2 Neural Operators

100 Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i =$
 101 $\mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to
 102 approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f) \sim \mathcal{P}} \left[\int_D \|u_i(x) - \mathcal{G}_\theta(a_i, f_i)(x)\|_2^2 dx \right]$$

103 Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural
 104 Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which
 105 perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which
 106 employ a trunk-branch decomposition to map input functions to output functions [Lu et al., 2021].

107 2.3 Greedy Optimization

108 Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \rightarrow \mathbb{R}$ is a set function defined
 109 on subsets of a set Ω . An exhaustive search is infeasible, so an efficient alternative is the greedy
 110 algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal
 111 gain, i.e. $\omega^* = \arg \max_{\omega \in \Omega \setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements
 112 never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a
 113 $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g
 114 is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$;
 115 intuitively, delaying an addition cannot increase its benefit. For set function minimization problems,
 116 supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves
 117 constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

118 The suboptimality of the greedy algorithm has been extensively studied for both set-function mini-
 119 mization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011,
 120 Bian et al., 2017, Harshaw et al., 2019] when standard assumptions—non-negativity, monotonicity, and
 121 sub/supermodularity—are weakened. In contrast, results on greedy sequence maximization [Streeter
 122 and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al.,
 123 2024, Tschitschek et al., 2017]—where the ordering of the elements affects the function—have led
 124 to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the
 125 suboptimality of the greedy solution to minimizing the final error.

126 3 General framework for Hybrid Solvers

127 To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t} \left(f_h - \mathcal{L}_h^a u_h^{(t)} \right) \quad (4)$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \dots, K\}$ indexes the function chosen at step t . Here, we use “preconditioning function” broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given by $S_t = \mathbf{1}_{t \bmod \tau > 0} + 1$, which selects C_1 every τ steps and C_2 otherwise.

Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t . Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t} \left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)} \right) = (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(t)})$$

where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j . Here, “ $\circ$ ” denotes function composition (matrix multiplication for linear updates). The objective is to select a sequence $S = (S_1, \dots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \leq T} h(S) := \left\| (I - C_{S_{|S|}} \circ \mathcal{L}_h^a) \circ \dots \circ (I - C_{S_1} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \quad (5)$$

with compositions applied from right to left, so that the S_1 update acts on the initial error.

4 Greedy Algorithm

Equation (5) defines a combinatorial optimization problem with a search space exponential in T , making exact optimization intractable for large T or K . As a starting point, we consider an “omniscient” greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is Bayes consistent with this omniscient rule, thereby recovering the guarantees shown below.

As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule—such as the one described in Algorithm 1—enjoys constant-factor approximation guarantees. However, classical results focus on *set* functions, with only recent extensions made to sequences. Building on this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
 $S^0 \leftarrow \emptyset$
for $t < T$ **do**
 $S^{t+1} \leftarrow S^t \oplus \arg \min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
 $e_h^{(t+1)} \leftarrow (I - C_{S^{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
end for
return S^T

Let Ω^* denote the space of sequences with elements in Ω . For $S, S' \in \Omega^*$, we denote their concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function $g : \Omega^* \rightarrow \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \leq g(S)$ for all $S, S' \in \Omega^*$, and postfix monotonically non-increasing if $g(S' \oplus S) \leq g(S)$ for all $S, S' \in \Omega^*$. A prefix non-increasing function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \geq g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \quad (6)$$

However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular

161 with respect to $S' \in \Omega^*$ if, for any $S \in \Omega^{|S'|}$, there exists $\alpha(S') \geq 1$ such that

$$g(S) - g(S \oplus S') \leq \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \quad (7)$$

162 The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence super-
 163 modularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) -$
 164 $g(S \oplus (S'_1, \dots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is
 165 controlled by the effect of appending S'_i directly to S . Thus, postponing the inclusion of S'_i cannot
 166 yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies
 167 the extent to which future gains from delays may exceed immediate gains.

168 Having introduced these notions, we now characterize the suboptimality of greedy solutions of weakly
 169 supermodular and postfix monotonic sequence functions in Theorem 4.1.

170 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal*
 171 *solution $O = \arg \min_{S \in \Omega^T} h(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let*
 172 *the greedy solution of length T be S^T . If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O))) (g(\emptyset) - g(O))$$

173 The proof of Theorem 4.1 appears in Section A.2. As $T \rightarrow \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases to
 174 $1 - e^{-1/\alpha(O)}$, so the worst-case performance of the greedy rule saturates rather than degrades with
 175 horizon. Additionally, larger $\alpha(O)$, which indicates higher reward for delayed inclusions, loosens the
 176 suboptimality bound. Theorem 4.1 requires that the sequence objective h be weakly supermodular
 177 with respect to the optimal solution and postfix monotone—properties established in Proposition 4.2.
 178 We defer the proof to Section A.3.

179 **Proposition 4.2.** *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is*
 180 *ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is*
 181 *weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

182 Furthermore, if $I - C_j \circ \mathcal{L}_h^a$ is invertible for all $j \in [K]$, h is also postfix non-increasing.

183 The conditions of Proposition 4.2 are natural. For classical solvers, the Lipschitz constant is $\|I_N -$
 184 $C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs (i.e., the update is damping
 185 errors). For neural networks, Lipschitz continuity can be enforced via weight regularization [Gouk
 186 et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$ well enough to make the
 187 Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both natural and desirable: it
 188 precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical
 189 schemes, and it can be enforced for any learned model by excluding bias terms.

190 The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is
 191 governed by the collective contraction factors of the solvers chosen by the optimal solution: as
 192 $\sum_{i=1}^T \rho_{O_i}^2 \rightarrow T$, $\alpha(O)$ grows, resulting in a weaker bound. Invertibility of the error propagation
 193 functions is often satisfied with Jacobi and Gauss-Seidel updates. However, neural networks with di-
 194 mension changes or ReLU activations may yield non-invertible error propagation maps. Nevertheless,
 195 our experiments show that greedy routers remain effective even when invertibility is not met.

196 Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is
 197 nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis,
 198 supermodularity is established. This occurs, for example, for linear, constant-coefficient PDEs
 199 with periodic boundary conditions where the solver ensemble includes Jacobi, Gauss-Seidel, and a
 200 single-layer linear Fourier Neural Operator. The proof of Theorem 4.3 is deferred to Section A.5.

201 **Proposition 4.3.** *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$. Then, h is*
 202 *supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware without changing the analysis: let c_j denote the (measured) cost of one application of C_j , take the cost u of one neural-operator call as the unit, and replace each preconditioner by the *macro-action* $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \text{round}(u/c_j)\}$ consecutive iterations, so that all actions cheaper than the unit cost approximately u ; an action dearer than the unit (a multigrid cycle, say) is applied once and its error is compared per unit of cost, $\|e\|^{u/(m_j c_j)}$, which reduces to the plain comparison when the costs match. For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m -fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is ρ_j -Lipschitz, zero-preserving and invertible, then so is its m_j -fold composition (with constant $\rho_j^{m_j} < 1$), and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to the macro-action ensemble. Running Algorithm 1 on macro-actions selects, at every decision epoch, the action with the largest error reduction per unit cost, and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$; Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t , router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j , resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) = \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \quad (8)$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} [l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router's own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a) (e_h^{(t-1)}) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*} (f_h - \mathcal{L}_h^a u_h^{(t-1)})$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h) . At each t , l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$. Thus the input distribution seen by r depends on (a_h, f_h) , not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a) (e_h^{(t)}) \right\|_2^2 \quad (9)$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

244 5.1 Surrogate Loss

245 Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice.
 246 We instead introduce a surrogate loss that (1) is convex, enabling efficient optimization; (2) upper
 247 bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the
 248 original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes
 249 consistent with respect to the loss l if

$$\lim_{n \rightarrow \infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* \implies \lim_{n \rightarrow \infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^*$$

250 where $\mathcal{R}_l(f) = \mathbb{E}[l(f(X), Y)]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data,
 251 if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ , it also
 252 converges to the optimal risk with respect to the original loss l .

253 To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$
 254 with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \rightarrow \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \arg\max_{j \in [K]} g_j(a, f, u^{(t)})$. For
 255 example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss
 256 yields the same decision as minimizing Equation (8):

$$\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) = - \sum_{j=1}^K \sum_{k=1}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h) \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{m=1}^K \exp(g_m(a_h, f_h, u_h^{(t)}))} \right) \quad (10)$$

257 where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ -risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) =$
 258 $\mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to $\mathbf{g}$ follows from the convex-
 259 ity of log-softmax function in its inputs. Moreover, Ψ upper bounds l_{route} up to a constant factor, and
 260 we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that Ψ achieves Bayes
 261 consistency with respect to l_{route} ; the proof can be found in Section B.3.

262 **Theorem 5.1.** *Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that*
 263 *$\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete*
 264 *operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .*

265 Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy
 266 for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional
 267 class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy
 268 with instance-dependent weights $\sum_{k \neq j}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h)$. The uniform upper bound holds when all
 269 preconditioning functions are error-damping. It is also reasonable to assume at least one solver cannot
 270 annihilate the error in one step, yielding the lower bound. Under these conditions, minimizing Ψ
 271 recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). Following
 272 the work of Mao et al. [2024], the proof uses standard conditional-risk calibration: we relate the
 273 excess risks of l_{route} and Ψ and take the infinite-sample limit.

274 6 Related Works

275 **Hybrid PDE Solvers:** Early data-driven solvers sought convergence guarantees by predicting
 276 parameters—e.g., preconditioning matrices, multi-grid smoothers or restriction matrices—within iterative
 277 schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025,
 278 Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates’
 279 ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer
 280 only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers
 281 that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi
 282 steps, then one DeepONet correction), achieving faster convergence than the standalone numerical
 283 solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error
 284 mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin,
 285 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two
 286 ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly
 287 rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in
 288 significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics—e.g., thresholding a cheap model’s uncertainty estimates [Chuang et al., 2024, 2025]—often poorly balance the cost-accuracy tradeoff, motivating many systems to instead learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: “experts” (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We empirically demonstrate the fast, uniform convergence of the approximate greedy router on the Poisson and Convection-Diffusion (ConvDiff) equations posed on the unit domain $D = [0, 1]^2$:

$$\forall x \in D \quad \textbf{Poisson: } -\Delta u(\mathbf{x}) = f(\mathbf{x}) \quad \textbf{ConvDiff: } -\Delta u(\mathbf{x}) + 20 \frac{\partial}{\partial x_1} u(\mathbf{x}) + 20 \frac{\partial}{\partial x_2} u(\mathbf{x}) = f(\mathbf{x})$$

We impose periodic boundary conditions for both equations, and center f to satisfy the compatibility condition $\int_D f(\mathbf{x}) d\mathbf{x} = 0$; Dirichlet results are deferred to Section D.8. The domain D is discretized on a 31×31 uniform grid; results on a finer grid are provided in Section D.7. Data is sampled from a zero-mean Hierarchical Gaussian Random Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ vary across samples (See Section C). We run two experimental settings. In the first, we restrict to solver pairs ($K = 2$) and compare against HINTS and standalone classical solvers. In the second, we assess performance as the solver ensemble grows; since HINTS does not support this setting, we simply compare against individual solvers. In both cases, we consider a range of iterative methods described below. Performance is reported over 128 test samples via the mean final error $\|e_h^{(T)}\|_2$ and the mean area under the curve (AUC), defined as $\sum_{t=1}^T \|e_h^{(t)}\|_2$; the “curve” refers to the error over iterations. AUC characterizes the full convergence behavior, which is critical in practice where solutions must be reached in minimal time.

Paired Solver Experiments: We seek to optimally solve the aforementioned PDEs with access to a single numerical scheme and a trained DeepONet. In particular, we consider settings coupling the DeepONet with Jacobi, Gauss-Seidel (GS), and Symmetric Gauss-Seidel (SymGS) solvers. For each setup, we compare our learned router against the single-solver schedules (Jacobi only, GS only, and SymGS only) and HINTS variants (HINTS-Jacobi, HINTS-GS, HINTS-SymGS), where the DeepONet correction are applied every 24 iterations. We train LSTM-based routers using the loss in Equation (10) separately for each pairwise settings. The LSTM captures the sequential nature of routing decisions where the benefit of each action depends on the error trajectory and past choices. Training details of the DeepONet and the routers can be found in Section C. To ensure fairness, comparisons are restricted to methods with identical solver access, and are grouped by solver family (e.g., Jacobi-related methods). In addition, we include a *true greedy* oracle, which selects at each iteration the solver that yields the largest immediate error reduction. This oracle has access to the ground-truth error and is therefore not implementable in practice, but helps evaluate how well the learned router approximates the optimal policy. Each method is run for 300 iterations.

As shown in Table 1, the greedy router achieves the lowest final error and AUC across all solver families, outperforming their single-solver and HINTS counterparts. Moreover, its performance closely tracks that of the true greedy oracle, indicating that our surrogate-based training procedure effectively approximates the greedy strategy. This can be further seen in the routing visualizations in

Table 1: Final error and AUC of L_2^2 error (lower is better). Values are mean ($\pm$ standard error) over 128 test instances; Error is reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Bold indicates the best method within each solver family. True-Greedy denotes an oracle policy with access to exact error information

Equation	Poisson		ConvDiff	
Methods	$\ e_h^{(T)}\ \times 10^3$	AUC	$\ e_h^{(T)}\ \times 10^3$	AUC
Jacobi-related Solvers				
Jacobi Only	0.383 (1.029)	0.821 (2.154)	0.136 (0.364)	0.312 (0.788)
HINTS-Jacobi	0.759 (0.016)	0.393 (0.590)	0.447 (0.013)	0.202 (0.256)
Learned Greedy-Jacobi	0.054 (0.142)	0.165 (0.368)	0.033 (0.097)	0.098 (0.239)
True-Greedy-Jacobi	0.021 (0.018)	0.094 (0.170)	0.012 (0.012)	0.049 (0.091)
GS-related Solvers				
GS only	0.019 (0.051)	0.435 (1.141)	$< 10^{-3}$	0.088 (0.219)
HINTS-GS	0.724 (0.000)	0.325 (0.488)	0.456 (0.000)	0.130 (0.154)
Learned Greedy-GS	0.002 (0.004)	0.083 (0.152)	$< 10^{-3}$	0.031 (0.078)
True-Greedy-GS	0.001 (0.001)	0.057 (0.107)	$< 10^{-3}$	0.019 (0.038)
SymGS-related Solvers				
SymGS only	$< 10^{-3}$	0.227 (0.593)	$< 10^{-3}$	0.071 (0.178)
HINTS-SymGS	0.709 (0.000)	0.252 (0.380)	0.463 (0.000)	0.116 (0.131)
Learned Greedy-SymGS	$< 10^{-3}$	0.052 (0.102)	$< 10^{-3}$	0.022 (0.039)
True-Greedy-SymGS	$< 10^{-3}$	0.034 (0.066)	$< 10^{-3}$	0.016 (0.032)

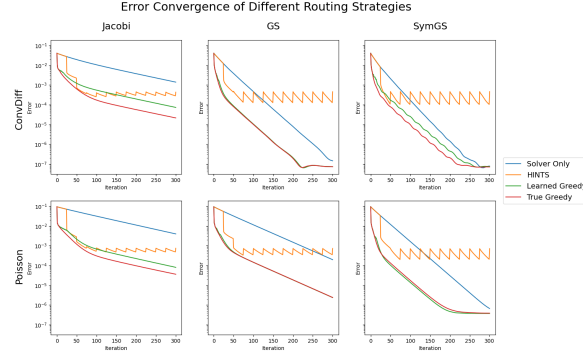


Figure 1: Convergence histories for representative test instances. Rows: Convection Diffusion (top) and Poisson (bottom). Columns: Jacobi, Gauss–Seidel (GS), and Symmetric Gauss–Seidel (SymGS). Greedy yields near-monotone decay and the lowest errors, whereas HINTS shows sawtooth behaviors.

Section D.6, which demonstrate that the router learns qualitatively similar strategies to the oracle. The improvements over HINTS are particularly notable: while HINTS can improve over single-solver schedules, it exhibits oscillatory error behavior (See Figure 1) due to invoking the DeepONet at suboptimal times. In contrast, the greedy routers selectively applies updates that reduce error, resulting in near-monotone convergence. These gains arise from adapting routing decisions across both samples for a given PDE and PDEs. For instance, the neural operator is used more frequently in the convection–diffusion setting than in Poisson (Figure 2), a behavior that must be manually

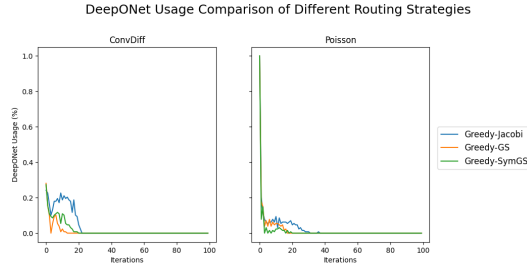


Figure 2: DeepONet usage across iterations for different greedy routing strategies on convection–diffusion (left) and Poisson (right). The variability across iterations highlights the non-uniform, sample-dependent nature of the learned routing strategy. See Section D.6 for detailed routing trajectories. For readability, we display the first 100 iterations (out of 300 total).

specified in HINTS but is learned automatically by our method. This shows that improvements stem from selective use of the learned component, rather than frequent invocation as in HINTS.

Solver Ensembles Experiments: We consider routing over a collection of more than two solvers. Specifically, we study ensembles $\mathcal{W}$ of increasing size, consisting of a trained DeepONet and a diverse set of numerical solvers, including Jacobi, GS, SymGS, Damped Jacobi ($\omega = 0.67$) and Successive Over-relaxation (SOR) with $\omega = 1.5$. Our goal is to evaluate whether enlarging the solver set yields gains beyond pairwise configurations. To this end, we compare Router($\text{NO} \cup \mathcal{W}$) against the best pairwise version of our method, Router($\text{NO} \cup \{\text{solver}\}$) for solver $\in \mathcal{W}$. As shown in Table 3, larger ensembles often yield improvements over both final error and AUC relative to the best pairwise configuration, indicating that additional solver diversity can be leveraged. The router achieves these gains by learning to route to each member of the ensemble for a non-trivial proportion of the iterates, as measured in Section D.6. Finally, despite the increased size of the action space, the learned router continues to produce improvements, demonstrating robustness of both the training procedure and the convex surrogate of Section 5.1; true greedy comparisons are deferred to Section D.1. These results show that our method can scale to richer solver sets, enabling improved performance without sacrificing learnability.

Wall-clock time-to-solution: Iteration counts favor methods whose iterations are expensive, so we complement the results above with an end-to-end wall-clock study on 128^2 , 256^2 and 512^2 grids for all five classical solvers, geometric multigrid, and both PDEs, plus a third, harder PDE—anisotropic diffusion $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ on the same periodic domain and data, the standard case in which point smoothers and multigrid with point smoothing degrade; Section E details the protocol and collects all results per equation. Three ingredients make the comparison meaningful and the router deployable: (i) all classical solvers are implemented as $O(N^2)$ stencil or cached sparse-triangular sweeps (matching the dense implementations used elsewhere to machine precision), so the classical baseline is not artificially slowed; (ii) the DeepONet corrector acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly, so one call solves the smooth band of the spectrum to $\approx 10^{-6}$ at the cost of about 12 Jacobi or 3 Gauss–Seidel sweeps at 128^2 (and only 2.5 Jacobi sweeps at 512^2) and never injects high-frequency error; and (iii) the router is made *cost-aware* through the cost-equalised macro-actions of Section 4, and the LSTM is replaced by a router over $6+K$ scalar features (log residual, its recent decay rate, the previous action, and call counts) with a two-layer MLP whose decision costs $\approx 5 \mu\text{s}$, trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle. Table 2 reports, for every pairing, equation and grid, the median wall-clock time to reach truncation-level relative error $\varepsilon = h^2$ —the accuracy beyond which further algebraic refinement is not meaningful on an $N \times N$ grid—for HINTS with the published period $\tau=25$, for HINTS with the best period $\tau \in \{5, 10, 25, 50\}$ chosen *a posteriori* on the test set for that cell, and for the learned router, together with the classical methods a practitioner would use on these problems (multigrid alone and multigrid-preconditioned CG or BiCGSTAB). On the 128^2 grid the router is $238\times$ – $1983\times$ faster than the classical solver alone on the two isotropic equations (on the anisotropic equation no point smoother reaches h^2 at all within 60,000 iterations), $2.68\times$ – $11.8\times$ faster than HINTS ($\tau=25$) and $1.21\times$ – $3.22\times$ faster than the best period in every one of the 15 solver–equation pairings; on 256^2 it is $1.58\times$ – $10.9\times$ and $1.11\times$ – $2.33\times$ faster in all 13 pairings; and on 512^2 still $1.05\times$ – $3.10\times$ and $1.09\times$ – $1.24\times$ faster in all 6 stationary pairings, with the $\{\text{NO}, \text{multigrid}\}$ router the fastest iterative method we measured at that size (all paired Wilcoxon $p < 0.01$; the margins shrink at 512^2 for the Jacobi and GS pairings because the corrector’s fixed band leaves those smoothers a slowly damped band that every schedule has to sweep through, Section E). Against the classical methods without corrector, the router with its best stationary pairing is $3.62\times$ – $3.71\times$ faster than geometric multigrid alone and $3.85\times$ – $5.31\times$ faster than the best multigrid-preconditioned Krylov method at 128^2 , and routing over $\{\text{NO}, \text{multigrid}\}$ is itself $1.93\times$ – $2.03\times$ faster than multigrid alone. The router tracks the cost-aware oracle within $1.06\times$ in median time, its decisions are identical across five independent training seeds, and the iteration-based metrics of Table 1 carry over to the finer grid. Figure 3 shows where the gain comes from: the router calls the corrector once at the very start (where the error is smooth and the corrector removes four orders of magnitude at once), spends a solver-dependent number of sweeps on the high-frequency band, and calls it again only if the smooth error re-emerges—between one and three calls in total—while HINTS spends $\tau-1$ sweeps before its first call and keeps calling the corrector when it no longer helps. Finally, the same construction extends to ensembles of classical solvers (Section E.5): a router over a larger set is never slower than the best pairwise router of its members

Table 2: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ for every solver pairing $\{\text{NO, solver}\}$, equation and grid ($64, 32$ and 16 test instances at $128^2, 256^2$ and 512^2): HINTS with the published period $\tau=25$, HINTS with the best period chosen per cell on the test set itself (subscript), and the learned router (paired median speedup over HINTS-25 / over the best period in parentheses; bold: faster than both). The classical solver alone is 10^2 – $10^3 \times$ slower than the router on the isotropic equations and never reaches h^2 on the anisotropic one (Section E). The last rows of each block give geometric multigrid alone and the best multigrid-preconditioned Krylov method, without corrector, on the same instances. $\dagger n$: n instances did not reach h^2 within the iteration cap.

Equation	Pairing	128 $\times$ 128			256 $\times$ 256			512 $\times$ 512		
		HINTS-25	best τ	router (ours)	HINTS-25	best τ	router (ours)	HINTS-25	best τ	router (ours)
Poisson	Jacobi	2.65 ms	1.10 ms _{$\tau=5$}	987 μs ($2.68 \times / 1.21 \times$)	5.95 ms	2.47 ms _{$\tau=5$}	2.38 ms ($2.52 \times / 1.56 \times$)	87.0 ms	87.0 ms _{$\tau=25$}	78.8 ms ^{†2} ($1.11 \times / 1.11 \times$)
	Jacobi (0.67)	2.71 ms	1.13 ms _{$\tau=5$}	912 μs ($2.90 \times / 1.28 \times$)	6.41 ms	3.43 ms _{$\tau=5$}	2.83 ms ($2.26 \times / 1.73 \times$)	—	—	—
	GS	6.65 ms	1.99 ms _{$\tau=5$}	1.01 ms ($6.75 \times / 1.94 \times$)	22.0 ms	6.07 ms _{$\tau=5$}	3.92 ms ($5.77 \times / 1.70 \times$)	142.8 ms	142.8 ms _{$\tau=25$}	131.5 ms ($1.11 \times / 1.11 \times$)
	SymGS	10.3 ms	2.60 ms _{$\tau=5$}	1.13 ms ($9.21 \times / 2.34 \times$)	37.3 ms	8.71 ms _{$\tau=5$}	4.22 ms ($9.96 \times / 2.27 \times$)	—	—	—
	SOR (1.5)	12.2 ms	3.37 ms _{$\tau=5$}	1.02 ms ($10.3 \times / 3.22 \times$)	40.7 ms	9.06 ms _{$\tau=5$}	3.80 ms ($10.9 \times / 2.33 \times$)	182.7 ms	74.4 ms _{$\tau=5$}	60.7 ms ($3.10 \times / 1.24 \times$)
	Multigrid	3.83 ms	3.81 ms _{$\tau=5$}	2.05 ms ($1.96 \times / 1.88 \times$)	14.7 ms	14.6 ms _{$\tau=5$}	5.97 ms ($2.46 \times / 2.39 \times$)	92.0 ms	90.5 ms _{$\tau=10$}	38.7 ms ($2.08 \times / 2.10 \times$)
	Multigrid alone (no corrector)	—	—	4.39 ms	—	—	14.1 ms	—	—	81.9 ms
ConvDiff	PCG (MG) (no corrector)	—	—	4.35 ms	—	—	15.2 ms	—	—	85.6 ms
	Jacobi	3.44 ms	1.28 ms _{$\tau=5$}	1.14 ms ($2.95 \times / 1.34 \times$)	9.37 ms	4.95 ms _{$\tau=10$}	4.76 ms ($1.92 \times / 1.30 \times$)	158.0 ms	147.4 ms _{$\tau=50$}	144.3 ms ($1.17 \times / 1.14 \times$)
	Jacobi (0.67)	3.40 ms	1.31 ms _{$\tau=5$}	1.10 ms ($3.14 \times / 1.32 \times$)	9.31 ms	5.68 ms _{$\tau=10$}	5.76 ms ($1.80 \times / 1.25 \times$)	—	—	—
	GS	8.27 ms	3.60 ms _{$\tau=5$}	1.35 ms ($6.76 \times / 2.60 \times$)	31.0 ms	10.5 ms _{$\tau=5$}	7.05 ms ($5.52 \times / 1.53 \times$)	214.1 ms	192.7 ms _{$\tau=10$}	179.9 ms ($1.05 \times / 1.09 \times$)
	SymGS	11.1 ms	2.96 ms _{$\tau=5$}	1.28 ms ($8.70 \times / 2.40 \times$)	42.3 ms	10.9 ms _{$\tau=5$}	6.96 ms ($6.26 \times / 1.69 \times$)	—	—	—
	SOR (1.5)	13.6 ms	3.64 ms _{$\tau=5$}	1.59 ms ($8.89 \times / 2.40 \times$)	46.1 ms	10.6 ms _{$\tau=5$}	5.80 ms ($7.99 \times / 1.96 \times$)	206.6 ms	89.3 ms _{$\tau=5$}	83.6 ms ($2.48 \times / 1.18 \times$)
	Multigrid	4.68 ms	4.66 ms _{$\tau=5$}	2.32 ms ($2.07 \times / 2.01 \times$)	16.7 ms	16.5 ms _{$\tau=5$}	7.26 ms ($2.19 \times / 2.20 \times$)	97.6 ms	92.5 ms _{$\tau=10$}	53.7 ms ($1.83 \times / 1.75 \times$)
AnisoDiff	Multigrid alone (no corrector)	—	—	4.52 ms	—	—	18.8 ms	—	—	99.7 ms
	BiCGSTAB (MG) (no corrector)	—	—	6.17 ms	—	—	24.0 ms	—	—	93.5 ms
	Jacobi	2.90 ms	1.15 ms _{$\tau=5$}	911 μs ($3.16 \times / 1.25 \times$)	14.9 ms	10.3 ms _{$\tau=5$}	9.55 ms ($1.58 \times / 1.11 \times$)	—	—	—
	Jacobi (0.67)	2.85 ms	1.18 ms _{$\tau=5$}	850 μs ($3.31 \times / 1.34 \times$)	—	—	—	—	—	—
	GS	6.48 ms	1.74 ms _{$\tau=5$}	852 μs ($7.61 \times / 1.99 \times$)	29.7 ms	12.8 ms _{$\tau=5$}	9.30 ms ($3.18 \times / 1.37 \times$)	—	—	—
	SymGS	10.3 ms	2.40 ms _{$\tau=5$}	871 μs ($11.8 \times / 2.72 \times$)	—	—	—	—	—	—
	SOR (1.5)	6.62 ms	1.80 ms _{$\tau=5$}	869 μs ($7.36 \times / 2.04 \times$)	29.7 ms	12.8 ms _{$\tau=5$}	9.93 ms ($2.98 \times / 1.38 \times$)	—	—	—

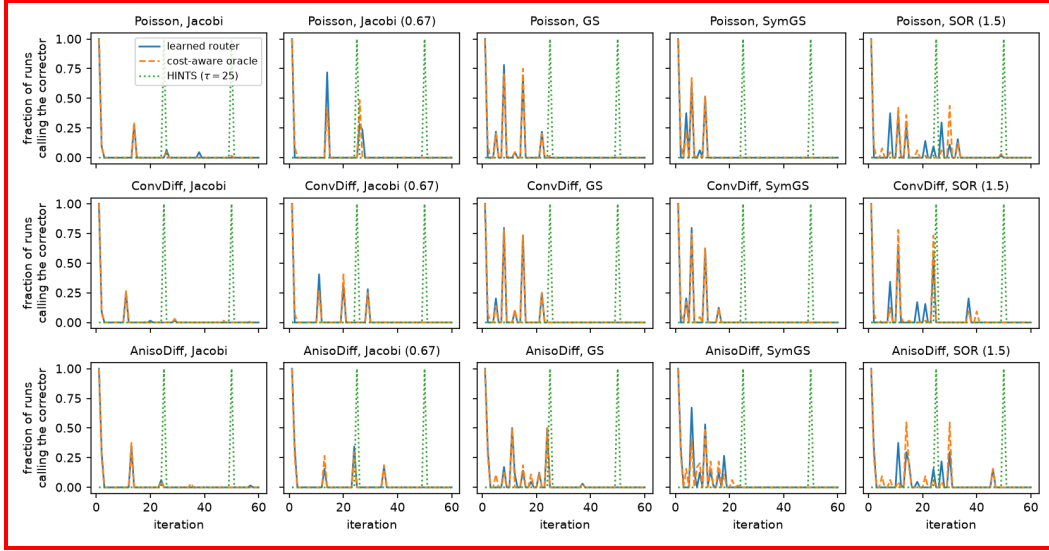


Figure 3: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every solver pairing (columns) and all three equations (rows); the same figure for the 256^2 and 512^2 grids is in Section E.

(it learns to ignore members that never win a cost-aware comparison), and it is strictly faster—by $0.89 \times$ – $1.00 \times$ over the best single solver, $p < 0.01$ —exactly where the error must pass through regimes that favour different solvers, namely at tolerances far below h^2 and on larger grids, where the router over $\{\text{Jacobi, Jacobi (0.67)}\}$ learns a nonstationary damping schedule that no single member can reproduce; at truncation level a single member suffices and the ensemble router matches it. We also verify numerically that the assumptions of Sections 4 and 5 hold in these settings, in the energy norm for the non-normal sweeps (Section E.7).

Additional experiments are provided in Section D.

Table 3: Final error and AUC of squared L^2 error for ensembles of increasing size. $\mathcal{W}$ denotes the set of numerical solvers and NO the neural operator (DeepONet). Each Router($\text{NO} \cup \mathcal{W}$) is compared against the best pairwise routing using our method, i.e., Router($\text{NO} \cup \{\text{solver}\}$) for solver $\in \mathcal{W}$. Values are mean ($\pm$ s.e.) over 128 test instances; Error is reported in $\times 10^{-3}$. p -values from paired one-sided t -tests comparing the AUC of the ensemble to that of the best pairwise router.

$\mathcal{W}$	Method	Poisson			ConvDiff		
		$\ e_h^{(T)}\ \times 10^3$	AUC	p -value (vs pairwise)	$\ e_h^{(T)}\ \times 10^3$	AUC	p -value (vs pairwise)
{Jacobi, GS}	Best Router($\text{NO} \cup \{\text{solver}\}$)	0.002 (0.004)	0.083 (0.152)	-	$< 10^{-3}$	0.031 (0.078)	-
	Router($\text{NO} \cup \mathcal{W}$)	0.002 (0.002)	0.078 (0.132)	$< 10^{-3}$	$< 10^{-3}$	0.021 (0.040)	0.004
{Jacobi, GS, SymGS}	Best Router($\text{NO} \cup \{\text{solver}\}$)	$< 10^{-3}$	0.052 (0.102)	-	$< 10^{-3}$	0.022 (0.039)	-
	Router($\text{NO} \cup \mathcal{W}$)	$< 10^{-3}$	0.041 (0.091)	$< 10^{-3}$	$< 10^{-3}$	0.018 (0.035)	$< 10^{-3}$
{Jacobi, GS, SymGS, Jacobi (0.67)}	Best Router($\text{NO} \cup \{\text{solver}\}$)	$< 10^{-3}$	0.052 (0.102)	-	$< 10^{-3}$	0.022 (0.039)	-
	Router($\text{NO} \cup \mathcal{W}$)	0.002 (0.003)	0.046 (0.083)	0.004	$< 10^{-3}$	0.019 (0.037)	0.002
{Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	Best Router($\text{NO} \cup \{\text{solver}\}$)	$< 10^{-3}$	0.052 (0.143)	-	$< 10^{-3}$	0.011 (0.030)	-
	Router($\text{NO} \cup \mathcal{W}$)	$< 10^{-3}$	0.043 (0.082)	0.03	$< 10^{-3}$	0.008 (0.019)	0.004

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is trained without accounting for its downstream role in correction term prediction; jointly training the ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; **Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—already delivers uniform wall-clock gains, and RL could extend it to non-myopic budgeted routing.** RL would additionally enable extending the ensemble routing to *continuous* parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

- Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.
- Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a review. *Monthly weather review*, 119(9):2206–2223, 1991.
- John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.
- Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*, volume 112. American Mathematical Soc., 2010.
- Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal processing magazine*, 18(6):14–30, 2001.
- Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33, 2008.
- Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*. Oxford university press, 1985.
- Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*. SIAM, 2007.
- Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*. Courier Corporation, 2003.
- Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.
- Susanne C Brenner. *The mathematical theory of fnite element methods*. Springer, 2008.

446 Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.

447 Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew
448 Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with
449 applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

450 Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks
451 with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on*
452 *neural networks*, 6(4):911–917, 1995.

453 Lizuo Liu and Wei Cai. Multiscale deepnet for nonlinear operators in oscillatory function spaces for
454 building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.

455 Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator
456 learning. *Journal of Computational Physics*, 506:112944, 2024.

457 Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for
458 oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.

459 Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating
460 spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint*
461 *arXiv:2503.13695*, 2025.

462 Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of
463 deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

464 Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua
465 Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference*
466 *on machine learning*, pages 5301–5310. PMLR, 2019.

467 Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle:
468 Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

469 Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for
470 general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

471 Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and
472 George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical
473 solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

474 John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

475 Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods:*
476 *Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

477 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew
478 Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations.
479 *arXiv preprint arXiv:2010.08895*, 2020a.

480 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik
481 Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial
482 differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

483 Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning
484 nonlinear operators via deepnet based on the universal approximation theorem of operators.
485 *Nature machine intelligence*, 3(3):218–229, 2021.

486 George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for
487 maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

488 Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions.
489 In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*
490 *(APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

491 Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular
492 minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

493 Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection,
494 sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3):
495 1–34, 2018.

496 Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions.
497 *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

498 Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschiatschek. Guarantees for
499 greedy maximization of non-submodular functions with applications. In *International conference*
500 *on machine learning*, pages 498–507. PMLR, 2017.

501 Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond
502 non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on*
503 *Machine Learning*, pages 2634–2643. PMLR, 2019.

504 Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions.
505 *Advances in Neural Information Processing Systems*, 21, 2008.

506 Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions
507 and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

508 Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions
509 with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

510 Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity.
511 In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30,
512 pages 38–47, 2020.

513 Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for
514 the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint*
515 *arXiv:2409.05020*, 2024.

516 Sebastian Tschiatschek, Adish Singla, and Andreas Krause. Selecting sequences of items via
517 submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
518 volume 31, 2017.

519 Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks
520 by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

521 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
522 neural networks. *Neural computation*, 1(2):270–280, 1989.

523 Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C
524 Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks.
525 *Advances in neural information processing systems*, 29, 2016.

526 Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence
527 prediction with recurrent neural networks. *Advances in neural information processing systems*, 28,
528 2015.

529 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Regression with multi-expert deferral. In *International*
530 *Conference on Machine Learning*, pages 34738–34759. PMLR, 2024.

531 Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic
532 multigrid coarsening using reinforcement learning. *Advances in neural information processing*
533 *systems*, 34:12129–12140, 2021.

534 Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid
535 methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*,
536 167:217–231, 2024.

537 Alena Kopaničáková and George Em Karniadakis. Deeponet based preconditioning strategies for
538 solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1):
539 C151–C181, 2025.

540 Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks.
541 *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

542 Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters.
543 *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

544 Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay
545 Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential
546 equations. *Computational Mechanics*, 72(3):471–484, 2023.

547 Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product.
548 *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

549 Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical
550 examples. *Mathematics of Computation*, 2025.

551 Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic
552 systems. *Mathematics*, 10(21):4014, 2022.

553 Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson,
554 and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint*
555 *arXiv:2309.15789*, 2023.

556 Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt
557 Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing
558 system. *arXiv preprint arXiv:2403.12031*, 2024.

559 Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS
560 Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query
561 routing. *arXiv preprint arXiv:2404.14618*, 2024.

562 Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and
563 Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level
564 scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

565 Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki,
566 Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint*
567 *arXiv:2410.13284*, 2024.

568 Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui,
569 Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based
570 on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*,
571 2025.

572 Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to
573 large language models. *arXiv preprint arXiv:2308.11601*, 2023.

574 Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to
575 large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

576 Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language
577 model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on*
578 *Web Search and Data Mining*, pages 606–615, 2024.

579 Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of
580 local experts. *Neural computation*, 3(1):79–87, 1991.

581 Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural*
582 *computation*, 6(2):181–214, 1994.

583 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and
584 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv*
585 *preprint arXiv:1701.06538*, 2017.

586 Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang,
587 Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional
588 computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

589 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
590 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
591 2022.

592 Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris
593 Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al.
594 Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

595 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and
596 applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.

597 Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In
598 *Backpropagation*, pages 137–169. Psychology Press, 2013.

599 Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*,
600 volume 11. University of Cambridge Department of Engineering Cambridge, 1987.

601 Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model.
602 *Neural networks*, 1(4):339–356, 1988.

603 Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and
604 George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with prac-
605 tical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*,
606 393:114778, 2022.

607 Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured
608 prediction to no-regret online learning. In *Proceedings of the Fourteenth International Confer-*
609 *ence on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference
610 Proceedings, 2011.

611 A Proofs for Section 4

612 A.1 Proof of Proposition A.1

613 **Proposition A.1.** Any prefix monotonically non-increasing sequence supermodular function g is
614 weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$

615 *Proof.* This proof is adapted from Liberty and Sviridenko [2017].

616 If g is sequence supermodular then,

$$\begin{aligned}
& g(S) - g(S \oplus S') \\
&= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) - g(S \oplus (S'_1, \dots, S'_i)) \\
&\stackrel{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\
&\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i)
\end{aligned}$$

617 (a) by supermodularity

618

□

619 A.2 Proof of Theorem 4.1

620 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal*
 621 *solution $O = \arg \min_{S \in \Omega^T} h(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let*
 622 *the greedy solution of length T be S^T . If $\phi_T(\alpha) = \left(1 - \frac{1}{\alpha T}\right)^T$, then*

$$g(S^T) \leq (1 - \phi_T(\alpha(O)))g(O) + \phi_T(\alpha(O))g(\emptyset)$$

623 *Proof.* This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko
 624 [2017]

$$\begin{aligned} g(S^t) - g(O) &\stackrel{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\ &= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \dots, o_{i-1})) - g(S^t \oplus (o_1, \dots, o_i)) \\ &\stackrel{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\ &\leq \alpha(O)|O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\ &\leq \alpha(O)Tg(S^t) - \alpha(O)T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\ &= \alpha(O)Tg(S^t) - \alpha(O)Tg(S^{t+1}) \end{aligned}$$

625 (a) by μ -postfix monotonicity, (b) by supermodularity.

626 After rearranging the inequality, we get:

$$\begin{aligned} g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} (g(O) - (\alpha(O)T - 1)g(S^t)) \\ &= \frac{1}{\alpha(O)T} g(O) + \left(1 - \frac{1}{\alpha(O)T}\right) g(S^t) \end{aligned}$$

627 When recursively applying this inequality, we get:

$$\begin{aligned} g(S^T) &\leq \frac{1}{\alpha(O)T} g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\ &= \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right) g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \end{aligned}$$

628 □

629 A.3 Proof of Proposition 4.2

630 **Proposition 4.2.** *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is*
 631 *ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is*
 632 *weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

633 Furthermore, if $I - C_j \circ \mathcal{L}_h^a$ is invertible for all $j \in [K]$, h is also postfix monotonically non-increasing.

634 *Proof.* For brevity, we use the notation $(g_1 \circ \dots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is
 635 applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz
 636 continuous functions:

- 637 • **Property 1:** If g is ρ -Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 =$
 638 $\|g(x) - g(0)\|_2 \leq \rho\|x - 0\|_2 = \rho\|x\|_2$
- 639 • **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
 640 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is $\rho_1 + \rho_2$.
- 641 • **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
 642 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is $\rho_1 \rho_2$.

643 In order to prove weakly- α -supermodularity, we must first prove prefix monotonicity.

644 **Prefix monotonicity:** Let $S \preceq S'$ where $S' = S \oplus N$.

$$\begin{aligned}
 h(S') &= \left\| \circ_{t=1}^{|S'|} (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &= \left\| \circ_{t=|S|+1}^{|S'|+1} (I - C_{S'_t} \circ \mathcal{L}_h^a) \circ \circ_{t=1}^{|S|} (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\stackrel{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\leq h(S)
 \end{aligned}$$

645 (a) by Property 1

646 **Weak supermodularity:** We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) -$

647 $h(S \oplus O)$ and a lower bound for $\sum_{i=1}^T h(S) - h(S \oplus O_i)$.

$$\begin{aligned}
 h(S) - h(S \oplus O) &= \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| \circ_{t=1}^{|O|} (I - C_{O_t} \circ \mathcal{L}_h^a) \circ \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\stackrel{(a)}{\leq} \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) - \circ_{t=1}^{|O|} (I - C_{O_t} \circ \mathcal{L}_h^a) \circ \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &= \left\| \left(I - \circ_{t=1}^{|O|} (I - C_{O_t} \circ \mathcal{L}_h^a) \right) \circ \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\stackrel{(b)}{\leq} \left(1 + \prod_{t=1}^{|O|} \rho_{O_t} \right)^2 \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\stackrel{(c)}{\leq} 4 \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2
 \end{aligned}$$

648 (a) by reverse triangle property and $h(S) - h(S \oplus S') > 0$ by prefix monotonicity, (b) since the

649 Lipschitz constant of $I - \circ_{t=1}^{|S'|} (I - C_{S'_t} \circ \mathcal{L}_h^a)$ is $1 + \prod_{t=1}^{|S'|} \rho_{S'_t}$ by Property 2 and 3, (c) since

650 $\rho_j < 1$

651 To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$\begin{aligned}
 \sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| (I - C_{O_i} \circ \mathcal{L}_h^a) \circ \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &\geq \sum_{i=1}^{|O|} \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=1}^{|S|} (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
 &= \left\| \circ_{t=1}^{|S|} (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right)
 \end{aligned}$$

652 Finally,

$$\begin{aligned} \frac{h(S) - h(S \oplus O)}{T \max_i h(S) - h(S \oplus O_i)} &\leq \max \left\{ \frac{4 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2}{\left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 (T - \sum_{i=1}^T \rho_{O_i}^2)}, 1 \right\} \\ &= \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\} \end{aligned}$$

653 **Postfix monotonicity:** Let $S' = S \oplus N$.

$$\begin{aligned} h(S') &= \left\| \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h) e_h^{(0)} \right\|_2^2 \\ &= \left\| \circ_{t=|N|}^1 (I - C_{N_t} \circ \mathcal{L}_h) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h) e_h^{(0)} \right\|_2^2 \\ &\stackrel{(a)}{=} \left\| \circ_{t=|N|}^1 (I - C_{N_t} \circ \mathcal{L}_h) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h) \circ \left(\circ_{t=|N|}^1 (I - C_{N_t} \circ \mathcal{L}_h) \right)^{-1} \circ \circ_{t=|N|}^1 (I - C_{N_t} \circ \mathcal{L}_h) e_h^{(0)} \right\|_2^2 \\ &\leq \prod_{t=1}^{|N|} \rho_{N_t}^2 \prod_{t=1}^{|S|} \rho_{S_t}^2 \prod_{t=1}^{|N|} \rho_{N_t}^{-2} \left\| \circ_{t=|N|}^1 (I - C_{N_t} \circ \mathcal{L}_h) e_h^{(0)} \right\|_2^2 \\ &\leq h(N) \end{aligned}$$

654 (a) due the invertibility of $I_N - C_j \mathcal{L}_h$

655

□

656 A.4 Proof of Theorem A.2

657 **Lemma A.2.** Let $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ where P is an orthogonal matrix and $\Lambda_j =$
 658 $\text{diag}(\lambda_{j1}, \dots, \lambda_{jN})$. If $P^{-1} e_h^{(0)} = z$, then the following equality holds:

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \quad (11)$$

659 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned}
h(S) &= \left\| \prod_{t=|S|}^1 (I_N - C_{S_t} \mathcal{L}_h^a) e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^1 (P \Lambda_{S_t} P^{-1}) e_h^{(0)} \right\|^2 \\
&= \left\| P \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\
&= \sum_{i=1}^N z_i^2 \prod_{t=T}^1 \lambda_{S_t i}^2 \\
&= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

660

□

661 A.5 Proof of Theorem 4.3

662 **Proposition 4.3.** *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$. Then, h is*
663 *supermodular.*

664 *Proof.* Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}$$

665 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j . Recall that h is
666 considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

667

$$\begin{aligned}
h(S) - h(S \oplus \omega) &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^N z_i^2 \lambda_{\omega i}^2 \prod_{k=1}^K \lambda_{ki}^{2m_k(S)} \\
&= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

668 Similarly,

$$\begin{aligned}
h(S') - h(S \oplus \omega) &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S')} \\
&\stackrel{(a)}{=} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2(m_j(S) + m_j(B))} \\
&\stackrel{(b)}{\leq} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\
&= h(S) - h(S \oplus \omega)
\end{aligned}$$

669 (a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|+1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=k} + \sum_{t=1}^{|B|} \mathbf{1}_{B_t=j} =$
670 $m_j(S) + m_j(B)$, (b) since $\rho(I_N - C_j \mathcal{L}_h^a) < 1$ and $m_j(B) \geq 0$ for all $j \in [K]$
671 □

672 B Proofs for Section 5

673 B.1 Proof of Theorem B.1

674 **Lemma B.1.** *For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r , any*
675 $a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, *the following equality holds true:*

$$\begin{aligned} l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \end{aligned}$$

676 *Proof.* Note that $\sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K-1$

$$\begin{aligned} l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad + (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad + \sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\ &\quad - (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \end{aligned}$$

677

$$\begin{aligned} &= \sum_{j=1}^K \left(\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - (K-2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\ &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - (K-2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \end{aligned}$$

679 B.2 Proof of Theorem B.2

680 **Proposition B.2.** For any router r defined by $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$, any $a_h \in \mathcal{A}$,
 681 $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:

$$\log(2)l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)$$

682 *Proof.* By Theorem B.1, we know that

$$\begin{aligned} \log(2)l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) &= \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - \log(2)(K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \\ &\leq \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\stackrel{(a)}{\leq} - \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \\ &= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) \end{aligned}$$

683 (a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} < 0.5$ which implies that

684 $-\log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$

686 B.3 Proof of Theorem 5.1

687 **Theorem 5.1.** Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that
 688 $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete
 689 operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .

690 *Proof.* For a given a_h, f_h , let u_h be $\mathcal{G}_h(a_h, f_h)$ where $\mathcal{G}_h$ denotes the solution operator acting on the
 691 grid G_h . Furthermore, let's consider routers of the form

$$r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$$

692 For a given $a_h, f_h, u_h^{(t)} \in \mathcal{A} \times \mathcal{F} \times \mathcal{U}$, let the optimal loss under l_{route} be
 693 $l_{\text{route}}^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{r}} l_{\text{route}}(\tilde{r}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Similarly, let the optimal loss un-
 694 der Ψ be $\Psi^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{\mathbf{g}}} \Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Let $B_j(a_h, f_h, u_h^{(t)}) =$
 695 $\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a)(\mathcal{G}_h(a_h, f_h) - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j}$.

$$\begin{aligned}
& l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \\
& \stackrel{(a)}{=} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
& \quad - \inf_{\tilde{r}} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} + (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
& = \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \\
& = \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right)
\end{aligned}$$

696 (a) by Theorem B.1

697 Let $\mathcal{X} = \mathcal{A} \times \mathcal{F} \times \mathcal{U}$ and $\mathcal{Y} = [K]$. Let $\mathcal{P}_{\mathcal{X}}$ denote the degenerate distribution supported at the
698 point $(a_h, f_h, u_h^{(t)})$. We define the conditional distribution - $P(Y = j \mid X = (a_h, f_h, u_h^{(t)})) =$
699 $\frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}$ for $j \in [K]$. The risk and optimal risk of 0 - 1 loss under this distribution can
700 be written as:

$$\begin{aligned}
\mathcal{R}_{0-1}(r) &= \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
\mathcal{R}_{0-1}^* &= \inf_r \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}
\end{aligned}$$

701 If $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [K]} g_j(a_h, f_h, u_h^{(t)})$ for all $x \in \mathcal{X}$, then the he risk and optimal risk of
702 cross entropy loss $(l_{ce}(\mathbf{g}, x, y) - \log \left(\frac{\exp(g_y(x))}{\sum_{k=1}^K \exp(g_k(x))} \right))$ under this distribution can be written as:

$$\begin{aligned}
\mathcal{R}_{ce}(\mathbf{g}) &= - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \\
\mathcal{R}_{ce}^* &= \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right)
\end{aligned}$$

703 From Theorem 3.1 of [Mao et al., 2023], $\mathcal{R}_{0-1}(r) - \mathcal{R}_{0-1}^* \leq \Gamma^{-1}(\mathcal{R}_{ce}(\mathbf{g}) - \mathcal{R}_{ce}^*)$ if
704 $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [K]} g_j(a_h, f_h, u_h^{(t)})$ where $\Gamma(z) = \frac{1+z}{2} \log(1+z) + \frac{1-z}{2} \log(1-z)$.
705 Then,

$$\begin{aligned}
& l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right) \\
&\leq \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(a)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(b)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&= \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right)
\end{aligned}$$

706 (a) since $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 < \bar{E}$ for all $j \in [K]$, (b) since Γ^{-1} is non-decreasing and

707 $\exists j \in [K]$ such that $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 > E_{\min}$

708 Finally,

$$\begin{aligned}
& \lim_{n \rightarrow \infty} \mathcal{R}_{\text{route}}(r_n) - \mathcal{R}_{\text{route}}^* \\
&\stackrel{(a)}{=} \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}} \left(r_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right] \\
&\leq \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right) \right] \\
&\stackrel{(b)}{\leq} \lim_{n \rightarrow \infty} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\Psi(\mathbf{g}_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)}) \right]}{(K-1)E_{\min}} \right)
\end{aligned}$$

$$\begin{aligned}
&= \lim_{n \rightarrow \infty} \bar{E}K(K-1)\Gamma^{-1} \left(\frac{\mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K-1)E_{min}} \right) \\
&\stackrel{(c)}{=} \bar{E}K(K-1)\Gamma^{-1} \left(\frac{\lim_{n \rightarrow \infty} \mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K-1)E_{min}} \right) \\
&= \bar{E}K(K-1)\Gamma^{-1}(0) \\
&\stackrel{(d)}{=} 0
\end{aligned}$$

710 (a) $\mathcal{R}_{\text{route}}^* = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right]$ since the infimum is taken over all measurable
711 functions, (b) by Jensen's inequality since Γ^{-1} is concave, (c) by continuity of Γ^{-1} at 0, (d) $\Gamma^{-1}(0) =$
712 0

713

□

714 C Training Details

715 Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random
716 Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ are sampled
717 as:

$$\begin{aligned}
\alpha &\sim \text{Log-Uniform}(0.01, 100) \\
\beta &\sim \text{Log-Uniform}(0.1, 1000) \\
\gamma &\sim \text{Uniform}(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\})
\end{aligned}$$

718 Given (α, β, γ) , samples are generated in the Fourier space. For each non-zero frequency mode
719 k , we draw an independent complex coefficient from a Gaussian distribution with mean 0 and
720 variance $\alpha(4\pi^2\|k\|_2^2 + \beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the
721 zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier
722 Transform to obtain the field in physical space.

723 For each sample, we compute reference solutions with a least squares solver and treat them as ground
724 truth.

725 This data is used to trained our DeepONet models and LSTM routers. All the models were imple-
726 mented using PyTorch and all the models were trained on one Nvidia A40 GPU.

727 Table 4 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train.
728 We then use the model with the best validation loss.

Table 4: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

729 The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$,
730 we also supply the routers with the iteration index t , current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing

731 decisions from the previous iteration. While the iteration index and residual are deterministic
732 functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous
733 routing decisions is a mild deviation from the idealized setting. Nevertheless, this modification is
734 motivated by the same considerations that justify the use of scheduled sampling: under deployment,
735 routing decisions are predicted autoregressively and influence future inputs, whereas training under
736 teacher forcing assumes independence from past predictions. Including previous routing decisions
737 bridges the gap induced by exposure bias by providing the model with trajectory-level information.

738 All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-
739 forcing probability $p_{tf}(e) = ss_{start}$. After the warm-up, the p_{tf} decays geometrically by a factor of
740 $\gamma_{tf} < 1$ per epoch and is floored by ss_{end} :

$$p_{tf}(e) = \begin{cases} ss_{start} & e \leq e_w \\ \max(ss_{start}\gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

741 At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we
742 feed the router’s own predicted iterate.

743 Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated
744 backpropagation through time (TBPTT) [Mozer, 2013, Robinson and Fallside, 1987, Werbos, 1988]:
745 the forward pass unrolls the entire trajectory, but gradients are propagated only through the most
746 recent $w_{bptt}(e)$ steps at epoch e . Hidden states are passed forward between segments, while earlier
747 segments are treated as stop-gradient.

748 We employ a curriculum learning approach analogous to scheduled sampling. Let T_{max} be the
749 horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{bptt}(e) = \begin{cases} w_{start} & e \leq e_w \\ \min\left(T_{max}, w_{start}\gamma_{bptt}^{\lfloor \frac{e-e_w}{f_{bptt}} \rfloor}\right) & e > e_w \end{cases} \quad (12)$$

750 so the window grows geometrically by a factor of $\gamma_{bptt} > 1$ every f_{bptt} epochs and is capped at the
751 full trajectory length.

752 Table 5 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver
753 experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver
754 Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best
755 validation loss for testing. The architecture and dataset size is larger for the Solver ensemble
756 experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 5: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95	0.95
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

We present additional visualizations and tables that further highlight the strengths of our method. In the paired solver experiments, we include Damped Jacobi with $\omega = 0.67$ and Successive Over-Relaxation (SOR) with $\omega = 1.5$. We also compare against a True Greedy oracle, which has access to true error at each step (See Section 4). This comparison demonstrates that the learned router closely approximates the behavior of the greedy policy.

D.1 Error Comparisons with Paired t -Test Results

We supplement the paired-solver experiments in Table 1 with additional solver families (Jacobi (0.67) and SOR (1.5)) and statistical significance analysis. For each solver family, we compare the learned greedy router against baseline methods (single-solver only and HINTS) using paired t -tests with a one-sided alternative hypothesis that the learned router achieves lower error or AUC. Results are reported in Table 6.

Across all five solver families, the resulting p -values indicate statistically significant improvements of the learned greedy router over both single-solver and HINTS baselines.

We support the Large Solver ensemble experiments in Table 3 with a statistical significance analysis. We use paired t -tests across the 128 test instances. For each baseline solver j (e.g., Jacobi, GS, SymGS, etc.), we compare the performance of the solver ensemble $\mathcal{W}$ against the corresponding pairwise learned router $\text{Router}(\text{NO} \cup \{j\})$ if $j \in \mathcal{W}$. The test is performed on per-instance differences in final error (and AUC), using a one-sided alternative hypothesis that the solver ensemble achieves lower error than the baseline.

Across both Poisson and convection–diffusion problems, the solver ensembles yield statistically significant improvements over all pairwise baselines in nearly every setting, with p -values typically well below 0.01. The gains are enhanced as the ensemble size increases, while the performance of the learned router remains close to that of the true greedy oracle. These results support the claim that enlarging the solver set provides measurable benefits and that our learned policy effectively captures the resulting improvements. Results are reported in Table 7.

Table 6: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value
Jacobi-related solvers								
Jacobi Only	0.383 (1.029)	$< 10^{-3}$	0.821 (2.154)	$< 10^{-3}$	0.136 (0.364)	$< 10^{-3}$	0.312 (0.788)	$< 10^{-3}$
HINTS-Jacobi	0.759 (0.016)	$< 10^{-3}$	0.393 (0.590)	$< 10^{-3}$	0.447 (0.013)	$< 10^{-3}$	0.202 (0.256)	$< 10^{-3}$
Learned Greedy-Jacobi	0.054 (0.142)	-	0.165 (0.368)	-	0.033 (0.097)	-	0.098 (0.239)	-
True-Greedy-Jacobi	0.021 (0.018)	-	0.094 (0.170)	-	0.012 (0.012)	-	0.049 (0.091)	-
GS-related solvers								
GS only	0.019 (0.051)	$< 10^{-3}$	0.435 (1.141)	$< 10^{-3}$	$< 10^{-3}$	0.006	0.088 (0.219)	$< 10^{-3}$
HINTS-GS	0.724 (0.000)	$< 10^{-3}$	0.325 (0.488)	$< 10^{-3}$	0.456 (0.000)	$< 10^{-3}$	0.130 (0.154)	0.0
Learned Greedy-GS	0.002 (0.004)	-	0.083 (0.152)	-	$< 10^{-3}$	-	0.031 (0.078)	-
True-Greedy-GS	0.001 (0.001)	-	0.057 (0.107)	-	$< 10^{-3}$	-	0.019 (0.038)	-
SymGS-related solvers								
SymGS only	$< 10^{-3}$	$< 10^{-3}$	0.227 (0.593)	$< 10^{-3}$	$< 10^{-3}$	0.028	0.071 (0.178)	$< 10^{-3}$
HINTS-SymGS	0.709 (0.000)	$< 10^{-3}$	0.252 (0.380)	$< 10^{-3}$	0.463 (0.000)	$< 10^{-3}$	0.116 (0.131)	$< 10^{-3}$
Learned Greedy-SymGS	$< 10^{-3}$	-	0.052 (0.102)	-	$< 10^{-3}$	-	0.022 (0.039)	-
True-Greedy-SymGS	$< 10^{-3}$	-	0.034 (0.066)	-	$< 10^{-3}$	-	0.016 (0.032)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.066 (2.861)	$< 10^{-3}$	1.128 (2.954)	$< 10^{-3}$	0.349 (0.931)	$< 10^{-3}$	0.428 (1.077)	$< 10^{-3}$
HINTS-Jacobi (0.67)	0.789 (0.000)	$< 10^{-3}$	0.429 (0.642)	0.005	0.473 (0.000)	$< 10^{-3}$	0.223 (0.284)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.232 (0.654)	-	0.309 (0.791)	-	0.069 (0.102)	-	0.117 (0.210)	-
True-Greedy-Jacobi (0.67)	0.057 (0.041)	-	0.131 (0.233)	-	0.026 (0.021)	-	0.065 (0.117)	-
SOR (1.5)-related solvers								
SOR (1.5) only	$< 10^{-3}$	0.057	0.156 (0.409)	$< 10^{-3}$	$< 10^{-3}$	0.086	0.008 (0.020)	0.988
HINTS-SOR (1.5)	0.700 (0.000)	$< 10^{-3}$	0.207 (0.316)	$< 10^{-3}$	0.462 (0.000)	$< 10^{-3}$	0.021 (0.020)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	$< 10^{-3}$	-	0.052 (0.143)	-	$< 10^{-3}$	-	0.011 (0.030)	-
True-Greedy-SOR (1.5)	$< 10^{-3}$	-	0.032 (0.071)	-	$< 10^{-3}$	-	0.007 (0.019)	-

Table 7: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each solver ensemble against the corresponding pairwise learned router (e.g., NO + Jacobi, NO + GS), using a one-sided alternative that the ensemble achieves lower error/AUC. Reported p -values correspond to these tests.

$\mathcal{W}$	$\ e_h^{(T)}\ \times 10^3$	AUC	Jacobi p -value	GS p -value	SymGS p -value	Jacobi (0.67) p -value	SOR (1.5) p -value
Poisson							
Learned Greedy {Jacobi, GS}	0.003 (0.002)	0.068 (0.131)	$< 10^{-3}$	$< 10^{-3}$	-	-	-
True Greedy {Jacobi, GS}	0.001 (0.001)	0.057 (0.107)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.038 (0.071)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.034 (0.066)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.042 (0.074)	$< 10^{-3}$	$< 10^{-3}$	0.004	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.034 (0.066)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.039 (0.076)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.04
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.031 (0.069)	-	-	-	-	-
ConvDiff							
Learned Greedy {Jacobi, GS}	$< 10^{-3}$	0.021 (0.040)	$< 10^{-3}$	0.005	-	-	-
True Greedy {Jacobi, GS}	$< 10^{-3}$	0.019 (0.038)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.020 (0.036)	$< 10^{-3}$	0.004	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.016 (0.032)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.020 (0.039)	$< 10^{-3}$	0.003	0.003	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.016 (0.032)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.008 (0.019)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.004
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.007 (0.018)	-	-	-	-	-

783 D.2 Convergence Histories

784 See Figure 4 for convergence histories for the Jacobi (0.67)-related solvers and Successive Over-
785 relaxation (1.5)-related solvers.

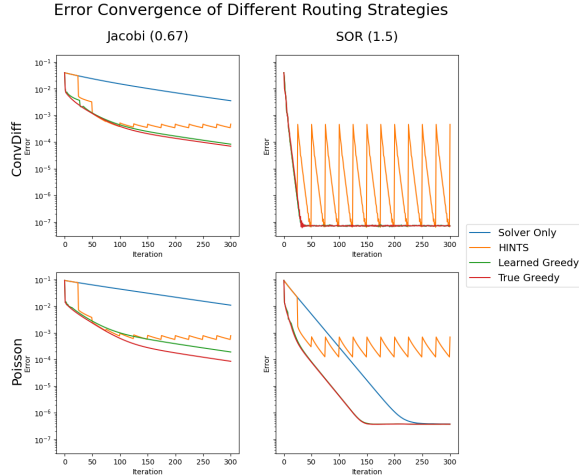


Figure 4: Convergence histories for representative test instances. Rows: ConvDiff (top) and Poisson (bottom). Columns: Jacobi (0.67) and Successive Over-relaxation (1.5). Greedy yields near-monotone decay and the lowest errors, whereas HINTS shows sawtooth behaviors.

786 D.3 Prediction and Error Visualizations

787 Figures 5 and 6 provide qualitative visualizations of sample predictions across routing strategies and
788 numerical solvers for both equations. While these prediction may appear identical to the ground
789 truth, Figures 7 and 8 reveals distinct error patterns with varying magnitudes across all predictions.
790 In particular, our learned method consistently achieves the lower-magnitude errors compared to its
791 respective baselines.

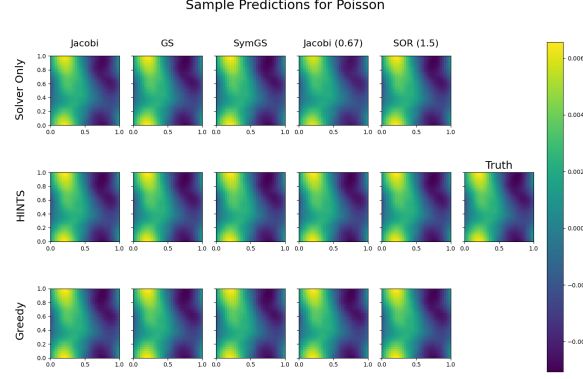


Figure 5: Sample predictions for the Poisson equation across different solver pairings. Each column corresponds to a numerical solver (Jacobi, Gauss–Seidel (GS), Symmetric GS, Damped Jacobi, and Successive Over-relaxation), while rows show predictions from the corresponding solver-only baseline (top), HINTS (middle), and the learned greedy router (bottom). The ground truth solution is shown on the right.

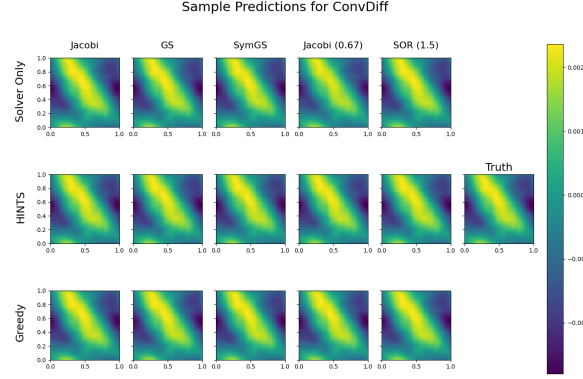


Figure 6: Sample predictions for the convection diffusion equation across different solver pairings. Each column corresponds to a numerical solver (Jacobi, Gauss–Seidel (GS), Symmetric GS, Damped Jacobi, and Successive Over-relaxation), while rows show predictions from the corresponding solver-only baseline (top), HINTS (middle), and the learned greedy router (bottom). The ground truth solution is shown on the right.

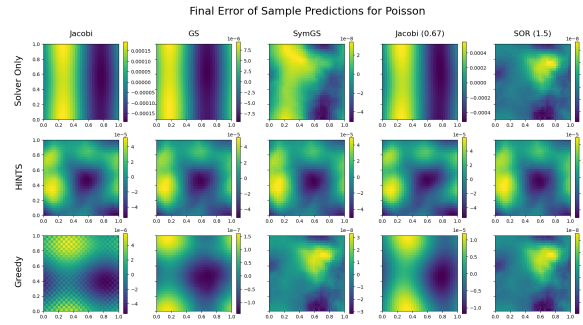


Figure 7: Pointwise error maps for Poisson sample predictions across solver families. Columns correspond to different numerical solvers, while rows show solver-only (top), HINTS (middle), and learned greedy router (bottom). Errors are computed as $u_h - u_h^{(T)} = e_h^{(T)}$. The learned greedy router consistently yields lower-magnitude errors compared to both solver-only and HINTS baselines.

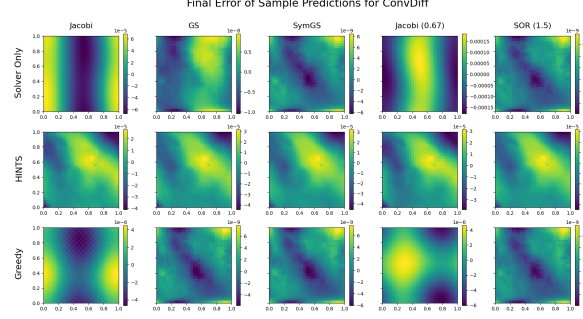


Figure 8: Pointwise error maps for convection diffusion sample predictions across solver families. Columns correspond to different numerical solvers, while rows show solver-only (top), HINTS (middle), and learned greedy router (bottom). Errors are computed as $u_h - u_h^{(T)} = e_h^{(T)}$. The learned greedy router consistently yields lower-magnitude errors compared to both solver-only and HINTS baselines.

792 D.4 Residual Comparison

Table 8: Final residual $\|\mathcal{L}_h^a e_h^{(T)}\|$ and AUC of residuals over iterations (lower is better). Values are reported as mean ($\pm$ standard error (s.e.)) over 128 test instances; residuals are scaled by $\times 10^{-3}$. If the residual norm is reported as $< 10^{-3}$, the corresponding raw value is $< 10^{-6}$. Statistical significance is evaluated via paired t -tests comparing each baseline (single-solver only and HINTS) to the learned greedy router under a one-sided alternative hypothesis that the learned router achieves lower residual and AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ \mathcal{L}_h^a e_h^{(T)}\ \times 10^3$	$\ \mathcal{L}_h^a e_h^{(T)}\ $ p-value	AUC	AUC p-value	$\ \mathcal{L}_h^a e_h^{(T)}\ \times 10^3$	$\ \mathcal{L}_h^a e_h^{(T)}\ $ p-value	AUC	AUC p-value
Jacobi-related solvers								
Jacobi only	23.939 (66.755)	0.021	43.475 (109.764)	$< 10^{-3}$	27.208 (74.555)	0.012	45.523 (114.774)	$< 10^{-3}$
HINTS-Jacobi	193.327 (37.362)	$< 10^{-3}$	39.763 (70.671)	$< 10^{-3}$	266.421 (34.699)	$< 10^{-3}$	42.851 (70.748)	$< 10^{-3}$
Learned Greedy-Jacobi	20.870 (62.246)	-	32.041 (83.647)	-	20.786 (66.511)	-	34.844 (84.666)	-
GS-related solvers								
GS only	0.761 (2.043)	$< 10^{-3}$	20.776 (52.270)	$< 10^{-3}$	0.002 (0.005)	$< 10^{-3}$	10.720 (26.892)	$< 10^{-3}$
HINTS-GS	183.359 (0.000)	$< 10^{-3}$	22.581 (27.984)	$< 10^{-3}$	246.877 (0.000)	$< 10^{-3}$	18.654 (19.201)	$< 10^{-3}$
Learned Greedy-GS	0.092 (0.145)	-	11.960 (27.863)	-	0.001 (0.003)	-	7.322 (16.968)	-
SymGS-related solvers								
SymGS only	0.004 (0.009)	$< 10^{-3}$	10.343 (25.957)	$< 10^{-3}$	0.002 (0.004)	0.004	8.604 (21.776)	$< 10^{-3}$
HINTS-SymGS	185.601 (0.001)	$< 10^{-3}$	15.489 (18.255)	$< 10^{-3}$	245.629 (0.000)	$< 10^{-3}$	16.515 (15.990)	$< 10^{-3}$
Learned Greedy-SymGS	0.002 (0.005)	-	7.149 (18.364)	-	0.001 (0.002)	-	6.481 (13.190)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	42.246 (112.968)	$< 10^{-3}$	55.250 (138.258)	$< 10^{-3}$	45.451 (121.431)	$< 10^{-3}$	56.747 (141.901)	$< 10^{-3}$
HINTS-Jacobi (0.67)	188.719 (0.008)	$< 10^{-3}$	38.970 (53.035)	$< 10^{-3}$	256.407 (0.005)	$< 10^{-3}$	39.604 (52.578)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	9.290 (26.109)	-	35.905 (86.881)	-	9.010 (13.426)	-	32.516 (83.488)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.003 (0.008)	0.5	9.044 (22.962)	$< 10^{-3}$	0.002 (0.004)	0.189	2.817 (7.499)	$< 10^{-3}$
HINTS-SOR (1.5)	188.187 (0.001)	$< 10^{-3}$	15.894 (19.565)	$< 10^{-3}$	248.226 (0.001)	$< 10^{-3}$	11.420 (7.498)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.003 (0.008)	-	7.784 (19.939)	-	0.002 (0.004)	-	5.530 (19.679)	-

Table 9: Final residual and AUC of squared L^2 residual for varying numbers of solvers. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. are reported in $\times 10^{-3}$.

Equation	Poisson		ConvDiff	
$\mathcal{W}$	$\ \mathcal{L}_h^a e_h^{(T)}\ $	AUC	$\ \mathcal{L}_h^a e_h^{(T)}\ $	AUC
{Jacobi, GS}	0.135 (0.096)	11.815 (33.768)	0.023 (0.044)	5.765 (14.581)
{Jacobi, GS, SymGS}	0.002 (0.004)	5.118 (12.920)	0.002 (0.002)	4.810 (11.264)
{Jacobi, GS, SymGS, Jacobi (0.67)}	0.013 (0.011)	6.120 (14.651)	0.010 (0.013)	4.859 (11.473)
{Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.004 (0.006)	8.196 (22.516)	0.001 (0.002)	3.209 (8.676)

793 Table 8 summarizes the performance of single-solver schedules, HINTS, and greedy with respect
794 to the final residuals $r_h^{(T)} = \|f_h - \mathcal{L}_h^a u_h^{(T)}\|$ or $\|\mathcal{L}_h^a e_h^{(T)}\|$ and its AUC $AUC_T = \sum_{t=1}^T \|r_h^{(t)}\|_2^2$.
795 Greedy outperforms its HINTS and single-solver counterparts in most equations. We must note that
796 our greedy router is trained to reduce error, not residual. The same error can induce very different
797 residuals depending on the spectrum $\mathcal{L}_h^a$. Table 9 exhibits how residuals are affected by the number of
798 solvers in the solver ensemble. Similar to error, we observe both the final residual and AUC decrease
799 as the number of solvers increase.

800 D.5 Fourier mode-wise error comparison

801 We assess frequency-resolved performance by projecting the error onto the discrete Fourier basis.
 802 Table 10 report, for modes 1, 5, and 10, the mode-wise final error and mode-wise AUC, comparing
 803 single-solver baselines, HINTS, and the greedy router. As a result of including a deep learning
 804 model, Greedy consistently achieves the smallest mode-1 error/AUC across equations and solver
 805 families. For modes 5 and 10, single-solver schedules sometimes have an edge, reflecting the
 806 tendency of classical smoothers to damp high-frequency components more aggressively than ML
 807 surrogates (spectral bias). Overall, greedy delivers more uniform convergence across the spectrum:
 808 it routes to whichever solver most decreases the full L^2 error, and by Parseval’s identity $|e_h^{(t)}|_2^2 =$
 809 $\sum_m |\hat{u}_m^{(t)} - \hat{u}_m|^2$, reductions in the objective correspond to reducing energy across all modes rather
 810 than giving preferential treatment to a subset. Additionally, in Table 11, we observe that all mode-wise
 811 errors/AUCs reduce with the inclusion of more solvers.

Table 10: Final error and AUC of squared L^2 error for Mode 1, 5, and 10 (lower is better) for Poisson and ConvDiff. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of final errors are reported in $\times 10^{-3}$. If the residual norm is reported as $< 10^{-3}$, the corresponding raw value is $< 10^{-6}$.

Method	Mode 1		Poisson Mode 5		Mode 10		Mode 1		ConvDiff Mode 5		Mode 10	
	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC
Jacobi Only	0.135 (0.295)	3.231 (7.056)	$< 10^{-3}$	0.003 (0.010)	$< 10^{-3}$	0.000 (0.001)	0.079 (0.171)	1.084 (2.369)	$< 10^{-3}$	0.003 (0.011)	$< 10^{-3}$	0.000 (0.001)
HINTS-Jacobi	5.730 (0.240)	2.734 (3.371)	0.028 (0.002)	0.004 (0.012)	0.006 (0.000)	0.000 (0.001)	0.994 (0.220)	0.697 (1.091)	0.061 (0.001)	0.005 (0.012)	0.022 (0.001)	0.001 (0.001)
Learned Greedy-Jacobi	0.032 (0.085)	0.841 (2.081)	$< 10^{-3}$	0.009 (0.028)	$< 10^{-3}$	0.001 (0.003)	0.021 (0.055)	0.324 (0.780)	$< 10^{-3}$	0.009 (0.025)	$< 10^{-3}$	0.001 (0.002)
GS-related solvers												
GS Only	0.002 (0.004)	1.695 (3.702)	$< 10^{-3}$	0.003 (0.011)	$< 10^{-3}$	0.000 (0.001)	$< 10^{-3}$	0.205 (0.452)	$< 10^{-3}$	0.002 (0.006)	$< 10^{-3}$	$< 10^{-3}$
HINTS-GS	5.377 (0.000)	2.113 (2.498)	0.027 (0.000)	0.005 (0.012)	0.007 (0.000)	0.000 (0.001)	0.661 (0.000)	0.271 (0.432)	0.065 (0.000)	0.003 (0.006)	0.019 (0.000)	$< 10^{-3}$
Learned Greedy-GS	0.000 (0.001)	0.465 (1.125)	$< 10^{-3}$	0.009 (0.029)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.075 (0.151)	$< 10^{-3}$	0.005 (0.017)	$< 10^{-3}$	0.001 (0.001)
SymGS-related solvers												
SymGS Only	$< 10^{-3}$	0.883 (1.919)	$< 10^{-3}$	0.002 (0.006)	$< 10^{-3}$	0.000 (0.001)	$< 10^{-3}$	0.180 (0.400)	$< 10^{-3}$	0.001 (0.005)	$< 10^{-3}$	0.000 (0.001)
HINTS-SymGS	5.085 (0.000)	1.432 (1.700)	0.025 (0.000)	0.002 (0.007)	0.007 (0.000)	0.000 (0.001)	0.672 (0.000)	0.240 (0.381)	0.070 (0.000)	0.002 (0.005)	0.019 (0.000)	0.000 (0.001)
Learned Greedy-SymGS	$< 10^{-3}$	0.334 (0.943)	$< 10^{-3}$	0.007 (0.025)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.079 (0.193)	$< 10^{-3}$	0.004 (0.012)	$< 10^{-3}$	0.001 (0.001)
Jacobi (0.67)-related solvers												
Jacobi (0.67) Only	1.063 (2.321)	4.771 (10.421)	$< 10^{-3}$	0.005 (0.017)	$< 10^{-3}$	$< 10^{-3}$	0.428 (0.934)	1.535 (3.352)	$< 10^{-3}$	0.004 (0.016)	$< 10^{-3}$	$< 10^{-3}$
HINTS-Jacobi (0.67)	5.804 (0.001)	2.983 (3.666)	0.029 (0.000)	0.007 (0.019)	0.006 (0.000)	$< 10^{-3}$	1.080 (0.000)	0.776 (1.158)	0.065 (0.000)	0.008 (0.017)	0.021 (0.000)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.390 (1.195)	1.804 (5.363)	$< 10^{-3}$	0.017 (0.058)	$< 10^{-3}$	0.001 (0.002)	0.096 (0.208)	0.435 (0.910)	$< 10^{-3}$	0.013 (0.044)	$< 10^{-3}$	0.001 (0.004)
SOR (1.5)-related solvers												
SOR (1.5) Only	$< 10^{-3}$	0.727 (1.594)	$< 10^{-3}$	0.005 (0.019)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.037 (0.082)	$< 10^{-3}$	0.002 (0.007)	$< 10^{-3}$	0.000 (0.001)
HINTS-SOR (1.5)	4.826 (0.000)	1.239 (1.484)	0.027 (0.000)	0.007 (0.020)	0.008 (0.000)	0.001 (0.002)	0.645 (0.000)	0.080 (0.082)	0.068 (0.000)	0.005 (0.007)	0.019 (0.000)	0.001 (0.001)
Learned Greedy-SOR (1.5)	$< 10^{-3}$	0.302 (0.757)	$< 10^{-3}$	0.010 (0.032)	$< 10^{-3}$	0.001 (0.003)	$< 10^{-3}$	0.049 (0.119)	$< 10^{-3}$	0.007 (0.030)	$< 10^{-3}$	0.001 (0.002)

Table 11: Final error and AUC of squared L^2 error of Mode 1, 5, and 10 for varying numbers of solvers. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. are reported in $\times 10^{-3}$.

Equation	Poisson				ConvDiff			
	Mode 1 Error	Mode 1 AUC	Mode 5 Error	Mode 5 AUC	Mode 1 Error	Mode 1 AUC	Mode 5 Error	Mode 5 AUC
JW Mode								
[Jacobi, GS]	0.000 (0.001)	0.267 (0.571)	$< 10^{-3}$	0.006 (0.025)	$< 10^{-3}$	0.001 (0.005)	$< 10^{-3}$	0.055 (0.112)
[Jacobi, GS, SymGS]	0.000 (0.001)	0.185 (0.415)	$< 10^{-3}$	0.003 (0.009)	$< 10^{-3}$	0.000 (0.002)	$< 10^{-3}$	0.054 (0.114)
[Jacobi, GS, SymGS, Jacobi (0.67)]	0.000 (0.001)	0.172 (0.333)	$< 10^{-3}$	0.003 (0.010)	$< 10^{-3}$	0.000 (0.002)	$< 10^{-3}$	0.056 (0.119)
[Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)]	0.000 (0.001)	0.210 (0.399)	$< 10^{-3}$	0.006 (0.023)	$< 10^{-3}$	0.001 (0.004)	$< 10^{-3}$	0.026 (0.061)

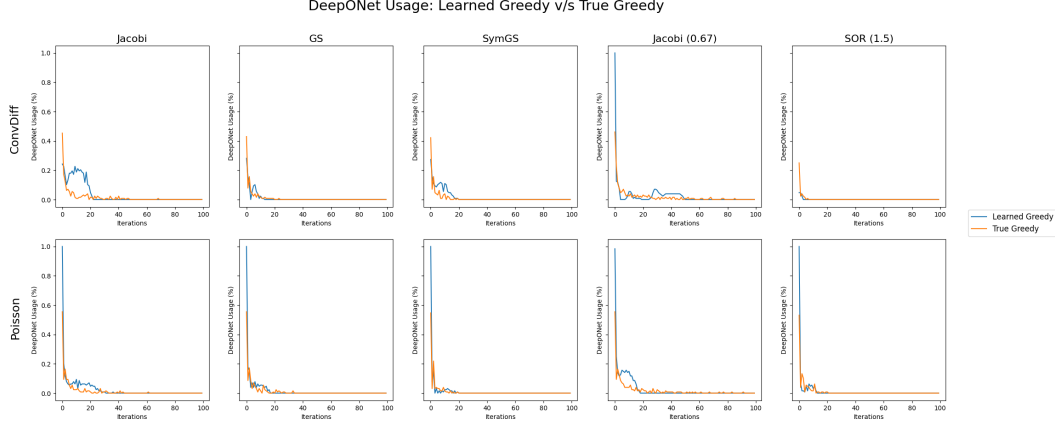


Figure 9: DeepONet usage over iterations for learned and true greedy routing across solver families. Columns correspond to different numerical solvers, and rows correspond to the two equations. The learned router closely follows the true greedy policy, capturing its nonuniform usage of DeepONet across iterations. For readability, we display the first 100 iterations (out of 300 total)

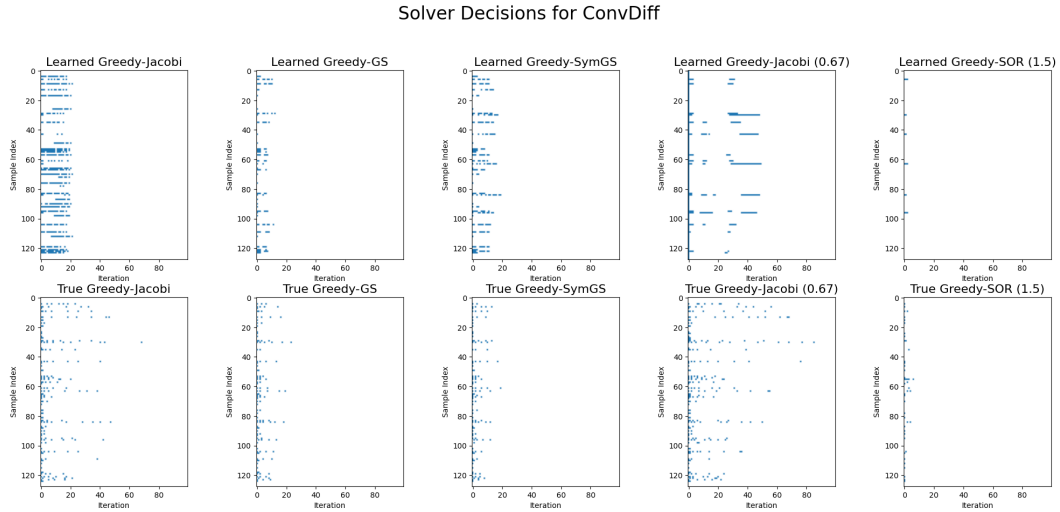


Figure 10: Solver selection patterns for the convection–diffusion equation across different solver pairings and test samples. Each column corresponds to a solver family, while rows show decisions from the learned greedy router (top) and the true greedy policy (bottom). Colored entries indicate iterations where the neural operator is selected, with blank regions corresponding to numerical solver updates. The learned router exhibits instance-dependent routing behavior.

Solver Decisions for Poisson

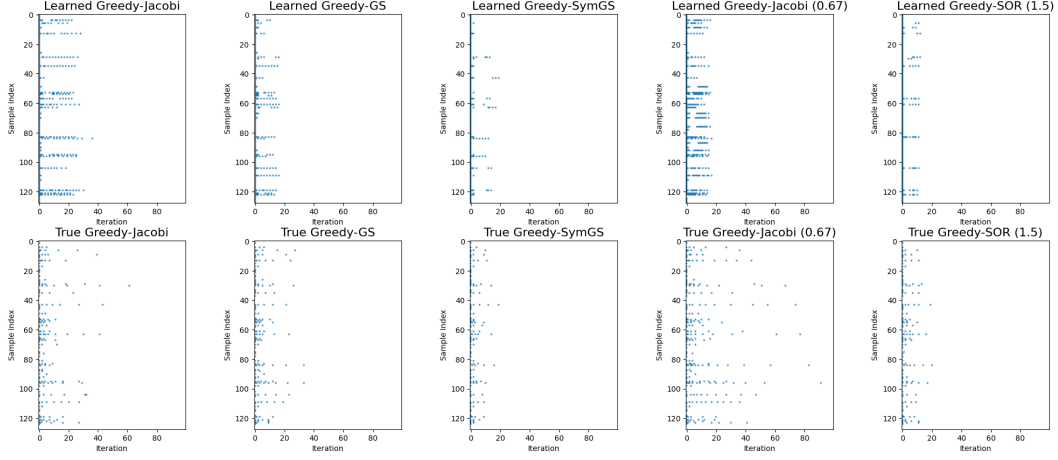


Figure 11: Solver selection patterns for the Poisson equation across different solver pairings and test samples. Each column corresponds to a solver family, while rows show decisions from the learned greedy router (top) and the true greedy policy (bottom). Colored entries indicate iterations where the neural operator is selected, with blank regions corresponding to numerical solver updates. The learned router exhibits instance-dependent routing behavior.

Figure 9 highlights the proportion of DeepONet calls across iterations, while Figures 10 and 11 visualize the routing decisions of the learned and true greedy policies across all test samples and solver families.

Two key observations emerge. First, the routing strategy is highly sample-dependent, indicating that a fixed routing schedule across all instances would be suboptimal and can slow convergence. Second, the learned routing policy closely tracks the behavior of the true greedy policy, providing empirical support for the approximation guarantees discussed in Section 5. In particular, the learned policy can be viewed as a smoothed approximation of the true greedy strategy, capturing its overall structure while exhibiting minor deviations due to learning and finite-sample effects.

Table 12: Average component selection frequencies for the Poisson equation. Rows indicate the router access set. Entries are mean ($\pm$ s.e.) over 128 test instances. Here NO denotes the neural operator; “-” indicates that the component is not available to the router.

$\mathcal{W}$	Jacobi	GS	SSOR	Jacobi (0.67)	SOR (1.5)	DeepONet
{Jacobi, GS}	0.405 (0.143)	0.588 (0.134)	-	-	-	0.007 (0.011)
{Jacobi, GS, SSOR}	0.000 (0.000)	0.000 (0.000)	0.996 (0.005)	-	-	0.004 (0.005)
{Jacobi, GS, SSOR, Jacobi (0.67)}	0.227 (0.112)	0.212 (0.118)	0.434 (0.212)	0.121 (0.102)	-	0.005 (0.007)
{Jacobi, GS, SSOR, Jacobi (0.67), SOR (1.5)}	0.009 (0.033)	0.570 (0.057)	0.260 (0.040)	0.000 (0.000)	0.156 (0.013)	0.005 (0.005)

Table 13: Average component selection frequencies for the Convection-Diffusion equation. Rows indicate the router access set. Entries are mean ($\pm$ s.e.) over 128 test instances. Here NO denotes the neural operator; “-” indicates that the component is not available to the router.

$\mathcal{W}$	Jacobi	GS	SSOR	Jacobi (0.67)	SOR (1.5)	DeepONet
{Jacobi, GS}	0.417 (0.307)	0.579 (0.307)	-	-	-	0.004 (0.006)
{Jacobi, GS, SSOR}	0.109 (0.109)	0.396 (0.110)	0.493 (0.175)	-	-	0.003 (0.005)
{Jacobi, GS, SSOR, Jacobi (0.67)}	0.168 (0.096)	0.190 (0.166)	0.410 (0.227)	0.229 (0.146)	-	0.003 (0.004)
{Jacobi, GS, SSOR, Jacobi (0.67), SOR (1.5)}	0.307 (0.150)	0.061 (0.131)	0.088 (0.170)	0.202 (0.089)	0.342 (0.066)	0.001 (0.001)

Tables 12 and 13 report solver selection frequencies across different solver ensembles. Several interesting observations emerge from these results.

First, the router typically relies on a small subset of solvers, with larger ensembles often leading to the dominance of a single solver (e.g., SOR(1.5) in convection–diffusion). This explains the diminishing returns observed in the multi-solver setting, as the solver ensemble grows.

Additionally, the neural operator is selected only rarely across all configurations. This reflects the strength of the numerical solvers rather than a limitation of our method. Our framework is designed to exploit this structure by relying primarily on numerical updates and invoking the learned component only when beneficial, demonstrating that selective use of learned corrections can yield performance gains without frequent deployment like in HINTS or similar fixed schedules.

Finally, the non-zero standard deviations indicate that solver usage varies across test instances, suggesting that no single global routing strategy emerges.

D.7 Results on a finer grid

We have replicated the Paired Solver and Solver Ensemble experiments for a grid of 63×63 .

In Table 14, we notice that, similar to the case with 31×31 grid, the learned greedy router closely matches the performance of the true greedy oracle and outperforms its single solver and HINTS baselines with great statistical significance.

Similar to Table 3, we notice, in Table 15, the learned greedy router seems to outperform the various pairwise configurations mostly with great statistical significance.

Table 14: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances for the grid 63×63 ; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value
Jacobi-related solvers								
Jacobi Only	4.483 (12.776)	$< 10^{-3}$	2.092 (5.889)	$< 10^{-3}$	1.470 (4.164)	0.001	0.757 (2.116)	0.001
HINTS-Jacobi	0.823 (0.163)	0.043	0.571 (1.205)	0.021	1.512 (0.433)	$< 10^{-3}$	0.565 (0.723)	$< 10^{-3}$
Learned Greedy-Jacobi	0.506 (2.201)	-	0.429 (1.293)	-	0.654 (1.603)	-	0.393 (0.969)	-
True-Greedy-Jacobi	0.199 (0.277)	-	0.289 (0.661)	-	0.317 (0.593)	-	0.277 (0.635)	-
GS-related solvers								
GS only	2.094 (6.009)	$< 10^{-3}$	1.519 (4.296)	$< 10^{-3}$	0.357 (1.025)	0.004	0.430 (1.211)	0.003
HINTS-GS	0.631 (0.003)	$< 10^{-3}$	0.444 (0.939)	$< 10^{-3}$	0.996 (0.004)	$< 10^{-3}$	0.369 (0.397)	0.03
Learned Greedy-GS	0.138 (0.581)	-	0.228 (0.623)	-	0.219 (0.697)	-	0.282 (0.820)	-
True-Greedy-GS	0.072 (0.064)	-	0.180 (0.403)	-	0.053 (0.065)	-	0.132 (0.279)	-
SymGS-related solvers								
SymGS only	1.318 (3.541)	$< 10^{-3}$	1.106 (2.978)	$< 10^{-3}$	0.267 (0.725)	0.012	0.362 (0.974)	0.014
HINTS-SymGS	0.566 (0.002)	$< 10^{-3}$	0.375 (0.782)	$< 10^{-3}$	1.007 (0.000)	$< 10^{-3}$	0.367 (0.329)	$< 10^{-3}$
Learned Greedy-SymGS	0.162 (0.155)	-	0.169 (0.310)	-	0.141 (0.355)	-	0.239 (0.599)	-
True-Greedy-SymGS	0.084 (0.094)	-	0.130 (0.266)	-	0.146 (0.285)	-	0.130 (0.241)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	5.833 (16.544)	$< 10^{-3}$	2.365 (6.639)	$< 10^{-3}$	1.985 (5.595)	0.005	0.876 (2.443)	0.004
HINTS-Jacobi (0.67)	1.177 (0.445)	$< 10^{-3}$	0.687 (1.414)	$< 10^{-3}$	1.970 (0.989)	$< 10^{-3}$	0.652 (0.883)	0.001
Learned Greedy-Jacobi (0.67)	0.693 (1.959)	-	0.489 (1.217)	-	1.060 (2.422)	-	0.517 (1.211)	-
True-Greedy-Jacobi (0.67)	0.396 (0.741)	-	0.406 (0.979)	-	0.528 (1.133)	-	0.352 (0.831)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.115 (0.331)	$< 10^{-3}$	0.652 (1.850)	$< 10^{-3}$	$< 10^{-3}$	0.056	0.078 (0.220)	0.002
HINTS-SOR (1.5)	0.480 (0.000)	$< 10^{-3}$	0.304 (0.680)	$< 10^{-3}$	0.608 (0.000)	$< 10^{-3}$	0.147 (0.165)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.017 (0.084)	-	0.148 (0.516)	-	$< 10^{-3}$	-	0.039 (0.090)	-
True-Greedy-SOR (1.5)	0.004 (0.003)	-	0.081 (0.188)	-	$< 10^{-3}$	-	0.030 (0.067)	-

Table 15: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances for the grid 63×63 ; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each solver ensemble against the corresponding pairwise learned router (e.g., NO + Jacobi, NO + GS), using a one-sided alternative that the ensemble achieves lower error/AUC. Reported p -values correspond to these tests.

$\mathcal{W}$	$\ e_h^{(T)}\ \times 10^3$	AUC	Jacobi p -value	GS p -value	SymGS p -value	Jacobi (0.67) p -value	SOR (1.5) p -value
Poisson							
Learned Greedy {Jacobi, GS}	0.077 (0.064)	0.188 (0.410)	0.002	0.08	-	-	-
True Greedy {Jacobi, GS}	0.072 (0.064)	0.180 (0.403)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	0.112 (0.086)	0.151 (0.275)	0.002	0.015	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	0.054 (0.059)	0.126 (0.264)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.109 (0.103)	0.152 (0.282)	0.002	0.013	$< 10^{-3}$	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.054 (0.059)	0.126 (0.264)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.014 (0.006)	0.092 (0.205)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.054
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.004 (0.003)	0.082 (0.189)	-	-	-	-	-
ConvDiff							
Learned Greedy {Jacobi, GS}	0.136 (0.447)	0.170 (0.394)	$< 10^{-3}$	0.015	-	-	-
True Greedy {Jacobi, GS}	0.053 (0.065)	0.132 (0.279)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	0.063 (0.098)	0.142 (0.335)	$< 10^{-3}$	0.006	0.001	-	-
True Greedy {Jacobi, GS, SymGS}	0.025 (0.037)	0.105 (0.220)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.114 (0.216)	0.181 (0.413)	$< 10^{-3}$	0.024	0.011	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.025 (0.037)	0.105 (0.220)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.037 (0.087)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.069
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.030 (0.067)	-	-	-	-	-

841 D.8 Results with Dirichlet Boundary Conditions

842 We have replicated the Paired Solver and Solver Ensemble experiments for solutions with zero
843 dirichlet boundary conditions.

844 In Table 16, we notice that, similar to PDEs with periodic boundary conditions, the learned greedy
845 router closely matches the performance of the true greedy oracle and outperforms its single solver
846 and HINTS baselines with great statistical significance. Despite the learned router matching the
847 performance of the true greedy oracle, we observe that the learned router doesn't seem to provide
848 much improvement over single-solver baseline in the case of SOR (1.5)-related solvers. This may be
849 a result of the deepnet correction always being weaker relative to the SOR (1.5) correction in terms
850 of error reduction.

851 Similar to Table 3, we notice, in Table 15, the learned greedy router seems to outperform the various
852 pairwise configurations mostly with great statistical significance.

Table 16: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances with dirichlet boundaries; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ p$ -value	AUC	AUC p -value	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ p$ -value	AUC	AUC p -value
Jacobi-related solvers								
Jacobi Only	0.704 (2.151)	0.001	0.703 (1.929)	$< 10^{-3}$	0.003 (0.006)	0.057	0.169 (0.430)	$< 10^{-3}$
HINTS-Jacobi	2.674 (7.315)	$< 10^{-3}$	0.869 (2.396)	$< 10^{-3}$	0.474 (0.001)	$< 10^{-3}$	0.199 (0.254)	$< 10^{-3}$
Learned Greedy-Jacobi	0.243 (0.851)	-	0.236 (0.719)	-	0.003 (0.006)	-	0.076 (0.162)	-
True-Greedy-Jacobi	0.042 (0.047)	-	0.099 (0.252)	-	0.003 (0.007)	-	0.042 (0.089)	-
GS-related solvers								
GS only	0.138 (0.431)	0.001	0.408 (1.139)	$< 10^{-3}$	0.003 (0.006)	0.028	0.057 (0.146)	$< 10^{-3}$
HINTS-GS	2.668 (7.316)	$< 10^{-3}$	0.841 (2.360)	$< 10^{-3}$	0.470 (0.001)	$< 10^{-3}$	0.115 (0.137)	$< 10^{-3}$
Learned Greedy-GS	0.032 (0.158)	-	0.115 (0.433)	-	0.003 (0.006)	-	0.024 (0.048)	-
True-Greedy-GS	0.010 (0.015)	-	0.063 (0.166)	-	0.003 (0.006)	-	0.017 (0.036)	-
SymGS-related solvers								
SymGS only	2.924 (8.865)	0.01	0.926 (2.740)	0.006	1.331 (3.687)	0.004	0.413 (1.133)	0.004
HINTS-SymGS	2.748 (7.375)	0.008	0.856 (2.427)	0.008	0.917 (1.015)	0.006	0.332 (0.697)	0.003
Learned Greedy-SymGS	1.333 (5.715)	-	0.416 (1.722)	-	0.628 (1.468)	-	0.196 (0.449)	-
True-Greedy-SymGS	0.342 (0.906)	-	0.147 (0.375)	-	0.184 (0.323)	-	0.067 (0.118)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.265 (3.768)	$< 10^{-3}$	0.910 (2.468)	$< 10^{-3}$	0.008 (0.021)	0.003	0.253 (0.642)	$< 10^{-3}$
HINTS-Jacobi (0.67)	2.686 (7.311)	$< 10^{-3}$	0.886 (2.412)	$< 10^{-3}$	0.487 (0.002)	$< 10^{-3}$	0.220 (0.279)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.134 (0.275)	-	0.215 (0.660)	-	0.004 (0.010)	-	0.127 (0.393)	-
True-Greedy-Jacobi (0.67)	0.096 (0.220)	-	0.145 (0.408)	-	0.004 (0.013)	-	0.061 (0.131)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.004 (0.011)	0.012	0.145 (0.412)	$< 10^{-3}$	0.003 (0.006)	1.0	0.006 (0.014)	1.0
HINTS-SOR (1.5)	2.650 (7.321)	$< 10^{-3}$	0.791 (2.295)	$< 10^{-3}$	0.468 (0.001)	$< 10^{-3}$	0.014 (0.014)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.004 (0.011)	-	0.039 (0.091)	-	0.003 (0.006)	-	0.006 (0.014)	-
True-Greedy-SOR (1.5)	0.004 (0.011)	-	0.028 (0.070)	-	0.003 (0.006)	-	0.006 (0.014)	-

E Cost-Aware Routing and Wall-Clock Evaluation

The experiments of Section 7 compare methods per iteration, which favors methods whose iterations are expensive. This appendix evaluates the end-to-end *wall-clock time to a target accuracy* of the learned router against HINTS, the classical solver alone and the classical methods a practitioner would reach for on these problems (geometric multigrid and preconditioned Krylov methods), for all five classical solvers and three PDEs, on 128^2 , 256^2 and 512^2 grids, using the cost-aware instantiation of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion equations of Section 7 we add *anisotropic diffusion*, $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ (diffusion tensor $\text{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard model problem on which point-relaxation methods fail to smooth: an error component oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep, and geometric multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-coarsening would be needed). Section E.1 describes the protocol; Sections E.2 to E.4 collect all results per equation (every grid, every tolerance, every baseline, with significance tests and seed trials); Section E.5 studies ensembles of one, two and three classical solvers; Section E.6 accounts for overheads and training cost; and Section E.7 verifies numerically that the assumptions of Theorem 4.1, Propositions 4.2–4.3 and Theorem 5.1 hold in these settings.

E.1 Protocol

Fast, matched implementations. A wall-clock comparison is meaningful only if the classical baseline is not handicapped by implementation. All solvers are therefore re-implemented for the periodic constant-coefficient setting as $O(N^2)$ operations: Jacobi and damped Jacobi as 5-point stencil sweeps, Gauss–Seidel, SOR and symmetric GS via cached sparse triangular factorizations, geometric multigrid as V(2,2)-cycles with lexicographic Gauss–Seidel smoothing, full-weighting restriction, bilinear prolongation, rediscrctised coarse operators and a direct solve on the 32×32 coarsest grid (error contraction ≈ 0.03 per cycle on the isotropic equations). Their one-step iterates match the dense implementations used in the rest of the paper to 10^{-12} in double precision. Reference solutions are computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of the discrete system, so true errors are available for evaluation at no charged cost; the FFT solve is used only as this reference (it is a direct method that exists because the test problems have constant coefficients and periodic boundaries, and is not a competitor of the iterative methods compared here). All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch, one thread) on an otherwise idle machine.

Corrector. The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}_h^a u_h^{(t)}$ and evaluated on a 64×64 sensor grid, i.e., on the coarser grid obtained by band-limited (FFT) restriction to the modes $|k_1|, |k_2| \leq 31$ at every grid size (for the anisotropic equation the sensor grid is $N \times N/4$, i.e., full resolution along x_1 , so that the band contains every mode the point smoothers cannot damp; the sensor grid is the only PDE-specific hyperparameter of the pipeline); its output is prolonged back to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction and never injects error into the high-frequency band that the classical smoother owns (Figure 12). The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it is fit by least squares on 32,000–64,000 exact (residual, error) pairs of the discrete operator drawn from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and reaches a validation relative error of $\approx 2 \times 10^{-6}$ on every mode of the band. The corrector is applied scale-equivariantly, $C_{\text{NO}}(r) = \|Rr\| \mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\text{NO}}(0) = 0$ (Proposition 4.2). Its cost is dominated by a 4096×4096 matrix–vector product and is therefore almost independent of N (0.65 ms at 128^2 , 3.6 ms at 512^2 , Table 41), i.e., about 12 Jacobi or 3 GS sweeps at 128^2 and about 2.5 Jacobi sweeps at 512^2 (Tables 20, 26 and 32). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j (residual evaluation plus the update) are measured once per pairing and grid on the benchmark machine (minimum over seven repeated blocks of the block median, cached), the unit of cost u is one corrector call, and every operation is applied $m_j = \max\{1, \text{round}(u/c_j)\}$ times per decision; for the five stationary solvers this equalises costs to within 8%. An operation dearer than the unit (a multigrid cycle) is applied once and compared

per unit of cost, $\|e_j\|^{u/(m_j c_j)}$; the surrogate weights $\tilde{c}_j$ use the same exponent. Ensembles reuse the cached pairwise costs of their members, so every member has the same macro-action size in the ensemble as in its pairwise run. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* rows run the cost-agnostic Algorithm 1 on single operations, as in the main text.

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing $\approx 5 \mu\text{s}$ per decision independently of N . It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε -exploration (probability 0.1), followed by two to three DAgger rounds [Ross et al., 2011] in which the current router’s own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 32–128 instances per round (fewer on the larger grids) drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing and ensemble is trained in seconds to a few minutes. At 512^2 the routers are trained from 32 oracle rollouts of at most 400 iterations (stopping error 10^{-8}); for the convection–diffusion Jacobi pairing, where the first such router missed h^2 within the cap on 4 of 16 instances, the reported router uses 64 rollouts of at most 800 iterations—the only per-cell change to the recipe.

Baselines and metric. HINTS applies the corrector every τ -th iteration; we report $\tau = 25$ (the schedule used in the main text and in Zhang et al. [2024]) and the best of $\tau \in \{5, 10, 25, 50\}$ selected *per cell and per tolerance* on the test instances themselves (an optimistic baseline). The classical baselines without corrector run on the same stencil with the same accounting: multigrid V(2,2) alone; conjugate gradients, CG preconditioned by symmetric GS and by a symmetric V-cycle for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles and was not run at 512^2 , where it was two orders of magnitude slower than the router on the smaller grids). They are not run for anisotropic diffusion, where point-smoothed multigrid loses its grid-independent contraction (≈ 0.85 per cycle) and would not be informative. For each test instance (64 at 128^2 , 32 at 256^2 , 16 at 512^2) we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). We report the median first time at which the error drops below a tolerance ε , paired per-instance median speedups, and one-sided Wilcoxon signed-rank tests on the paired log time ratios (alternative: the router is faster); a run that does not reach ε within the iteration cap (60,000 at 128^2 , 40,000 at 256^2 , 8,000 at 512^2) is marked †, reported as a lower bound when it is the majority, counted against the router when it is the router’s, and entered at the time it spent up to the cap on the baseline side of the tests (a lower bound, hence conservative). We highlight $\varepsilon = h^2$: the discretization error of the second-order scheme scales as $O(h^2)$, so driving the algebraic error far below h^2 does not improve the computed solution; we also report 10^{-3} and 10^{-8} , and the iteration-based metrics of the main text (final relative error and $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\|/\|u_h\|$ over $T = 300$ iterations) on the 128^2 grid. Every router was additionally retrained from scratch with five independent seeds (different training instances, exploration and initialisation; the corrector and the test set are fixed) and re-run on the test set; because those trials ran in separate sessions, whose live timings are not comparable with the main run’s, they are compared in *work units* (measured per-iteration costs times executed operations, plus $5 \mu\text{s}$ per decision), which are determined by the decision sequence alone.

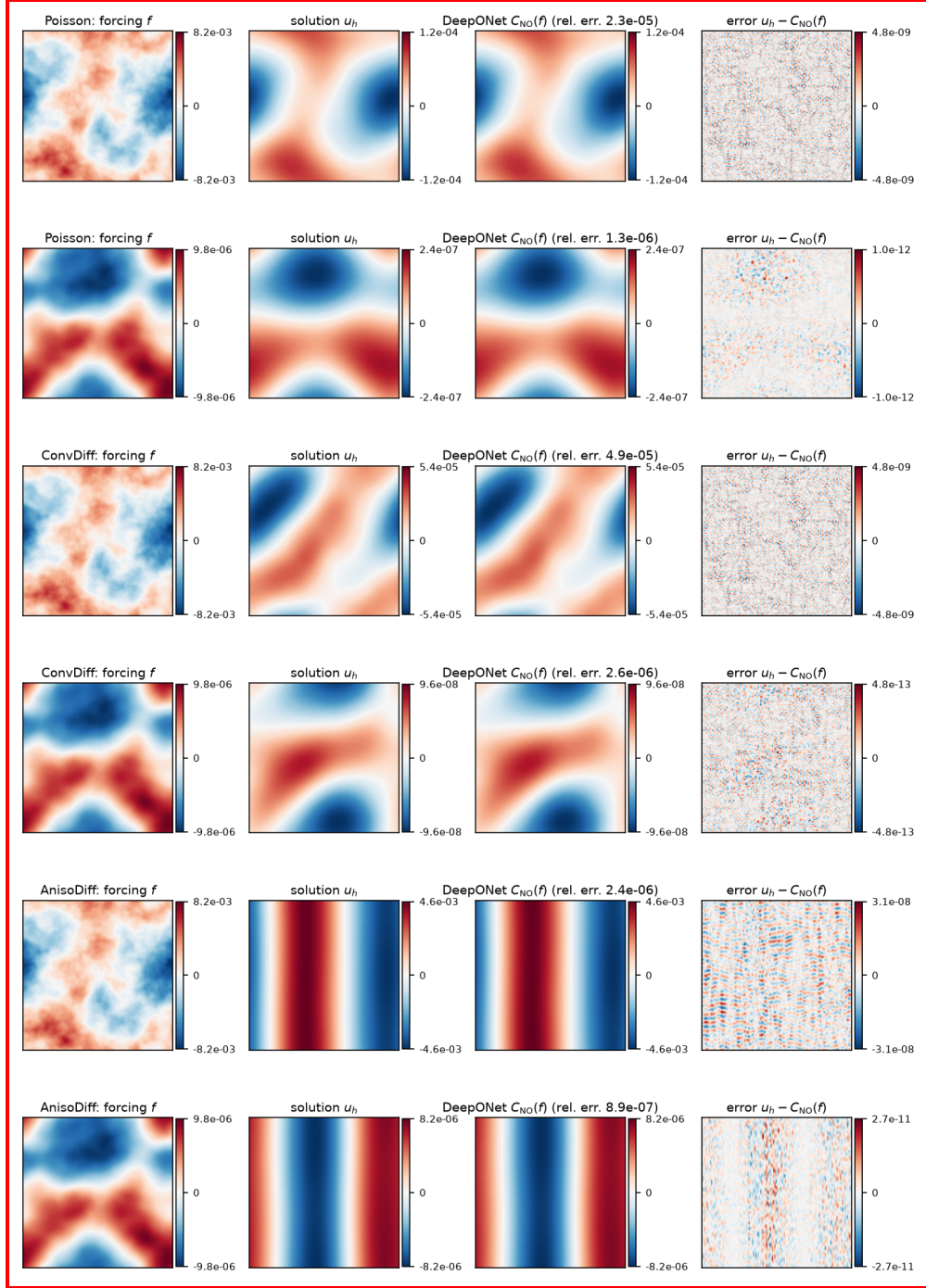


Figure 12: One-shot corrector predictions on the 128×128 grid (two test instances per equation): forcing, exact discrete solution, the DeepONet correction applied to the initial residual ($u_h^{(0)} = 0$), and the remaining error, which is confined to the modes outside the corrector's band (the relative error of the prediction is 10^{-6} – 10^{-5} on the isotropic equations and 10^{-6} on anisotropic diffusion, whose solution is dominated by modes that are smooth in x_2).

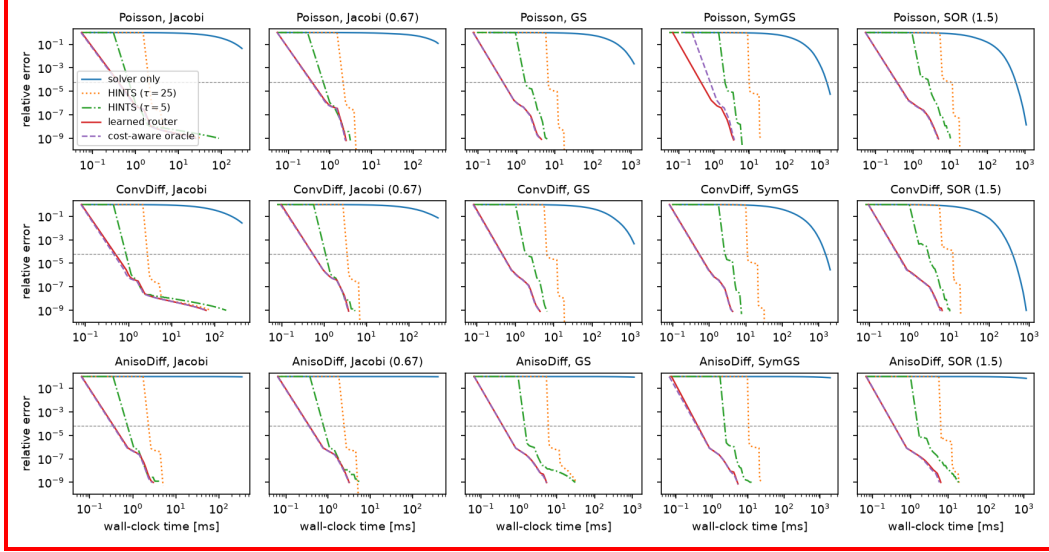


Figure 13: Relative error versus charged wall-clock time on the 128×128 grid for one representative test instance of every pairing (columns) and equation (rows); the dashed line is $\varepsilon = h^2$. The learned router and the cost-aware oracle are indistinguishable.

956 E.2 Poisson

957 Table 17 collects the median time to 10^{-3} , h^2 and 10^{-8} for every pairing, every policy and the
 958 classical baselines on all three grids; Table 18 the paired speedups of the learned router with their p -
 959 values; Figure 14 and table 19 where the iterations go; Table 20 the measured costs and macro-action
 960 sizes; Table 21 the iteration-based metrics; and Table 22 the seed trials.

961 **Uniform gains and their mechanism.** On the 128^2 grid the learned router is the fastest deployable
 962 method at $\varepsilon = h^2$ in every pairing: hundreds to thousands of times faster than the solver alone,
 963 several times faster than HINTS ($\tau=25$) and still faster than the best fixed period chosen per cell and
 964 per tolerance on the test set, with all paired $p < 0.01$ (Table 18); at 10^{-8} the margins over the best
 965 fixed period shrink but remain significant in every pairing, and the best period itself varies across
 966 pairings and tolerances ($\tau=5$ at truncation level, $\tau \in \{5, 10, 50\}$ at 10^{-8}), so no single fixed schedule
 967 is competitive everywhere. Table 19 explains the numbers: with a corrector that removes the smooth
 968 band to 10^{-6} in one call, the cost-aware oracle and the router call it first and reach h^2 after one or
 969 two further sweeps, i.e., in about two iterations, whereas HINTS reaches the same accuracy only
 970 at its first scheduled call (iteration τ), having spent $\tau-1$ sweeps on an error that is almost entirely
 971 smooth. Below h^2 the remaining error is the high-frequency band that the corrector cannot see; the
 972 router therefore stops calling it (one to three calls in total down to 10^{-8} ; Figure 14), whereas a fixed
 973 schedule keeps paying for a call every τ iterations. The cost-agnostic greedy oracle (Algorithm 1 on
 974 single operations) makes the same decisions as its cost-aware counterpart at 128^2 , because after the
 975 first call the corrector is never better than a single sweep on the remaining high-frequency error; the
 976 learned router reproduces the cost-aware oracle’s decisions almost exactly (same median iteration
 977 and call counts, median times within a few percent, the difference being its own decision cost and
 978 timer noise), and the five retrained routers reproduce the main router’s decision sequence to h^2 on
 979 essentially every test instance (Table 22).

980 **Classical baselines and larger grids.** Against the classical methods without corrector (bottom
 981 block of Table 17), the router with its best stationary pairing reaches truncation-level accuracy several
 982 times faster than multigrid alone (three V-cycles) and than multigrid-preconditioned CG at 128^2
 983 and 256^2 , and an order of magnitude faster than unpreconditioned or SymGS-preconditioned CG.
 984 The reason is the corrector’s accuracy on the smooth band: one call removes the modes $|k| \leq 31$ to
 985 10^{-6} at the price of a single 4096×4096 matrix–vector product, whereas a V(2,2)-cycle contracts
 986 the whole spectrum by ≈ 0.03 at the price of four Gauss–Seidel sweeps on the fine grid plus the

coarse-grid work. Routing over $\{\text{NO}, \text{multigrid}\}$ is itself about twice as fast as multigrid alone on every grid: the oracle and the router call the corrector once and let the V-cycle remove the rest, which is the case in which the per-unit-cost comparison of Section 4 matters (the cycle is dearer than the unit). On 256^2 the router remains the fastest iterative method in every pairing ($1.58\times\text{--}10.9\times$ over HINTS, $1.11\times\text{--}2.33\times$ over the best period across the three equations). At 512^2 the margins shrink but the ordering holds for the stationary pairings, and the $\{\text{NO}, \text{multigrid}\}$ router is the fastest iterative method we measured at this size. With Jacobi and Gauss–Seidel the gains at 512^2 are small because the corrector’s fixed band $|k| \leq 31$ leaves the smoother all modes $k \geq 32$, whose slowest member contracts by only 0.98 per Jacobi sweep at $N = 512$ (against 0.71 at $N = 128$): every policy then needs the same ≈ 80 sweeps after a single corrector call, the wall-clock time is dominated by sweeps, and the schedule barely matters (with Jacobi the router misses h^2 within the 8,000-iteration cap on 2 of 16 instances, marked in the table). This is the expected limit of a corrector whose band does not grow with the grid: the hybrid’s advantage comes from removing the modes the smoother cannot damp, and a corrector with a band that scales with N (at the price of a dense matrix–vector product that grows like the fourth power of the band) or a multigrid member restores the gains, as the multigrid pairing shows.

Table 17: Poisson: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids (64, 32 and 16 test instances) for every pairing $\{\text{NO}, \text{solver}\}$ and policy, and for the classical baselines without corrector. “HINTS (best τ)” is the best of $\tau \in \{5, 10, 25, 50\}$ chosen per cell on the test set itself (the chosen τ is the subscript). The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the router where it is the fastest deployable method of its pairing; $\dagger n$: n instances did not reach the tolerance within the iteration cap ($>$: lower bound).

Pairing	Method	128 × 128			256 × 256			512 × 512		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	806.1 ms	1.15 s	2.14 s	$>7.61 \text{ s}^{132}$	$>7.61 \text{ s}^{132}$	$>7.61 \text{ s}^{132}$	$>9.85 \text{ s}^{116}$	$>9.85 \text{ s}^{116}$	$>9.85 \text{ s}^{116}$
	HINTS ($\tau=25$)	2.61 ms	2.65 ms	226.1 ms	5.90 ms	5.95 ms	85.5 ms	33.1 ms	87.0 ms	309.2 ms ¹⁷
	HINTS (best τ)	1.06 ms _{$\tau=5$}	1.10 ms _{$\tau=5$}	182.8 ms _{$\tau=50$}	2.07 ms _{$\tau=5$}	2.47 ms _{$\tau=5$}	72.9 ms _{$\tau=50$}	9.27 ms _{$\tau=5$}	87.0 ms _{$\tau=25$}	293.9 ms ¹⁷ _{$\tau=50$}
	Greedy oracle (Alg. 1)	803 μs	854 μs	155.1 ms	1.28 ms	2.30 ms	67.2 ms	4.74 ms	78.2 ms	275.5 ms ¹⁷
	Cost-aware oracle	880 μs	942 μs	154.1 ms	1.42 ms	2.32 ms	67.0 ms	4.74 ms	74.2 ms	261.5 ms ¹⁷
	Learned router (ours)	918 μs	987 μs	151.3 ms	1.47 ms	2.38 ms	73.1 ms	4.86 ms	78.8 ms¹²	282.9 ms¹⁷
{NO, Jacobi (0.67)}	Solver only	1.16 s	1.62 s	3.16 s	$>7.74 \text{ s}^{132}$	$>7.74 \text{ s}^{132}$	$>7.74 \text{ s}^{132}$	–	–	–
	HINTS ($\tau=25$)	2.71 ms	2.71 ms	5.16 ms	6.02 ms	6.41 ms	16.8 ms	–	–	–
	HINTS (best τ)	1.11 ms _{$\tau=5$}	1.13 ms _{$\tau=5$}	3.03 ms _{$\tau=10$}	2.10 ms _{$\tau=5$}	3.43 ms _{$\tau=5$}	16.8 ms _{$\tau=25$}	–	–	–
	Greedy oracle (Alg. 1)	818 μs	915 μs	2.83 ms	1.30 ms	2.93 ms	16.2 ms	–	–	–
	Cost-aware oracle	820 μs	922 μs	2.93 ms	1.30 ms	2.77 ms	16.1 ms	–	–	–
	Learned router (ours)	839 μs	912 μs	2.93 ms	1.32 ms	2.83 ms	16.8 ms	–	–	–
{NO, GS}	Solver only	1.29 s	1.83 s	3.45 s	17.80 s	28.83 s	$>31.70 \text{ s}^{132}$	$>29.96 \text{ s}^{116}$	$>29.96 \text{ s}^{116}$	$>29.96 \text{ s}^{116}$
	HINTS ($\tau=25$)	6.43 ms	6.65 ms	13.9 ms	20.4 ms	22.0 ms	43.5 ms	92.5 ms	142.8 ms	439.6 ms
	HINTS (best τ)	1.72 ms _{$\tau=5$}	1.99 ms _{$\tau=5$}	5.55 ms _{$\tau=5$}	4.49 ms _{$\tau=5$}	6.07 ms _{$\tau=5$}	30.4 ms _{$\tau=10$}	20.0 ms _{$\tau=5$}	142.8 ms _{$\tau=25$}	439.6 ms _{$\tau=25$}
	Greedy oracle (Alg. 1)	835 μs	960 μs	4.34 ms	1.36 ms	3.85 ms	25.4 ms	4.65 ms	141.4 ms	447.1 ms
	Cost-aware oracle	827 μs	953 μs	4.37 ms	1.33 ms	3.78 ms	25.6 ms	4.87 ms	130.0 ms	433.0 ms
	Learned router (ours)	850 μs	1.01 ms	4.45 ms	1.36 ms	3.92 ms	26.0 ms	4.70 ms	131.5 ms	417.8 ms
{NO, SymGS}	Solver only	1.08 s	1.52 s	2.91 s	16.41 s	26.53 s	44.30 s	–	–	–
	HINTS ($\tau=25$)	10.2 ms	10.3 ms	20.2 ms	36.1 ms	37.3 ms	71.9 ms	–	–	–
	HINTS (best τ)	2.35 ms _{$\tau=5$}	2.60 ms _{$\tau=5$}	6.94 ms _{$\tau=5$}	7.18 ms _{$\tau=5$}	8.71 ms _{$\tau=5$}	29.8 ms _{$\tau=5$}	–	–	–
	Greedy oracle (Alg. 1)	829 μs	1.09 ms	3.87 ms	1.36 ms	4.18 ms	24.3 ms	–	–	–
	Cost-aware oracle	829 μs	1.13 ms	3.86 ms	1.35 ms	4.19 ms	23.8 ms	–	–	–
	Learned router (ours)	853 μs	1.13 ms	3.88 ms	1.34 ms	4.22 ms	24.4 ms	–	–	–
{NO, SOR (1.5)}	Solver only	424.4 ms	600.9 ms	1.15 s	5.94 s	9.60 s	15.97 s	$>29.74 \text{ s}^{116}$	$>29.74 \text{ s}^{116}$	$>29.74 \text{ s}^{116}$
	HINTS ($\tau=25$)	6.37 ms	12.2 ms	18.5 ms	20.5 ms	40.7 ms	60.9 ms	93.2 ms	182.7 ms	269.4 ms
	HINTS (best τ)	1.72 ms _{$\tau=5$}	3.37 ms _{$\tau=5$}	8.56 ms _{$\tau=10$}	4.57 ms _{$\tau=5$}	9.06 ms _{$\tau=5$}	27.2 ms _{$\tau=10$}	18.7 ms _{$\tau=5$}	74.4 ms _{$\tau=5$}	214.9 ms _{$\tau=10$}
	Greedy oracle (Alg. 1)	814 μs	1.03 ms	7.48 ms	1.31 ms	3.69 ms	22.7 ms	4.56 ms	61.1 ms	186.7 ms
	Cost-aware oracle	800 μs	1.02 ms	6.81 ms	1.30 ms	3.51 ms	22.4 ms	4.62 ms	60.3 ms	185.9 ms
	Learned router (ours)	823 μs	1.02 ms	7.05 ms	1.36 ms	3.80 ms	24.2 ms	4.69 ms	60.7 ms	188.5 ms
{NO, Multigrid}	Solver only	2.62 ms	3.84 ms	7.64 ms	9.61 ms	15.1 ms	28.6 ms	47.3 ms	90.8 ms	140.9 ms
	HINTS ($\tau=25$)	2.62 ms	3.83 ms	7.61 ms	9.73 ms	14.7 ms	28.7 ms	47.2 ms	92.0 ms	140.2 ms
	HINTS (best τ)	2.59 ms _{$\tau=5$}	3.81 ms _{$\tau=5$}	5.89 ms _{$\tau=5$}	9.56 ms _{$\tau=50$}	14.6 ms _{$\tau=5$}	20.5 ms _{$\tau=5$}	47.2 ms _{$\tau=25$}	90.5 ms _{$\tau=10$}	102.2 ms _{$\tau=5$}
	Greedy oracle (Alg. 1)	828 μs	2.01 ms	4.63 ms	1.39 ms	6.14 ms	15.8 ms	4.68 ms	39.5 ms	95.5 ms
	Cost-aware oracle	773 μs	2.02 ms	4.66 ms	1.31 ms	6.09 ms	16.2 ms	4.86 ms	38.6 ms	97.0 ms
	Learned router (ours)	783 μs	2.05 ms	4.72 ms	1.32 ms	5.97 ms	16.6 ms	4.76 ms	38.7 ms	99.5 ms
Classical baselines without corrector (same stencil, same accounting)										
	Multigrid V(2,2) alone	2.75 ms	3.99 ms	7.97 ms	9.27 ms	14.1 ms	27.0 ms	42.8 ms	81.9 ms	126.5 ms
	CG	9.37 ms	11.5 ms	16.3 ms	58.0 ms	73.1 ms	102.5 ms	689.5 ms	920.9 ms	1.18 s
	PCG (SymGS)	20.3 ms	30.8 ms	52.5 ms	146.5 ms	254.7 ms	365.0 ms	1.33 s	2.52 s	3.14 s
	PCG (multigrid)	2.97 ms	4.35 ms	7.36 ms	10.5 ms	15.2 ms	24.5 ms	44.2 ms	85.6 ms	113.2 ms

Table 18: Poisson: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), over the best fixed period for that cell, over multigrid alone and over multigrid-preconditioned CG, with the one-sided Wilcoxon p -value that the router is faster (or, where its median is larger, that it is slower). Bold: $\geq 1.1 \times$ with $p < 0.01$.

Pairing	ε	128 × 128				256 × 256				512 × 512			
		vs. HINTS-25	vs. best τ	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. multigrid	vs. MG-Krylov
{NO, Jacobi}	h^2	2.68 × ($< 10^{-10}$)	1.21 × (10^{-9})	3.43 × (10^{-9})	3.57 × (10^{-9})	2.52 × (10^{-10})	1.56 × (10^{-9})	6.70 × (10^{-9})	6.30 × (10^{-9})	1.11 × (0.001)	1.11 × (0.001)	1.14 × (0.121)	1.26 × (0.086)
{NO, Jacobi (0.67)}	10^{-8}	1.55 × ($< 10^{-10}$)	1.22 × (10^{-9})	0.06 × (slower, 10^{-9})	0.05 × (slower, 10^{-9})	1.23 × (10^{-10})	1.11 × (10^{-9})	0.37 × (slower, 0.009)	0.35 × (slower, 0.007)	0.90 × (slower, 0.590)	0.89 × (slower, 0.545)	0.45 × (slower, 0.082)	0.40 × (slower, 0.082)
	h^2	2.90 × ($< 10^{-10}$)	1.20 × (10^{-9})	3.62 × ($< 10^{-10}$)	3.85 × ($< 10^{-10}$)	2.36 × (10^{-9})	1.73 × (10^{-9})	5.62 × (10^{-9})	5.29 × (10^{-9})	—	—	—	—
{NO, GS}	10^{-8}	1.78 × ($< 10^{-10}$)	1.09 × (10^{-9})	2.65 × ($< 10^{-10}$)	2.42 × ($< 10^{-10}$)	1.10 × (10^{-9})	1.10 × (10^{-9})	1.65 × (10^{-9})	1.59 × (10^{-9})	—	—	—	—
	h^2	6.75 × ($< 10^{-10}$)	1.94 × ($< 10^{-10}$)	3.39 × ($< 10^{-10}$)	3.52 × ($< 10^{-10}$)	5.77 × (10^{-9})	1.70 × (10^{-9})	4.15 × (10^{-9})	3.85 × (10^{-9})	1.11 × (0.003)	1.11 × (0.003)	0.69 × (slower, 0.530)	0.76 × (slower, 0.590)
{NO, SymGS}	10^{-8}	3.25 × ($< 10^{-10}$)	1.25 × (10^{-9})	1.72 × ($< 10^{-10}$)	1.75 × ($< 10^{-10}$)	1.67 × (10^{-9})	1.20 × (10^{-9})	1.08 × (0.007)	0.98 × (slower, 0.701)	1.07 × (0.004)	1.07 × (0.004)	0.31 × (slower, 10^{-9})	0.28 × (slower, 10^{-9})
	h^2	9.21 × ($< 10^{-10}$)	2.34 × ($< 10^{-10}$)	3.20 × ($< 10^{-10}$)	3.42 × ($< 10^{-10}$)	9.96 × (10^{-9})	2.27 × (10^{-9})	4.00 × (10^{-9})	3.98 × (10^{-9})	—	—	—	—
{NO, SOR (1.5)}	10^{-8}	5.26 × ($< 10^{-10}$)	1.81 × ($< 10^{-10}$)	1.99 × ($< 10^{-10}$)	1.93 × ($< 10^{-10}$)	2.97 × (10^{-9})	1.22 × (10^{-9})	1.16 × (0.004)	1.05 × (0.006)	—	—	—	—
	h^2	10.3 × ($< 10^{-10}$)	3.22 × ($< 10^{-10}$)	3.28 × ($< 10^{-10}$)	3.45 × ($< 10^{-10}$)	10.9 × (10^{-9})	2.33 × (10^{-9})	4.28 × (10^{-9})	4.13 × (10^{-9})	3.10 × (10^{-9})	3.10 × (10^{-9})	1.49 × (0.052)	1.64 × (0.037)
{NO, Multigrid}	10^{-8}	2.65 × ($< 10^{-10}$)	1.25 × (10^{-9})	1.14 × (10^{-9})	1.08 × (0.011)	2.40 × (10^{-9})	1.09 × (10^{-9})	1.16 × (0.004)	1.04 × (0.060)	1.37 × (10^{-9})	1.09 × (10^{-9})	0.68 × (slower, 0.065)	0.62 × (slower, 0.065)
	h^2	1.96 × ($< 10^{-10}$)	1.88 × ($< 10^{-10}$)	2.62 × ($< 10^{-10}$)	3.42 × ($< 10^{-10}$)	2.46 × (10^{-9})	2.39 × (10^{-9})	2.35 × (10^{-9})	2.53 × (10^{-9})	2.08 × (10^{-9})	2.10 × (10^{-9})	1.96 × (10^{-9})	2.12 × (10^{-9})
	10^{-8}	1.66 × ($< 10^{-10}$)	1.27 × ($< 10^{-10}$)	1.75 × ($< 10^{-10}$)	1.60 × ($< 10^{-10}$)	1.71 × (10^{-10})	1.20 × (10^{-9})	1.72 × (10^{-10})	1.53 × (10^{-10})	1.39 × (10^{-9})	1.09 × (0.002)	1.31 × (10^{-9})	1.23 × (0.009)

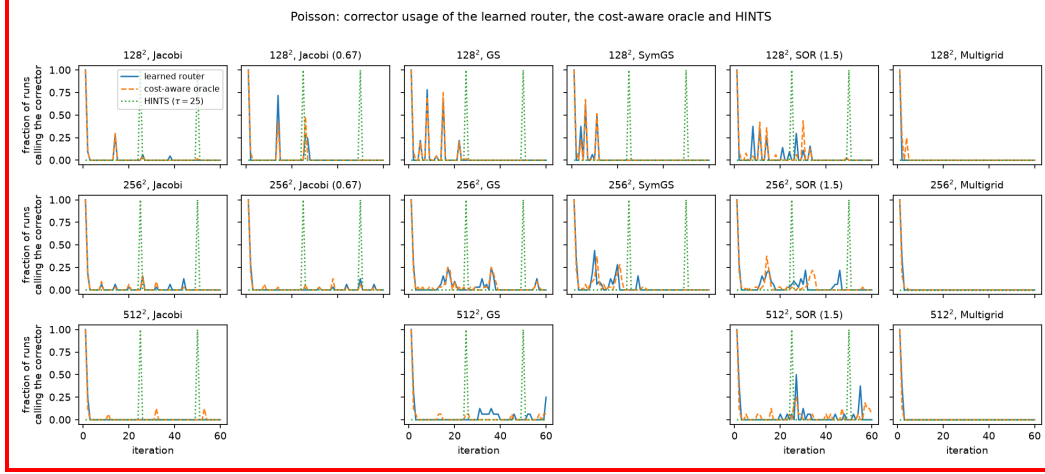


Figure 14: Poisson: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every pairing (columns) and grid (rows; all test instances of each grid).

Table 19: Poisson: median iterations (and corrector calls) to $\varepsilon = h^2$ for HINTS ($\tau=25$), the cost-aware oracle and the learned router on every grid.

Pairing	128 × 128			256 × 256			512 × 512		
	HINTS-25	oracle	router	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	2 (1)	2 (1)	25 (1)	6 (1)	6 (1)	65 (2)	64 (1)	64 (1)
{NO, Jacobi (0.67)}	25 (1)	2 (1)	2 (1)	25 (1)	8 (1)	8 (1)	—	—	—
{NO, GS}	26 (1)	2 (1)	2 (1)	27 (1)	4 (1)	4 (1)	38 (1)	35 (1)	35 (1)
{NO, SymGS}	26 (1)	2 (1)	2 (1)	26 (1)	3 (1)	3 (1)	—	—	—
{NO, SOR (1.5)}	50 (2)	2 (1)	2 (1)	50 (2)	4 (1)	4 (1)	50 (2)	17 (1)	17 (2)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)	3 (0)	2 (1)	2 (1)	4 (0)	2 (1)	3 (1)

Table 20: Poisson: measured cost of one classical iteration c_{solver} and of one corrector call c_{NO} (residual evaluation included), and the resulting macro-action size m_{solver} (sweeps per decision; the corrector is always applied once).

Pairing	128 × 128			256 × 256			512 × 512		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	52 μs	635 μs	12	167 μs	1.00 ms	6	1.07 ms	3.36 ms	3
{NO, Jacobi (0.67)}	52 μs	639 μs	12	150 μs	971 μs	6	—	—	—
{NO, GS}	200 μs	643 μs	3	780 μs	1.01 ms	1	3.50 ms	3.30 ms	1
{NO, SymGS}	366 μs	635 μs	2	1.40 ms	931 μs	1	—	—	—
{NO, SOR (1.5)}	197 μs	635 μs	3	771 μs	960 μs	1	3.41 ms	3.20 ms	1
{NO, Multigrid}	1.25 ms	698 μs	1	4.67 ms	950 μs	1	21.7 ms	2.86 ms	1

Table 21: Poisson, 128×128 : iteration-based metrics of the main text—relative error after $T = 300$ iterations and AUC over the T iterations (mean and standard error over 64 instances; p : one-sided paired t -test that the method’s AUC exceeds the router’s). Bold: router better than both solver-only and HINTS.

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	7.99e-01 (4.5e-03)	2.68e+02 (8.9e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	7.10e-06 (1.9e-06)	2.37e+01 (1.1e-02)	$< 10^{-10}$
Learned router (ours)	6.86e-06 (1.8e-06)	7.15e-03 (1.9e-03)	-
Cost-aware oracle	<i>6.86e-06 (1.8e-06)</i>	<i>7.15e-03 (1.9e-03)</i>	-
{NO, Jacobi (0.67)}			
Solver only	8.56e-01 (3.8e-03)	2.77e+02 (7.1e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	1.15e-12 (1.0e-13)	2.38e+01 (8.3e-03)	$< 10^{-10}$
Learned router (ours)	7.57e-10 (2.9e-11)	2.49e-03 (6.3e-04)	-
Cost-aware oracle	<i>8.19e-10 (1.2e-11)</i>	<i>2.49e-03 (6.3e-04)</i>	-
{NO, GS}			
Solver only	6.53e-01 (5.2e-03)	2.42e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.84e-11 (1.9e-12)	2.35e+01 (2.1e-02)	$< 10^{-10}$
Learned router (ours)	7.66e-10 (1.8e-11)	1.94e-03 (4.9e-04)	-
Cost-aware oracle	<i>7.08e-10 (2.5e-11)</i>	<i>1.94e-03 (4.9e-04)</i>	-
{NO, SymGS}			
Solver only	4.49e-01 (4.6e-03)	2.03e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	7.14e-10 (5.4e-11)	2.30e+01 (3.5e-02)	$< 10^{-10}$
Learned router (ours)	6.50e-10 (2.4e-11)	1.46e-03 (3.7e-04)	-
Cost-aware oracle	<i>6.32e-10 (2.5e-11)</i>	<i>1.46e-03 (3.7e-04)</i>	-
{NO, SOR (1.5)}			
Solver only	3.09e-01 (3.5e-03)	1.72e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.16e-10 (1.3e-11)	2.26e+01 (4.6e-02)	$< 10^{-10}$
Learned router (ours)	8.74e-10 (9.7e-12)	3.30e-03 (8.4e-04)	-
Cost-aware oracle	<i>8.36e-10 (1.5e-11)</i>	<i>3.29e-03 (8.4e-04)</i>	-

Table 22: Poisson, 128×128 : five independent router-training seeds per pairing—mean and standard deviation over seeds of the median work-unit time to h^2 , the share of test instances on which the seed router’s decision sequence to h^2 coincides with the main router’s, the range over seeds of the paired speedup over HINTS ($\tau=25$), and the number of seeds significantly faster ($p < 0.01$) than both HINTS ($\tau=25$) and the best fixed period.

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.74 $\pm$ 0.00 ms	100%	2.59–2.59 $\times$	5/5
{NO, Jacobi (0.67)}	0.75 $\pm$ 0.00 ms	100%	2.58–2.58 $\times$	5/5
{NO, GS}	0.90 $\pm$ 0.00 ms	100%	6.30–6.30 $\times$	5/5
{NO, SymGS}	1.06 $\pm$ 0.00 ms	100%	9.29–9.29 $\times$	5/5
{NO, SOR (1.5)}	0.89 $\pm$ 0.00 ms	100%	10.45–10.45 $\times$	5/5

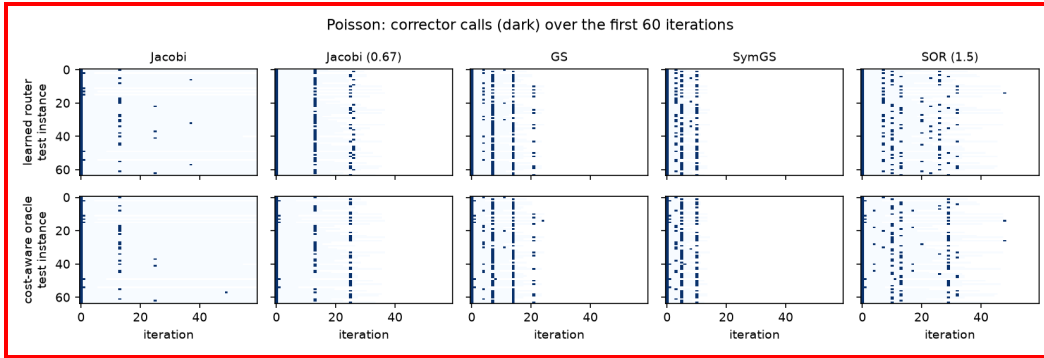


Figure 15: Poisson, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance, for every pairing.

1003 E.3 Convection–diffusion

1004 The convection–diffusion results (Tables 23 to 28 and fig. 16) mirror the Poisson ones: at 128^2 the
 1005 router is the fastest deployable method in every pairing at h^2 and at 10^{-8} , reproduces the cost-aware
 1006 oracle, and is $2\text{--}3\times$ faster than multigrid alone and than multigrid-preconditioned BiCGSTAB; on
 1007 256^2 the same holds in every pairing; at 512^2 the router is faster than HINTS and than the best
 1008 fixed period in the Jacobi, GS and SOR pairings and the {NO, multigrid} router is again the fastest
 1009 iterative method, with the same mechanism as on Poisson (the Jacobi and GS pairings are limited by
 1010 the modes outside the corrector’s band: the router, the oracle and HINTS all reach h^2 after 84–85
 1011 Jacobi iterations). The nonsymmetric operator changes only the details: the best fixed period is $\tau=10$
 1012 rather than 5 in several cells, the undamped-Jacobi pairing stalls at the near-Nyquist band below 10^{-6}
 1013 for every policy, and GMRES(20) is two orders of magnitude slower than the router.

Table 23: Convection–diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids for every pairing and policy, and for the classical baselines without corrector (conventions as in Table 17).

Pairing	Method	128 × 128			256 × 256			512 × 512		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	1.07 s	1.51 s	2.90 s	>12.73 s ¹²⁶	>12.73 s ¹³²	>12.73 s ¹³²	>13.99 s ¹¹⁶	>13.99 s ¹¹⁶	>13.99 s ¹¹⁶
	HINTS ($\tau=25$)	3.41 ms	3.44 ms	384.4 ms	9.15 ms	9.37 ms	500.8 ms	49.2 ms	158.0 ms	1.48 s ¹⁷
	HINTS (best τ)	1.26 ms _{$\tau=5$}	1.28 ms _{$\tau=5$}	321.5 ms _{$\tau=50$}	2.80 ms _{$\tau=5$}	4.95 ms _{$\tau=10$}	475.6 ms _{$\tau=50$}	13.1 ms _{$\tau=5$}	147.4 ms _{$\tau=50$}	1.43 s ¹⁷ _{$\tau=50$}
	Greedy oracle (Alg. 1)	874 μ s	992 μ s	281.0 ms	1.57 ms	4.43 ms	436.3 ms	6.23 ms	156.1 ms	1.37 s ¹⁷
	Cost-aware oracle	979 μ s	1.15 ms	281.8 ms	1.70 ms	4.81 ms	437.5 ms	6.23 ms	154.3 ms	1.31 s ¹⁷
	Learned router (ours)	1.01 ms	1.14 ms	292.7 ms	1.77 ms	4.76 ms	435.1 ms	6.67 ms	144.3 ms	1.29 s¹⁷
{NO, Jacobi (0.67)}	Solver only	1.55 s	2.22 s	4.32 s	>12.55 s ¹³²	>12.55 s ¹³²	>12.55 s ¹³²	–	–	–
	HINTS ($\tau=25$)	3.40 ms	3.40 ms	6.53 ms	9.03 ms	9.31 ms	30.4 ms	–	–	–
	HINTS (best τ)	1.28 ms _{$\tau=5$}	1.31 ms _{$\tau=5$}	3.79 ms _{$\tau=10$}	5.68 ms _{$\tau=5$}	5.68 ms _{$\tau=10$}	30.4 ms _{$\tau=25$}	–	–	–
	Greedy oracle (Alg. 1)	885 μ s	1.10 ms	3.56 ms	1.54 ms	5.93 ms	26.8 ms	–	–	–
	Cost-aware oracle	878 μ s	1.08 ms	3.61 ms	1.58 ms	5.94 ms	27.5 ms	–	–	–
	Learned router (ours)	912 μs	1.10 ms	3.70 ms	1.61 ms	5.76 ms	26.9 ms	–	–	–
{NO, GS}	Solver only	1.15 s	1.63 s	3.14 s	18.36 s	30.05 s	>36.70 s ¹³²	>32.99 s ¹¹⁶	>32.99 s ¹¹⁶	>32.99 s ¹¹⁶
	HINTS ($\tau=25$)	7.03 ms	8.27 ms	20.5 ms	23.5 ms	31.0 ms	59.3 ms	105.1 ms	214.1 ms	504.5 ms
	HINTS (best τ)	1.90 ms _{$\tau=5$}	3.60 ms _{$\tau=5$}	6.24 ms _{$\tau=5$}	5.29 ms _{$\tau=5$}	10.5 ms _{$\tau=5$}	36.7 ms _{$\tau=10$}	22.0 ms _{$\tau=5$}	192.7 ms _{$\tau=10$}	504.5 ms _{$\tau=25$}
	Greedy oracle (Alg. 1)	887 μ s	1.23 ms	5.32 ms	1.57 ms	7.20 ms	31.1 ms	5.70 ms	184.8 ms	493.9 ms
	Cost-aware oracle	909 μ s	1.32 ms	5.46 ms	1.61 ms	7.15 ms	31.0 ms	5.97 ms	185.9 ms	489.9 ms
	Learned router (ours)	906 μs	1.35 ms	5.59 ms	1.57 ms	7.05 ms	31.4 ms	5.79 ms	179.9 ms	486.0 ms
{NO, SymGS}	Solver only	1.10 s	1.56 s	2.99 s	16.90 s	27.77 s	46.85 s	–	–	–
	HINTS ($\tau=25$)	10.7 ms	11.1 ms	21.6 ms	39.2 ms	42.3 ms	78.1 ms	–	–	–
	HINTS (best τ)	2.52 ms _{$\tau=5$}	2.96 ms _{$\tau=5$}	7.35 ms _{$\tau=5$}	7.80 ms _{$\tau=5$}	10.9 ms _{$\tau=5$}	34.8 ms _{$\tau=5$}	–	–	–
	Greedy oracle (Alg. 1)	911 μ s	1.25 ms	4.91 ms	1.58 ms	6.93 ms	28.4 ms	–	–	–
	Cost-aware oracle	905 μ s	1.27 ms	4.48 ms	1.56 ms	6.89 ms	28.4 ms	–	–	–
	Learned router (ours)	924 μs	1.28 ms	4.54 ms	1.55 ms	6.96 ms	29.6 ms	–	–	–
{NO, SOR (1.5)}	Solver only	270.0 ms	383.6 ms	737.6 ms	5.14 s	8.46 s	14.22 s	>32.91 s ¹¹⁶	>32.91 s ¹¹⁶	>32.91 s ¹¹⁶
	HINTS ($\tau=25$)	6.92 ms	13.6 ms	20.2 ms	23.2 ms	46.1 ms	69.0 ms	103.8 ms	206.6 ms	308.3 ms
	HINTS (best τ)	1.98 ms _{$\tau=5$}	3.64 ms _{$\tau=5$}	9.62 ms _{$\tau=5$}	5.22 ms _{$\tau=5$}	10.6 ms _{$\tau=5$}	31.2 ms _{$\tau=5$}	22.1 ms _{$\tau=5$}	89.3 ms _{$\tau=5$}	234.2 ms _{$\tau=10$}
	Greedy oracle (Alg. 1)	919 μ s	1.39 ms	7.66 ms	1.56 ms	5.90 ms	28.1 ms	6.82 ms	83.3 ms	212.8 ms
	Cost-aware oracle	922 μ s	1.63 ms	7.97 ms	1.57 ms	5.74 ms	27.5 ms	5.68 ms	85.0 ms	217.1 ms
	Learned router (ours)	934 μs	1.59 ms	8.18 ms	1.57 ms	5.80 ms	27.6 ms	5.64 ms	83.6 ms	216.7 ms
{NO, Multigrid}	Solver only	3.18 ms	4.71 ms	9.28 ms	10.9 ms	16.4 ms	31.8 ms	49.5 ms	90.8 ms	148.5 ms
	HINTS ($\tau=25$)	3.17 ms	4.68 ms	9.28 ms	10.9 ms	16.7 ms	31.6 ms	50.0 ms	97.6 ms	150.4 ms
	HINTS (best τ)	3.17 ms _{$\tau=25$}	4.66 ms _{$\tau=5$}	7.14 ms _{$\tau=5$}	10.9 ms _{$\tau=5$}	16.5 ms _{$\tau=5$}	23.0 ms _{$\tau=5$}	49.4 ms _{$\tau=10$}	92.5 ms _{$\tau=10$}	112.8 ms _{$\tau=5$}
	Greedy oracle (Alg. 1)	896 μ s	2.34 ms	5.57 ms	1.58 ms	7.30 ms	20.8 ms	6.18 ms	50.7 ms	103.5 ms
	Cost-aware oracle	851 μ s	2.33 ms	5.49 ms	1.55 ms	7.23 ms	19.9 ms	5.74 ms	50.9 ms	104.1 ms
	Learned router (ours)	873 μs	2.32 ms	5.50 ms	1.60 ms	7.26 ms	20.2 ms	5.63 ms	53.7 ms	102.6 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>										
{NO, Multigrid}	Multigrid V(2,2) alone	3.06 ms	4.52 ms	8.92 ms	12.3 ms	18.8 ms	36.1 ms	52.4 ms	99.7 ms	150.7 ms
	BiCGSTAB	56.0 ms	67.6 ms	93.1 ms	377.7 ms	486.6 ms	605.9 ms	3.79 s	5.56 s	6.72 s ¹¹
	BiCGSTAB (multigrid)	3.12 ms	6.17 ms	9.21 ms	12.5 ms	24.0 ms	36.1 ms	46.1 ms	93.5 ms	144.8 ms
	GMRES(20)	301.7 ms	434.1 ms	882.8 ms	1.85 s	3.00 s	5.14 s	–	–	–

Table 24: Convection–diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), the best fixed period, multigrid alone and multigrid-preconditioned BiCGSTAB, with one-sided Wilcoxon p -values (conventions as in Table 18).

Pairing	ε	128 × 128			256 × 256			512 × 512		
		vs. HINTS-25	vs. best τ	vs. multigrid	vs. HINTS-25	vs. best τ	vs. multigrid	vs. HINTS-25	vs. best τ	vs. multigrid
{NO, Jacobi}	h^2	2.95 × (10^{-10})	1.34 × (10^{-10})	3.54 × (10^{-11})	5.18 × (10^{-11})	1.92 × (10^{-11})	1.30 × (10^{-11})	4.60 × (0.000)	5.06 × (0.005)	1.17 × (0.003)
	10^{-8}	1.33 × (10^{-10})	1.17 × (10^{-10})	0.03 × (slower, 10^{-10})	0.03 × (slower, 10^{-10})	0.03 × (slower, 10^{-10})	0.08 × (slower, 10^{-10})	0.08 × (slower, 0.990)	0.98 × (slower, 0.963)	0.22 × (slower, 0.006)
{NO, Jacobi (0.67)}	h^2	3.14 × (10^{-10})	1.32 × (10^{-10})	3.71 × (10^{-10})	3.21 × (10^{-10})	1.80 × (10^{-10})	1.25 × (10^{-10})	3.85 × (10^{-10})	4.42 × (10^{-10})	–
	10^{-8}	1.77 × (10^{-10})	1.07 × (10^{-10})	2.37 × (10^{-10})	2.56 × (10^{-10})	1.06 × (0.002)	1.06 × (0.002)	1.27 × (10^{-10})	1.26 × (10^{-10})	–
{NO, GS}	h^2	6.76 × (10^{-10})	2.60 × (10^{-10})	3.29 × (10^{-10})	4.42 × (10^{-10})	3.52 × (10^{-10})	1.83 × (10^{-10})	3.08 × (10^{-10})	3.47 × (10^{-10})	–
	10^{-8}	3.78 × (10^{-10})	1.17 × (10^{-10})	1.40 × (10^{-10})	1.91 × (10^{-10})	1.05 × (0.32)	1.17 × (10^{-10})	1.16 × (0.002)	1.19 × (0.003)	–
{NO, SymGS}	h^2	8.70 × (10^{-10})	2.40 × (10^{-10})	3.46 × (10^{-10})	4.73 × (10^{-10})	6.26 × (10^{-10})	1.69 × (10^{-10})	3.15 × (10^{-10})	3.54 × (10^{-10})	–
	10^{-8}	4.75 × (10^{-10})	1.62 × (10^{-10})	1.91 × (10^{-10})	2.08 × (10^{-10})	2.63 × (10^{-10})	1.18 × (10^{-10})	1.24 × (10^{-10})	1.24 × (10^{-10})	–
{NO, SOR (1.5)}	h^2	8.89 × (10^{-10})	2.40 × (10^{-10})	3.07 × (10^{-10})	3.89 × (10^{-10})	7.99 × (10^{-10})	1.06 × (10^{-10})	3.72 × (10^{-10})	4.15 × (10^{-10})	–
	10^{-8}	2.58 × (10^{-10})	1.31 × (10^{-10})	1.16 × (10^{-10})	1.27 × (10^{-10})	2.50 × (10^{-10})	1.19 × (10^{-10})	1.32 × (10^{-10})	1.29 × (10^{-10})	–
{NO, Multigrid}	h^2	2.07 × (10^{-10})	2.01 × (10^{-10})	2.12 × (10^{-10})	2.84 × (10^{-10})	2.19 × (10^{-10})	2.20 × (10^{-10})	2.56 × (10^{-10})	3.27 × (10^{-10})	–
	10^{-8}	1.70 × (10^{-10})	1.28 × (10^{-10})	1.65 × (10^{-10})	1.74 × (10^{-10})	1.54 × (10^{-10})	1.14 × (10^{-10})	1.70 × (10^{-10})	1.81 × (10^{-10})	–

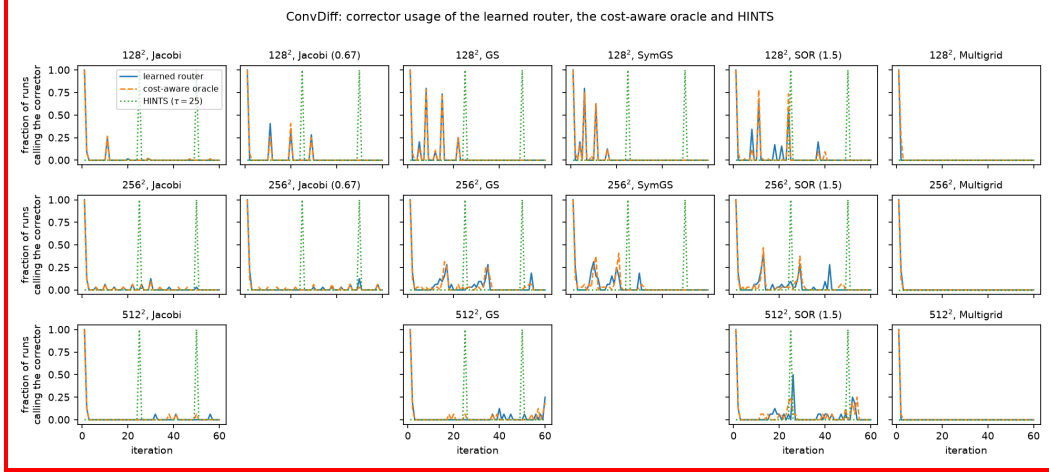


Figure 16: Convection–diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 25: Convection–diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$ on every grid.

Pairing	128 × 128			256 × 256			512 × 512		
	HINTS-25	oracle	router	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	2 (1)	2 (1)	25 (1)	10 (1)	10 (1)	85 (2)	84 (1)	84 (1)
{NO, Jacobi (0.67)}	25 (1)	4 (1)	4 (1)	25 (1)	15 (1)	15 (1)	—	—	—
{NO, GS}	30 (1)	2 (1)	2 (1)	33 (1)	7 (1)	7 (1)	51 (2)	44 (1)	44 (2)
{NO, SymGS}	26 (1)	2 (1)	2 (1)	27 (1)	4 (1)	4 (1)	—	—	—
{NO, SOR (1.5)}	50 (2)	4 (1)	4 (1)	50 (2)	6 (1)	6 (1)	50 (2)	20 (1)	20 (2)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)	3 (0)	2 (1)	2 (1)	4 (0)	3 (1)	3 (1)

Table 26: Convection–diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128			256 × 256			512 × 512		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	72 μs	658 μs	9	286 μs	1.13 ms	4	1.42 ms	3.61 ms	3
{NO, Jacobi (0.67)}	71 μs	659 μs	9	288 μs	1.15 ms	4	—	—	—
{NO, GS}	234 μs	673 μs	3	855 μs	1.08 ms	1	3.94 ms	3.67 ms	1
{NO, SymGS}	378 μs	665 μs	2	1.55 ms	1.11 ms	1	—	—	—
{NO, SOR (1.5)}	223 μs	660 μs	3	813 μs	1.03 ms	1	3.90 ms	3.62 ms	1
{NO, Multigrid}	1.47 ms	737 μs	1	5.68 ms	1.06 ms	1	24.7 ms	3.60 ms	1

Table 27: Convection–diffusion, 128×128 : iteration-based metrics (conventions as in Table 21).

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	7.43e-01 (9.1e-03)	2.57e+02 (1.9e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	1.55e-05 (4.2e-06)	2.36e+01 (2.7e-02)	$< 10^{-10}$
Learned router (ours)	1.50e-05 (4.1e-06)	1.55e-02 (4.1e-03)	-
Cost-aware oracle	1.50e-05 (4.1e-06)	1.55e-02 (4.1e-03)	-
{NO, Jacobi (0.67)}			
Solver only	8.11e-01 (7.9e-03)	2.69e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	1.46e-12 (2.7e-13)	2.37e+01 (2.1e-02)	$< 10^{-10}$
Learned router (ours)	7.96e-10 (1.6e-11)	5.39e-03 (1.4e-03)	-
Cost-aware oracle	8.18e-10 (1.3e-11)	5.39e-03 (1.4e-03)	-
{NO, GS}			
Solver only	5.46e-01 (9.9e-03)	2.20e+02 (2.4e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	8.78e-11 (4.8e-12)	2.32e+01 (5.4e-02)	$< 10^{-10}$
Learned router (ours)	6.84e-10 (2.8e-11)	4.03e-03 (1.0e-03)	-
Cost-aware oracle	6.73e-10 (2.8e-11)	4.03e-03 (1.0e-03)	-
{NO, SymGS}			
Solver only	3.77e-01 (9.1e-03)	1.84e+02 (2.7e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	3.64e-12 (2.3e-13)	2.26e+01 (8.1e-02)	$< 10^{-10}$
Learned router (ours)	5.19e-10 (3.4e-11)	3.16e-03 (8.0e-04)	-
Cost-aware oracle	5.00e-10 (3.2e-11)	3.16e-03 (8.0e-04)	-
{NO, SOR (1.5)}			
Solver only	1.23e-01 (3.4e-03)	1.17e+02 (1.9e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	8.06e-10 (2.3e-11)	2.12e+01 (1.2e-01)	$< 10^{-10}$
Learned router (ours)	8.08e-10 (1.3e-11)	6.49e-03 (1.7e-03)	-
Cost-aware oracle	7.77e-10 (1.8e-11)	6.49e-03 (1.7e-03)	-

Table 28: Convection–diffusion, 128×128 : five-seed router-training trials (conventions as in Table 22).

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.84 ± 0.00 ms	100%	$2.91\text{--}2.91\times$	5/5
{NO, Jacobi (0.67)}	0.91 ± 0.00 ms	100%	$2.67\text{--}2.67\times$	5/5
{NO, GS}	1.11 ± 0.00 ms	100%	$7.22\text{--}7.22\times$	5/5
{NO, SymGS}	1.12 ± 0.00 ms	100%	$9.10\text{--}9.10\times$	5/5
{NO, SOR (1.5)}	1.30 ± 0.00 ms	100%	$9.41\text{--}9.41\times$	5/5

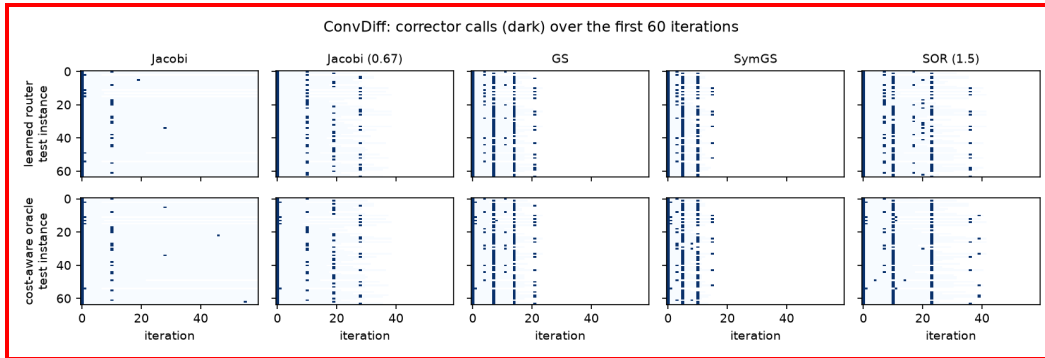


Figure 17: Convection–diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1014 E.4 Anisotropic diffusion

1015 The anisotropic equation is where the two components of the hybrid are most complementary and
1016 where the classical solvers are weakest: the point smoothers cannot damp the x_1 -oscillatory modes

at all (Table 29: none of the five solvers alone reaches h^2 within the iteration cap on either grid), whereas the corrector, whose sensor grid keeps full resolution along x_1 , removes exactly those modes together with the smooth ones in a single call and leaves the smoother only the x_2 -oscillatory band, which it damps at the isotropic rate. Consequently the learned router reaches h^2 in under 1 ms at 128^2 for every pairing—the same absolute time as on the isotropic equations—while HINTS ($\tau=25$) needs 3–10 ms and the best fixed period 1.2–2.4 ms; at 256^2 (Jacobi, GS and SOR pairings, 16 instances) the router needs 9–10 ms against 15–30 ms for HINTS and 10–13 ms for the best period (Table 30). The router again matches the cost-aware oracle, its decisions are identical across the five training seeds (Table 34), and it spends the iterations after the single corrector call on the x_2 -oscillatory band (Figure 18 and table 31). The multigrid and Krylov baselines are not run for this equation (Section E.1).

Table 29: Anisotropic diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 (64 instances, five pairings) and 256^2 (16 instances; Jacobi, GS and SOR pairings) grids for every policy (conventions as in Table 17; the solver-only runs are capped at 60,000 and 40,000 iterations and never reach h^2).

Pairing	Method	128 × 128			256 × 256		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	>4.62 s ¹⁶⁴	>4.62 s ¹⁶⁴	>4.62 s ¹⁶⁴	>9.29 s ¹¹⁶	>9.29 s ¹¹⁶	>9.29 s ¹¹⁶
	HINTS ($\tau=25$)	2.90 ms	2.90 ms	67.6 ms	14.9 ms	14.9 ms	127.4 ms
	HINTS (best τ)	1.15 ms $_{\tau=5}$	1.15 ms $_{\tau=5}$	58.8 ms $_{\tau=50}$	10.2 ms $_{\tau=5}$	10.3 ms $_{\tau=5}$	94.6 ms $_{\tau=50}$
	Greedy oracle (Alg. 1)	801 μ s	826 μ s	49.6 ms	9.19 ms	9.29 ms	65.1 ms
	Cost-aware oracle	876 μ s	892 μ s	49.9 ms	9.26 ms	9.42 ms	65.3 ms
	Learned router (ours)	903 μs	911 μs	55.6 ms	9.35 ms	9.55 ms	65.6 ms
{NO, Jacobi (0.67)}	Solver only	>4.52 s ¹⁶⁴	>4.52 s ¹⁶⁴	>4.52 s ¹⁶⁴	—	—	—
	HINTS ($\tau=25$)	2.85 ms	2.85 ms	5.46 ms	—	—	—
	HINTS (best τ)	1.16 ms $_{\tau=5}$	1.18 ms $_{\tau=5}$	4.21 ms $_{\tau=10}$	—	—	—
	Greedy oracle (Alg. 1)	787 μ s	810 μ s	3.70 ms	—	—	—
	Cost-aware oracle	802 μ s	823 μ s	3.81 ms	—	—	—
	Learned router (ours)	831 μs	850 μs	3.86 ms	—	—	—
{NO, GS}	Solver only	>13.63 s ¹⁶⁴	>13.63 s ¹⁶⁴	>13.63 s ¹⁶⁴	>34.18 s ¹¹⁶	>34.18 s ¹¹⁶	>34.18 s ¹¹⁶
	HINTS ($\tau=25$)	6.48 ms	6.48 ms	23.7 ms	29.7 ms	29.7 ms	64.5 ms
	HINTS (best τ)	1.73 ms $_{\tau=5}$	1.74 ms $_{\tau=5}$	16.5 ms $_{\tau=5}$	12.7 ms $_{\tau=5}$	12.8 ms $_{\tau=5}$	37.8 ms $_{\tau=10}$
	Greedy oracle (Alg. 1)	812 μ s	831 μ s	5.87 ms	9.24 ms	9.40 ms	38.7 ms
	Cost-aware oracle	802 μ s	833 μ s	5.33 ms	9.20 ms	9.32 ms	35.1 ms
	Learned router (ours)	827 μs	852 μs	5.40 ms	9.20 ms	9.30 ms	35.9 ms
{NO, SymGS}	Solver only	>23.15 s ¹⁶⁴	>23.15 s ¹⁶⁴	>23.15 s ¹⁶⁴	—	—	—
	HINTS ($\tau=25$)	10.3 ms	10.3 ms	20.5 ms	—	—	—
	HINTS (best τ)	2.40 ms $_{\tau=5}$	2.40 ms $_{\tau=5}$	7.45 ms $_{\tau=5}$	—	—	—
	Greedy oracle (Alg. 1)	811 μ s	832 μ s	5.63 ms	—	—	—
	Cost-aware oracle	826 μ s	844 μ s	5.59 ms	—	—	—
	Learned router (ours)	856 μs	871 μs	5.47 ms	—	—	—
{NO, SOR (1.5)}	Solver only	>13.80 s ¹⁶⁴	>13.80 s ¹⁶⁴	>13.80 s ¹⁶⁴	>33.90 s ¹¹⁶	>33.90 s ¹¹⁶	>33.90 s ¹¹⁶
	HINTS ($\tau=25$)	6.62 ms	6.62 ms	17.9 ms	29.6 ms	29.7 ms	60.2 ms
	HINTS (best τ)	1.78 ms $_{\tau=5}$	1.80 ms $_{\tau=5}$	12.8 ms $_{\tau=10}$	12.7 ms $_{\tau=5}$	12.8 ms $_{\tau=5}$	51.7 ms $_{\tau=10}$
	Greedy oracle (Alg. 1)	813 μ s	843 μ s	7.60 ms	9.21 ms	9.33 ms	43.3 ms
	Cost-aware oracle	823 μ s	862 μ s	6.71 ms	9.12 ms	9.27 ms	34.2 ms
	Learned router (ours)	845 μs	869 μs	7.68 ms	9.18 ms	9.93 ms	44.1 ms

Table 30: Anisotropic diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$) and the best fixed period, with one-sided Wilcoxon p -values (no classical baselines without corrector are run for this equation).

Pairing	ε	128 × 128				256 × 256			
		vs. HINTS-25	vs. best τ	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. multigrid	vs. MG-Krylov
{NO, Jacobi}	h^2	3.16 × ($< 10^{-10}$)	1.25 × (10^{-9})	—	—	1.58 × (10^{-5})	1.11 × (10^{-4})	—	—
	10^{-8}	1.44 × ($< 10^{-10}$)	1.21 × ($< 10^{-10}$)	—	—	1.93 × (10^{-5})	1.80 × (10^{-5})	—	—
{NO, Jacobi (0.67)}	h^2	3.31 × ($< 10^{-10}$)	1.34 × ($< 10^{-10}$)	—	—	—	—	—	—
	10^{-8}	1.43 × ($< 10^{-10}$)	1.17 × (10^{-9})	—	—	—	—	—	—
{NO, GS}	h^2	7.61 × ($< 10^{-10}$)	1.99 × ($< 10^{-10}$)	—	—	3.18 × (10^{-5})	1.37 × (10^{-5})	—	—
	10^{-8}	4.05 × ($< 10^{-10}$)	2.74 × ($< 10^{-10}$)	—	—	2.26 × (10^{-5})	1.64 × (10^{-5})	—	—
{NO, SymGS}	h^2	11.8 × ($< 10^{-10}$)	2.72 × ($< 10^{-10}$)	—	—	—	—	—	—
	10^{-8}	3.71 × ($< 10^{-10}$)	1.36 × ($< 10^{-10}$)	—	—	—	—	—	—
{NO, SOR (1.5)}	h^2	7.36 × ($< 10^{-10}$)	2.04 × ($< 10^{-10}$)	—	—	2.98 × (10^{-5})	1.38 × (10^{-5})	—	—
	10^{-8}	2.40 × ($< 10^{-10}$)	1.82 × ($< 10^{-10}$)	—	—	1.54 × (10^{-5})	1.35 × (10^{-5})	—	—

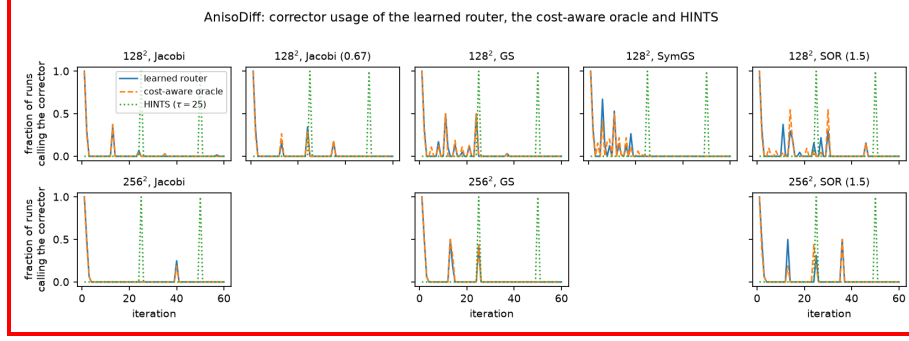


Figure 18: Anisotropic diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 31: Anisotropic diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

Pairing	128 × 128			256 × 256		
	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	25 (1)	1 (1)	1 (1)	—	—	—
{NO, GS}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)
{NO, SymGS}	25 (1)	1 (1)	1 (1)	—	—	—
{NO, SOR (1.5)}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)

Table 32: Anisotropic diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128			256 × 256		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	63 μs	695 μs	11	235 μs	8.89 ms	38
{NO, Jacobi (0.67)}	64 μs	692 μs	11	—	—	—
{NO, GS}	215 μs	711 μs	3	826 μs	8.91 ms	11
{NO, SymGS}	382 μs	711 μs	2	—	—	—
{NO, SOR (1.5)}	219 μs	730 μs	3	777 μs	8.80 ms	11

Table 33: Anisotropic diffusion, 128×128 : iteration-based metrics (conventions as in Table 21).

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	9.96e-01 (1.5e-04)	2.99e+02 (2.3e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	7.65e-07 (2.1e-07)	2.40e+01 (1.7e-04)	$< 10^{-10}$
Learned router (ours)	7.45e-07 (2.0e-07)	6.80e-04 (1.8e-04)	-
Cost-aware oracle	7.45e-07 (2.0e-07)	6.80e-04 (1.8e-04)	-
{NO, Jacobi (0.67)}			
Solver only	9.97e-01 (1.0e-04)	3.00e+02 (1.6e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	3.34e-10 (4.6e-11)	2.40e+01 (1.2e-04)	$< 10^{-10}$
Learned router (ours)	8.97e-10 (7.4e-12)	2.82e-04 (6.6e-05)	-
Cost-aware oracle	8.96e-10 (7.4e-12)	2.82e-04 (6.6e-05)	-
{NO, GS}			
Solver only	9.92e-01 (2.7e-04)	2.99e+02 (4.3e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	9.79e-10 (1.4e-12)	2.40e+01 (3.4e-04)	$< 10^{-10}$
Learned router (ours)	8.15e-10 (1.3e-11)	1.95e-04 (4.6e-05)	-
Cost-aware oracle	8.02e-10 (1.5e-11)	1.95e-04 (4.6e-05)	-
{NO, SymGS}			
Solver only	9.84e-01 (4.8e-04)	2.98e+02 (7.7e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	7.48e-10 (3.2e-11)	2.40e+01 (6.5e-04)	$< 10^{-10}$
Learned router (ours)	7.53e-10 (2.2e-11)	1.23e-04 (2.9e-05)	-
Cost-aware oracle	7.54e-10 (1.8e-11)	1.23e-04 (2.9e-05)	-
{NO, SOR (1.5)}			
Solver only	9.76e-01 (6.8e-04)	2.96e+02 (1.1e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	9.09e-10 (1.6e-11)	2.40e+01 (9.5e-04)	$< 10^{-10}$
Learned router (ours)	8.67e-10 (9.3e-12)	2.87e-04 (6.9e-05)	-
Cost-aware oracle	8.63e-10 (9.4e-12)	2.87e-04 (6.9e-05)	-

Table 34: Anisotropic diffusion, 128×128 : five-seed router-training trials (conventions as in Table 22).

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.76 $\pm$ 0.00 ms	100%	2.97–2.97 $\times$	5/5
{NO, Jacobi (0.67)}	0.76 $\pm$ 0.00 ms	100%	3.01–3.01 $\times$	5/5
{NO, GS}	0.78 $\pm$ 0.00 ms	100%	7.63–7.63 $\times$	5/5
{NO, SymGS}	0.78 $\pm$ 0.00 ms	100%	12.77–12.77 $\times$	5/5
{NO, SOR (1.5)}	0.80 $\pm$ 0.00 ms	100%	7.58–7.58 $\times$	5/5

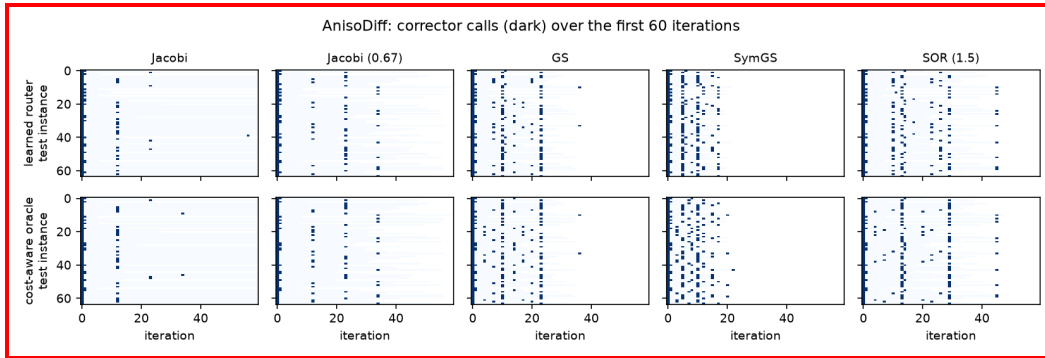


Figure 19: Anisotropic diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1028 E.5 Solver ensembles

1029 **When can a second classical solver help?** The cost-aware oracle is a per-decision comparison of
 1030 macro-actions, so a router over $\text{NO} \cup \mathcal{W}$ can only beat the best pairwise router of its members if,

1031 along the way to the tolerance, the error passes through regimes in which *different* members give the
1032 largest reduction per unit cost. We screened this ceiling exhaustively before training any router: for
1033 every subset $\mathcal{W}$ of up to three members of {Jacobi, Jacobi (0.67), GS, SymGS, SOR (1.5), multigrid}
1034 (multigrid on the isotropic equations only), every equation and grid, we ran the cost-aware oracle over
1035 $\text{NO} \cup \mathcal{W}$ on the test instances and compared its work-unit time to that of the oracle over each member
1036 alone (Table 35). Two facts emerge. At $\varepsilon = h^2$ no ensemble beats the best single solver anywhere:
1037 one corrector call followed by one or two macro-actions of the cheapest smoother already reaches
1038 truncation-level accuracy, so there is no second regime to exploit. At $\varepsilon = 10^{-8}$, after the corrector
1039 has removed the smooth band, the smoother has to damp the whole band $|k| > 31$, whose sub-bands
1040 favour different solvers: undamped Jacobi is the cheapest per unit cost on the modes near the band
1041 edge (it contracts them by $\cos(2\pi \cdot 32/N)$ -like factors that damped Jacobi cannot match), but it does
1042 not damp the near-Nyquist modes at all, which damped Jacobi and GS remove in a few sweeps. The
1043 oracle over {Jacobi, Jacobi (0.67)}—a *nonstationary* Jacobi iteration whose damping parameter is
1044 chosen per macro-action—is therefore faster than every single solver, by 13–28% on four of the six
1045 (equation, grid) settings at 128^2 and 256^2 , and adding a third member (GS) changes nothing, i.e., the
1046 gain saturates. At 512^2 multigrid dominates every subset (its cycle removes all sub-bands at once), so
1047 the oracle over any ensemble containing it equals the oracle over {NO, multigrid}: adding members
1048 neither helps nor hurts. The “difficulty” that matters for ensembles is thus not the equation but the
1049 width of the spectrum left to the classical members after the corrector, which grows with the grid
1050 (fixed band) and with the tolerance.

1051 **Nested ensembles.** Table 36 evaluates the learned router over the nested sets {Jacobi},
1052 {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} on every equation at 128^2
1053 and 256^2 ; the single-member routers are the pairwise routers of Sections E.2 to E.4, evaluated in the
1054 same session on the same instances, and all members keep the macro-action sizes of their pairwise
1055 runs. The table reports live and work-unit times to h^2 , 10^{-6} and 10^{-8} for the router and its oracle,
1056 and the paired speedup at 10^{-8} over the best single-member router with its p -value. Two properties
1057 are visible. *Monotonicity*: the router over a larger set is never slower than the best of its members
1058 beyond timer noise, because the oracle ignores a member that never gives the largest reduction per
1059 unit cost and the router imitates it (adding GS to {Jacobi, Jacobi (0.67)} leaves the decision sequence
1060 unchanged); the earlier ensembles of Table 37 show the same for four- and five-member sets at h^2 .
1061 *Strict separation*: at 10^{-8} the two-member router is significantly faster than every single-member
1062 router in 0 of the 2 ensemble cells (paired Wilcoxon $p < 0.01$; speedups $0.89\times$ – $1.00\times$ over the
1063 best single solver), reproducing its oracle, and it is never significantly slower. The learned policy
1064 is the nonstationary damping schedule described above: undamped Jacobi macro-actions while the
1065 band-edge modes dominate the residual, damped Jacobi once the near-Nyquist modes do, which the
1066 router detects from the decay rate of the residual over the last macro-action.

Table 35: Oracle-level screening of ensembles: for every equation and grid, the fastest single solver and the fastest ensemble of up to three members under the cost-aware oracle (median work-unit time over 16 test instances), at $\varepsilon = h^2$ and 10^{-8} ; the ratio is the speedup of the ensemble over the best single solver, and the last column is the ratio for {Jacobi, Jacobi (0.67)}.

Equation	N	$\varepsilon = h^2$			$\varepsilon = 10^{-8}$			{Jacobi, Jacobi (0.67)}
		best single	best ensemble	ratio	best single	best ensemble	ratio	
Poisson	128^2	Jacobi (772 μ s)	{Jacobi, GS} (772 μ s)	1.00 $\times$	Jacobi (0.67) (2.51 ms)	{Jacobi, Jacobi(0.67)} (2.45 ms)	1.02 $\times$	1.02 $\times$
	256^2	Jacobi (2.86 ms)	{Jacobi, Jacobi(0.67)} (2.86 ms)	1.00 $\times$	Jacobi (0.67) (14.2 ms)	{Jacobi, Jacobi(0.67)} (11.1 ms)	1.28 $\times$	1.28 $\times$
	512^2	Multigrid (36.5 ms)	{Jacobi, Multigrid} (36.5 ms)	1.00 $\times$	Multigrid (90.8 ms)	{Jacobi, Multigrid} (90.8 ms)	1.00 $\times$	–
Convection–diffusion	128^2	Jacobi (923 μ s)	{Jacobi, GS} (923 μ s)	1.00 $\times$	Jacobi (0.67) (3.06 ms)	{Jacobi, Jacobi(0.67)} (2.71 ms)	1.13 $\times$	1.13 $\times$
	256^2	Jacobi (5.49 ms)	{Jacobi, Jacobi(0.67)} (5.49 ms)	1.00 $\times$	Multigrid (24.1 ms)	{Jacobi, Jacobi(0.67)} (19.3 ms)	1.25 $\times$	1.25 $\times$
	512^2	Multigrid (54.3 ms)	{Jacobi, Multigrid} (54.3 ms)	1.00 $\times$	Multigrid (103.7 ms)	{Jacobi, Multigrid} (103.7 ms)	1.00 $\times$	–
Anisotropic diffusion	128^2	Jacobi (765 μ s)	{Jacobi, Jacobi(0.67)} (765 μ s)	1.00 $\times$	Jacobi (0.67) (3.58 ms)	{Jacobi, Jacobi(0.67)} (3.06 ms)	1.17 $\times$	1.17 $\times$

Table 36: Nested ensembles: median time to $\varepsilon \in \{h^2, 10^{-6}, 10^{-8}\}$ of the learned router (live and work units) and of the cost-aware oracle (work units) over $\text{NO} \cup \mathcal{W}$, for the single members and the nested two- and three-member sets, all evaluated in one session on the same instances (64 at 128^2 , 32 at 256^2 , 16 for anisotropic diffusion at 256^2). The last columns give the paired speedup at 10^{-8} over the best single-member router (resp. oracle) with the one-sided Wilcoxon p -value; bold: $\geq 1.1 \times$ with $p < 0.01$.

Equation	N	$\mathcal{W}$	$\varepsilon = h^2$			$\varepsilon = 10^{-6}$			$\varepsilon = 10^{-8}$			$\varepsilon = 10^{-8}$: vs. best single solver	
			router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	oracle
Poisson	128^2	$\{\text{Jacobi}\}$	1.28 ms	743 μs	732 μs	4.38 ms	2.21 ms	2.19 ms	256.9 ms ¹¹⁸	116.1 ms ¹¹⁸	115.2 ms	–	–
		$\{\text{Jacobi}(0.67)\}$	1.50 ms	748 μs	737 μs	2.18 ms	1.16 ms	1.15 ms	4.37 ms	2.33 ms	2.31 ms	(best single)	–
		$\{\text{GS}\}$	1.42 ms	901 μs	891 μs	3.38 ms	2.56 ms	2.53 ms	6.17 ms	4.41 ms	4.37 ms	–	–
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	1.42 ms	745 μs	735 μs	2.60 ms	1.32 ms	1.15 ms	5.46 ms	2.58 ms	2.56 ms	$1.00 \times (0.991)$	$1.00 \times (10^{-5})$
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	1.37 ms	896 μs	738 μs	2.01 ms	1.37 ms	1.15 ms	4.46 ms	2.59 ms	2.57 ms	$0.89 \times (\text{slower}, 10^{-5})$	$1.00 \times (10^{-5})$

1067 **Larger ensembles at truncation level.** Table 37 repeats the ensembles of Section 7 ($\{\text{Jacobi}, \text{GS}\}$
1068 up to all five solvers) at 128^2 and $\varepsilon = h^2$, where the screening predicts no gain: the ensemble router
1069 matches the best pairwise router of its members to within about $\pm 20\%$ in live time (ratio of medians
1070 $0.82 \times - 1.14 \times$ over the 12 cells) and within 1.5% in work units, without knowing in advance which
1071 member is fastest, and is $380 \times - > 10^3 \times$ faster than the fastest member alone; Table 38 lists how its
1072 iterations are distributed over the members. Its oracle is up to 25% faster in a few cells because, at h^2 ,
1073 several cost-equalised smoothers are near-ties and the oracle’s choice among them depends on the
1074 spectral content of the current error, which the router’s residual-norm features reflect only indirectly
1075 (agreement with the oracle 55–83% on its own rollouts against 88–99% for pairwise routers). More
1076 capacity or imitation data does not change this: routers with twice the hidden width, three times
1077 the DAGger rounds and twice the oracle rollouts (Table 40) take the same number of iterations and
1078 corrector calls to h^2 as the default routers on 53%–91% of the test instances and are significantly
1079 faster in 2 and significantly slower in 3 of the 8 cells ($p < 0.05$); the residual disagreements are
1080 near-ties whose resolution does not affect the time to h^2 . Deciding *when* to call the corrector—the
1081 decision that dominates the wall-clock gains—is learned equally well in the ensembles.

Table 37: Ensembles of Section 7 at 128×128 and $\varepsilon = h^2$: median live time of the best member solver alone, of the best pairwise router among the members (paired speedup of the ensemble router over it in parentheses), and of the router and the cost-aware oracle over $\text{NO} \cup \mathcal{W}$ (64 instances; pairwise baselines from the pairwise runs on the same instances).

Equation	$\mathcal{W}$	Best solver only	Best pairwise router	Router($\text{NO} \cup \mathcal{W}$)	Oracle($\text{NO} \cup \mathcal{W}$)
Poisson	$\{\text{Jacobi}, \text{GS}\}$	1.15 s (Jacobi)	987 μs (Jacobi)	964 μs (1.09 $\times$ vs pairwise)	834 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	1.15 s (Jacobi)	987 μs (Jacobi)	951 μs (1.07 $\times$ vs pairwise)	854 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	1.15 s (Jacobi)	912 μs (Jacobi (0.67))	868 μs (0.98 $\times$ vs pairwise)	842 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	600.9 ms (SOR (1.5))	912 μs (Jacobi (0.67))	1.12 ms (0.86 $\times$ vs pairwise)	900 μs
	$\{\text{Jacobi}, \text{GS}\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.25 ms (1.09 $\times$ vs pairwise)	974 μs
ConvDiff	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.01 ms (1.16 $\times$ vs pairwise)	996 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	1.51 s (Jacobi)	1.10 ms (Jacobi (0.67))	996 μs (1.00 $\times$ vs pairwise)	1.07 ms
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	383.6 ms (SOR (1.5))	1.10 ms (Jacobi (0.67))	1.01 ms (1.06 $\times$ vs pairwise)	1.08 ms
	$\{\text{Jacobi}, \text{GS}\}$	>4.62 s ¹ (none)	852 μs (GS)	898 μs (1.01 $\times$ vs pairwise)	870 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	>4.62 s ¹ (none)	852 μs (GS)	880 μs (1.03 $\times$ vs pairwise)	857 μs
AnisoDiff	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	>4.52 s ¹ (none)	850 μs (Jacobi (0.67))	880 μs (1.00 $\times$ vs pairwise)	858 μs
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	>4.52 s ¹ (none)	850 μs (Jacobi (0.67))	890 μs (1.00 $\times$ vs pairwise)	863 μs

Table 38: Ensembles at 128×128 : mean (standard deviation over instances) fraction of the iterations up to the h^2 crossing that the ensemble router spends in each member and in the corrector.

Equation	$\mathcal{W}$	Jacobi	GS	SymGS	Jacobi (0.67)	SOR (1.5)	DeepONet
Poisson	$\{\text{Jacobi}, \text{GS}\}$	0.228 (0.365)	0.144 (0.216)	–	–	–	0.628 (0.386)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	0.237 (0.337)	0.043 (0.134)	0.086 (0.165)	–	–	0.634 (0.378)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	0.235 (0.297)	0.008 (0.062)	0.101 (0.207)	0.000 (0.000)	–	0.656 (0.352)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	0.094 (0.195)	0.020 (0.051)	0.182 (0.265)	0.046 (0.125)	0.007 (0.023)	0.650 (0.366)
	$\{\text{Jacobi}, \text{GS}\}$	0.089 (0.255)	0.310 (0.337)	–	–	–	0.600 (0.373)
ConvDiff	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	0.279 (0.354)	0.067 (0.156)	0.055 (0.103)	–	–	0.599 (0.374)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	0.305 (0.340)	0.000 (0.000)	0.022 (0.038)	0.110 (0.214)	–	0.563 (0.410)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	0.166 (0.302)	0.028 (0.127)	0.000 (0.000)	0.233 (0.375)	0.000 (0.000)	0.573 (0.399)
	$\{\text{Jacobi}, \text{GS}\}$	0.173 (0.320)	0.000 (0.000)	–	–	–	0.827 (0.320)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}\}$	0.052 (0.153)	0.000 (0.000)	0.097 (0.229)	–	–	0.851 (0.273)
AnisoDiff	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67)\}$	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.185 (0.339)	–	0.815 (0.339)
	$\{\text{Jacobi}, \text{GS}, \text{SymGS}, \text{Jacobi}(0.67), \text{SOR}(1.5)\}$	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.185 (0.339)	0.000 (0.000)	0.815 (0.339)

Table 39: Ensembles at 128×128 : one-sided Wilcoxon p -values at $\varepsilon = h^2$ for the ensemble router against the fastest member solver alone (censored runs at their lower bound), against the best pairwise router in both directions, and against the cost-aware oracle over the same ensemble (alternative: the ensemble router is slower).

Equation	$\mathcal{W}$	vs. best solver only	vs. best pairwise router (faster)	vs. best pairwise router (slower)	vs. oracle($\text{NO} \cup \mathcal{W}$) (slower)
Poisson	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-7}	1.000	0.040
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	0.001	0.999	0.001
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.856	0.144	10^{-4}
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	1.000	10^{-8}	10^{-8}
ConvDiff	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-4}	1.000	10^{-5}
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	10^{-10}	1.000	0.492
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.783	0.217	0.001
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	0.047	0.953	0.566
AnisoDiff	$\{Jacobi, GS\}$	$< 10^{-10}$	0.315	0.685	0.014
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	0.055	0.945	10^{-4}
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.551	0.449	0.064
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	0.633	0.367	0.012

Table 40: Control on router capacity for the ensembles (128×128 , 64 instances): the default ensemble router (hidden width 64, 2 DAgger rounds, 128 oracle rollouts) against one with hidden width 128, 6 DAgger rounds and 256 rollouts. Times to h^2 are median work units net of the per-decision cost; “same decisions” is the share of instances on which both routers take the same number of iterations and corrector calls to h^2 ; p is the one-sided Wilcoxon p -value on the paired net times; “agreement” is each router’s agreement with the oracle on its own rollouts during training.

Equation	$\mathcal{W}$	default router	larger router	oracle	larger / default (medians)	same decisions	p (larger faster / slower)	agreement (default / larger)
Poisson	$\{Jacobi, GS\}$	858 μs	758 μs	758 μs	0.884 $\times$	84%	0.933 / 0.067	80% / 89%
	$\{Jacobi, GS, \text{SymGS}\}$	742 μs	742 μs	742 μs	1.000 $\times$	89%	0.525 / 0.494	73% / 73%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	740 μs	740 μs	740 μs	1.000 $\times$	61%	0.958 / 0.042	55% / 76%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	732 μs	732 μs	732 μs	1.000 $\times$	62%	10^{-4} / 0.999	65% / 64%
ConvDiff	$\{Jacobi, GS\}$	1.07 ms	887 μs	854 μs	0.829 $\times$	69%	0.005 / 0.995	83% / 86%
	$\{Jacobi, GS, \text{SymGS}\}$	854 μs	890 μs	856 μs	1.042 $\times$	91%	0.995 / 0.005	58% / 65%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	888 μs	888 μs	932 μs	1.000 $\times$	56%	0.459 / 0.541	57% / 70%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	892 μs	892 μs	936 μs	1.000 $\times$	53%	$1.000 / 10^{-8}$	65% / 74%

1082 E.6 Overheads and amortisation

1083 Table 41 lists the measured cost of every operation as a function of N , including one decision of
1084 the LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one
1085 LSTM decision costs 2.4 ms, about 46 Jacobi sweeps, which is why the wall-clock study uses the
1086 scalar-feature router ($\approx 5 \mu\text{s}$, independent of N). The learned router’s decisions therefore account
1087 for well under 1% of its solve time, and its corrector calls for 50–70% of the time to h^2 (one or two
1088 calls). Table 42 accounts for training: the corrector (shared with HINTS) is fit in minutes, the router
1089 in seconds to a few minutes per pairing, and the break-even number of solves after which the router’s
1090 own training cost is recovered relative to HINTS ($\tau=25$) with the same corrector is in the thousands
1091 at 128^2 and in the hundreds on the larger grids.

Table 41: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

Operation	$N = 32$	$N = 64$	$N = 128$	$N = 256$	$N = 512$
Jacobi iteration	14 μs	25 μs	53 μs	191 μs	1.07 ms
GS iteration	25 μs	64 μs	209 μs	820 μs	3.74 ms
SymGS iteration	38 μs	109 μs	378 μs	1.48 ms	6.85 ms
Multigrid V(2,2) cycle	—	390 μs	1.35 ms	5.27 ms	25.35 ms
DeepONet corrector call	—	—	663 μs	1.13 ms	3.62 ms
Router decision (ours)	5 μs	5 μs	5 μs	5 μs	5 μs
LSTM router decision (Section 7)	236 μs	696 μs	2.43 ms	9.55 ms	37.46 ms

Table 42: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and DAGger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS ($\tau=25$) and to the solver alone, and the number of solves after which the router’s training cost is recovered relative to HINTS.

Equation	N	Corrector data + fit	Router training (GS pairing)	Saving per solve vs HINTS ($\tau=25$)	Break-even solves (router)	Saving per solve vs solver only
Poisson	128^2	82 s	8 s	5.64 ms	1419	1.83 s
	256^2	98 s	34 s	18.1 ms	1883	28.82 s
	512^2	504 s	229 s	11.4 ms	20139	—
Convection–diffusion	128^2	79 s	8 s	6.92 ms	1157	1.63 s
	256^2	106 s	33 s	24.0 ms	1376	30.05 s
	512^2	493 s	254 s	34.2 ms	7423	—

E.7 Verification of the theoretical assumptions

Theorem 4.1 and Propositions 4.2–4.3 assume that every error propagation map $G_j = I - C_j \circ \mathcal{L}_h^a$ is Lipschitz with constant $\rho_j < 1$ (respectively ≤ 1 with a shared eigenbasis), zero-preserving, and—for postfix monotonicity—invertible; Theorem 5.1 assumes that the per-step errors $\tilde{c}_j$ are bounded above and that at least one solver cannot annihilate the error in one step. Table 43 verifies these conditions numerically for every operation of the study on the mean-free subspace (the constant mode is the null space of the periodic operator and is excluded from the data and from every iteration): the Euclidean Lipschitz constant $\|G_j\|_2$ (power iteration on $G_j^\top G_j$; exact from the Fourier symbol for Jacobi), the Lipschitz constant $\|G_j\|_A = \|A_s^{1/2} G_j A_s^{-1/2}\|_2$ in the energy norm of the symmetric part A_s of the discrete operator, the same for the macro-action $G_j^{m_j}$, the spectral radius, the smallest observed contraction $\sigma_{\min} = \min \|G_j x\|/\|x\|$, $\|C_j(0)\|$, and the commutator $\|G_j G_{\text{NO}} - G_{\text{NO}} G_j\|$ on random vectors. Table 44 evaluates the quantities of Theorem 4.1 and Theorem 5.1 along the cost-aware oracle’s own path to h^2 .

What holds, and in which norm. (i) Every classical solver is zero-preserving exactly and contracts in the sense that matters for convergence: the spectral radius of G_j is below 1 in every setting (at most 0.999997), and so is the energy-norm Lipschitz constant of the solver and of its macro-action (at most 0.999912). The *Euclidean* Lipschitz constant, however, exceeds 1 for Gauss–Seidel, SOR and symmetric GS (up to 1.89 for SOR), whose iteration matrices are non-normal; the proofs of Propositions 4.2–4.3 use only the triangle inequality and the composition rule for Lipschitz constants, so they hold verbatim with $\|\cdot\|_2$ replaced by any norm, and in the energy norm the Lipschitz condition is satisfied by all classical solvers in all settings. For (damped) Jacobi both constants agree (the maps are Fourier-diagonal); the values are close to 1 because the slowest modes (the smoothest ones, and the near-Nyquist mode for undamped Jacobi) are barely damped by a point smoother—which is exactly why the corrector is needed. (ii) The corrector’s map is zero-preserving, contracts every band mode to 1.7×10^{-3} or better (on average to 10^{-4} – 10^{-6}) and leaves the modes outside the band untouched, so its Lipschitz constant is exactly 1 (not < 1): Theorem 4.2 in its strict form does not apply to the corrector, but Theorem 4.3 does, and its second condition—a shared eigenbasis—holds exactly for the ensembles $\{\text{NO}, \text{Jacobi}\}$, $\{\text{NO}, \text{Jacobi} (0.67)\}$ and $\{\text{NO}, \text{Jacobi}, \text{Jacobi} (0.67)\}$ (all maps are diagonal in the Fourier basis; measured commutators $\approx 10^{-6}$, at the corrector’s float32 precision), so h is supermodular for them and Theorem 4.1 holds with $\alpha = 1$; for GS, SOR, symmetric GS and multigrid the commutators are small but nonzero (10^{-3} – 10^{-1}). (iii) Invertibility fails for most maps in exact arithmetic: a Gauss–Seidel, SOR or symmetric-GS sweep annihilates an error concentrated at the first grid point ($\sigma_{\min} = 0$), undamped Jacobi annihilates the mode $(N/4, N/4)$, and the corrector annihilates the band; postfix monotonicity is therefore not guaranteed, which Section 4 anticipates (“our experiments show that greedy routers remain effective even when invertibility is not met”). (iv) The worst-case ratio $\alpha(O)$ of Theorem 4.2, which only uses the Lipschitz constants, is vacuous here (Table 44: $\sum_i \rho_{O_i}^2 \approx T$ because each ρ is dominated by the slowest mode), whereas the empirically measured supermodularity ratio of Equation (7) along the oracle’s path—with S the oracle’s prefix and S' its suffix—is at most 1.000, i.e., the objective behaves supermodularly on the states the greedy rule actually visits, which is the regime in which Theorem 4.1 gives the $(1 - e^{-1})$ guarantee. (v) For Theorem 5.1, the per-step errors are bounded by the current error ($\max_j \tilde{c}_j / \|e^{(t)}\|^2 \leq 1$ at every visited state) and at least one solver leaves a positive fraction of it ($\min_t \max_j \tilde{c}_j / \|e^{(t)}\|^2 > 0$), so the surrogate’s Bayes consistency applies.

Table 43: Numerical verification of the assumptions of Propositions 4.2–4.3 for every operation of the study (mean-free subspace). $\|\cdot\|_2$: Euclidean Lipschitz constant of the error propagation map; $\|\cdot\|_A$: Lipschitz constant in the energy norm of the symmetric part of $\mathcal{L}_h$, for one iteration and for the macro-action of m_j iterations; ρ : spectral radius; $\sigma_{\min}$: smallest observed contraction (a unit error at the first grid point for the triangular sweeps, the minimum over Fourier modes otherwise; for the corrector, the largest remaining fraction of a band mode); $\|C_j(0)\|$: zero preservation; last column: commutator with the corrector’s map on random unit vectors (Proposition 4.3). For (damped) Jacobi the constants are exact from the Fourier symbol (the value with the Nyquist modes included; without them undamped Jacobi has $\|G\|_2 = \rho = 0.9994$ at 128^2).

Equation	Operation	m_j	$\ I - C_j \mathcal{L}_h\ _2$	$\ I - C_j \mathcal{L}_h\ _A$	$\ (I - C_j \mathcal{L}_h)^{m_j}\ _A$	$\rho(I - C_j \mathcal{L}_h)$	$\sigma_{\min}$	$\ C_j(0)\ $	$\ [G_j, G_{\text{NO}}]\ $
Poisson, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	12	0.999398	0.999398	0.992797	0.999398	0	0	5.7e-06
	Jacobi (0.67)	12	0.999596	0.999596	0.995168	0.999596	1.4e-04	0	4.1e-06
	GS	3	1.021515	0.998595	0.996368	0.998797	0	0	3.1e-02
	SymGS	2	1.006607	0.997396	0.995187	0.997623	0	0	2.2e-02
	SOR (1.5)	3	1.449417	0.996467	0.989336	0.996411	7.2e-01	0	8.5e-02
	Multigrid	1	0.098203	0.097860	0.097860	0.096877	1.9e-03	0	8.8e-03
Poisson, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	6	0.999849	0.999849	0.999097	0.999849	0	0	1.3e-06
	Jacobi (0.67)	6	0.999899	0.999899	0.999395	0.999899	5.6e-05	0	1.2e-06
	GS	1	1.022053	0.998925	0.998925	0.999699	0	0	1.1e-02
	SymGS	1	1.005470	0.998839	0.998839	0.999401	0	0	9.4e-03
	SOR (1.5)	1	1.451367	0.998352	0.998352	0.999098	7.2e-01	0	3.8e-02
	Multigrid	1	0.102361	0.102263	0.102263	0.101344	1.4e-03	0	4.2e-03
Poisson, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	2.1e-05 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	6.1e-07
	GS	1	1.022202	0.998983	0.998983	0.999925	0	0	4.5e-03
	SOR (1.5)	1	1.451920	0.999043	0.999043	0.999774	7.2e-01	0	1.7e-02
	Multigrid	1	0.103470	0.103434	0.103434	0.102511	1.0e-03	0	1.3e-03
Convection–diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.5e-05 (band)	0	–
	Jacobi	9	0.999398	0.999398	0.994593	0.999398	0	0	3.1e-06
	Jacobi (0.67)	9	0.999596	0.999596	0.996374	0.999596	1.4e-04	0	2.2e-06
	GS	3	1.025726	0.998460	0.995816	0.998589	0	0	3.1e-02
	SymGS	2	1.010112	0.997353	0.995168	0.997571	0	0	2.3e-02
	SOR (1.5)	3	1.554544	0.996981	0.983568	0.993973	7.5e-01	0	9.1e-02
	Multigrid	1	0.076967	0.074905	0.074905	0.074609	1.5e-03	0	6.9e-03
Convection–diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.1e-05 (band)	0	–
	Jacobi	4	0.999849	0.999849	0.999398	0.999849	0	0	7.6e-07
	Jacobi (0.67)	4	0.999899	0.999899	0.999596	0.999899	5.6e-05	0	6.8e-07
	GS	1	1.024025	0.998934	0.998934	0.999674	0	0	1.2e-02
	SymGS	1	1.008004	0.998846	0.998846	0.999398	0	0	9.8e-03
	SOR (1.5)	1	1.497088	0.998192	0.998192	0.998846	7.3e-01	0	4.1e-02
	Multigrid	1	0.084356	0.083809	0.083809	0.083459	1.2e-03	0	3.7e-03
Convection–diffusion, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	9.3e-06 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	3.4e-07
	GS	1	1.023154	0.998983	0.998983	0.999922	0	0	4.6e-03
Anisotropic diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-04 (band)	0	–
	Jacobi	11	0.999988	0.999988	0.999869	0.999988	1.1e-16	0	2.4e-05
	Jacobi (0.67)	11	0.999992	0.999992	0.999912	0.999992	4.4e-03	0	1.6e-05
	GS	3	1.131668	0.999528	0.999537	0.999976	0	0	5.7e-02
	SymGS	2	1.102789	0.999489	0.999255	0.999953	0	0	5.5e-02
	SOR (1.5)	3	1.889774	0.999393	0.999471	0.999928	8.3e-01	0	1.5e-01
Anisotropic diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.7e-03 (band)	0	–
	Jacobi	38	0.999997	0.999997	0.999887	0.999997	1.1e-16	0	9.7e-05
	GS	11	1.131845	0.999245	0.999485	0.999994	0	0	4.0e-02
	SOR (1.5)	11	1.890225	0.999352	0.999473	0.999982	8.3e-01	0	1.1e-01

Table 44: Quantities of Theorem 4.1, Proposition 4.2 and Theorem 5.1 along the cost-aware oracle’s path to h^2 (8 test instances; medians unless stated): number of macro-actions T , $\sum_i \rho_{O_i}^2$ with the energy-norm constants of the chosen macro-actions, the resulting $\alpha(O)$ of Proposition 4.2 (∞ when $T - \sum_i \rho_{O_i}^2 \leq 0$), the empirical supermodularity ratio $\hat{\alpha}$ of Equation (7) evaluated with S the oracle’s prefix and S' its suffix (maximum and median over prefixes and instances), and the normalised bounds of Theorem 5.1: the largest ratio $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ over visited states ($\bar{E}$) and the smallest value of $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ ($E_{\min}$).

Equation	Ensemble	T	$\sum_i \rho_{O_i}^2$	$\alpha(O)$ (Prop. 4.2)	$\hat{\alpha}$ max	$\hat{\alpha}$ median	$\bar{E} / \ e\ ^2$	$E_{\min} / \ e\ ^2$
Poisson, 128 ²	{Jacobi}	1	1.000	∞	1.000	1.000	1.000	9.8e-01
	{Jacobi(0.67)}	1	1.000	∞	1.000	1.000	1.000	9.8e-01
	{GS}	1	1.000	∞	1.000	1.000	1.000	9.9e-01
	{SymGS}	1	1.000	∞	1.000	1.000	1.000	9.8e-01
	{SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	8.0e-01
	{Multigrid}	1	1.000	∞	1.000	1.000	1.000	9.6e-06
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	9.9e-01
Poisson, 256 ²	{Jacobi}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{Jacobi(0.67)}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{GS}	1	1.000	∞	1.000	1.000	1.000	9.5e-01
	{SymGS}	1	1.000	∞	1.000	1.000	1.000	9.8e-01
	{SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	8.6e-01
	{Multigrid}	1	1.000	∞	1.000	1.000	1.000	1.1e-05
Poisson, 512 ²	{Jacobi}	9	8.989	∞	1.000	0.664	1.000	9.7e-01
	{GS}	14	14.061	∞	1.000	0.635	1.000	8.7e-01
	{SOR(1.5)}	7	13.648	∞	1.000	0.700	1.000	7.7e-01
	{Multigrid}	2	1.011	∞	1.000	0.750	1.000	1.0e-05
Convection–diffusion, 128 ²	{Jacobi}	2	1.500	∞	1.000	0.995	1.000	9.7e-01
	{Jacobi(0.67)}	2	1.496	∞	1.000	0.997	1.000	9.8e-01
	{GS}	2	1.516	∞	1.000	0.996	1.000	7.6e-01
	{SymGS}	2	1.504	∞	1.000	0.996	1.000	9.8e-01
	{SOR(1.5)}	2	1.849	∞	1.000	0.987	1.000	6.5e-01
	{Multigrid}	2	1.003	∞	1.000	0.875	1.000	8.3e-06
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	2	1.496	∞	1.000	0.997	1.000	9.8e-01
Convection–diffusion, 256 ²	{Jacobi}	2	2.497	∞	1.000	0.998	1.000	1.0e+00
	{Jacobi(0.67)}	2	2.497	∞	1.000	0.999	1.000	1.0e+00
	{GS}	4	3.622	∞	1.000	0.930	1.000	7.3e-01
	{SymGS}	2	2.524	∞	1.000	0.998	1.000	8.4e-01
	{SOR(1.5)}	4	6.603	∞	1.000	0.916	1.000	6.4e-01
	{Multigrid}	2	1.007	∞	1.000	0.750	1.000	2.8e-06
Anisotropic diffusion, 128 ²	{Jacobi}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{Jacobi(0.67)}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{GS}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{SymGS}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
Anisotropic diffusion, 256 ²	{Jacobi}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{GS}	1	1.000	∞	1.000	1.000	1.000	1.0e+00
	{SOR(1.5)}	1	1.000	∞	1.000	1.000	1.000	1.0e+00

F Limitations

A primary limitation of our approach is the higher on-time computational cost requires to learn the greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead. While this cost is amortized during inference time and leads to improved solution quality, it may be prohibitive in settings with limited computational resources or when rapid deployment is required. On the deployment side, Section E shows that the gains survive cost accounting: with a corrector that covers the smooth half of the spectrum and a classical smoother that covers the rest, one or two corrector calls suffice and the router’s advantage over fixed schedules is uniform across solvers, equations and tolerances. The corrector we use is specific to the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is restricted to single-thread CPU execution; with a corrector band that does not grow with the grid, the gains over fixed schedules shrink at 512² for the point smoothers, whereas pairing the corrector with geometric multigrid restores them (Section E). Ensembles of classical solvers pay off only where the error passes through regimes that favour different members; at truncation level a single well-chosen smoother suffices.

Additionally, the effectiveness of the learned router depends on the neural operator exhibiting heterogeneous performance across samples. When the neural operators produces predictions of relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity can be sensitive to training choices, and neural operators themselves can be finicky to train.

1156 **G LLM Usage**

1157 LLMs, specifically ChatGPT and Gemini, supported the writing process in an iterative manner. We
1158 drafted paragraphs and asked the models for feedback on grammar and clarity. We then incorporated
1159 selected suggestions into the writing and repeated this process until we were satisfied with the writing.

1160 The code developed for the experiments was written by the authors with the help of occasional code
1161 completions. The central components (e.g., the hybrid solver implementation and the greedy-router
1162 training pipelines) were implemented exclusively by the authors.

1163 All substantive intellectual contributions, which include ideas, theorems, and analyses, are our own.
1164 LLMs were occasionally used to verify the correctness of proofs, but all proof strategies originated
1165 from the authors and relevant literature.

1166 NeurIPS Paper Checklist

1167 The checklist is designed to encourage best practices for responsible machine learning research,
1168 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
1169 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
1170 follow the references and follow the (optional) supplemental material. The checklist does NOT count
1171 towards the page limit.

1172 Please read the checklist guidelines carefully for information on how to answer these questions. For
1173 each question in the checklist:

- 1174 • You should answer [Yes], [No], or [N/A].
- 1175 • [N/A] means either that the question is Not Applicable for that particular paper or the
1176 relevant information is Not Available.
- 1177 • Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

1178 **The checklist answers are an integral part of your paper submission.** They are visible to the
1179 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
1180 (after eventual revisions) with the final version of your paper, and its final version will be published
1181 with the paper.

1182 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
1183 While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a
1184 proper justification is given (e.g., error bars are not reported because it would be too computationally
1185 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
1186 [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we
1187 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
1188 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
1189 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
1190 please point to the section(s) where related material for the question can be found.

1191 IMPORTANT, please:

- 1192 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- 1193 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 1194 • **Do not modify the questions and only use the provided macros for your answers.**

1195 1. Claims

1196 Question: Do the main claims made in the abstract and introduction accurately reflect the
1197 paper’s contributions and scope?

1198 Answer: [Yes]

1199 Justification: The abstract clearly delineates the paper’s main contributions and scope, and
1200 the introduction reiterates these claims in a structured, bullet-point form. All stated claims
1201 are directly supported by the theoretical and experimental results presented in the paper.

1202 Guidelines:

- 1203 • The answer [N/A] means that the abstract and introduction do not include the claims
1204 made in the paper.
- 1205 • The abstract and/or introduction should clearly state the claims made, including the
1206 contributions made in the paper and important assumptions and limitations. A [No] or
1207 [N/A] answer to this question will not be perceived well by the reviewers.
- 1208 • The claims made should match theoretical and experimental results, and reflect how
1209 much the results can be expected to generalize to other settings.
- 1210 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1211 are not attained by the paper.

1212 2. Limitations

1213 Question: Does the paper discuss the limitations of the work performed by the authors?

1214 Answer: [Yes]

1215 Justification: The paper discusses key limitations in Appendix E.

1216 Guidelines:

- 1217 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1218 that the paper has limitations, but those are not discussed in the paper.
- 1219 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1220 • The paper should point out any strong assumptions and how robust the results are to
1221 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1222 model well-specification, asymptotic approximations only holding locally). The authors
1223 should reflect on how these assumptions might be violated in practice and what the
1224 implications would be.
- 1225 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1226 only tested on a few datasets or with a few runs. In general, empirical results often
1227 depend on implicit assumptions, which should be articulated.
- 1228 • The authors should reflect on the factors that influence the performance of the approach.
1229 For example, a facial recognition algorithm may perform poorly when image resolution
1230 is low or images are taken in low lighting. Or a speech-to-text system might not be
1231 used reliably to provide closed captions for online lectures because it fails to handle
1232 technical jargon.
- 1233 • The authors should discuss the computational efficiency of the proposed algorithms
1234 and how they scale with dataset size.
- 1235 • If applicable, the authors should discuss possible limitations of their approach to
1236 address problems of privacy and fairness.
- 1237 • While the authors might fear that complete honesty about limitations might be used by
1238 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1239 limitations that aren’t acknowledged in the paper. The authors should use their best
1240 judgment and recognize that individual actions in favor of transparency play an impor-
1241 tant role in developing norms that preserve the integrity of the community. Reviewers
1242 will be specifically instructed to not penalize honesty concerning limitations.

1243 3. Theory assumptions and proofs

1244 Question: For each theoretical result, does the paper provide the full set of assumptions and
1245 a complete (and correct) proof?

1246 Answer: [Yes]

1247 Justification: All theoretical results include clearly stated assumptions in the statement
1248 of theorems. Complete proofs are provided in Appendices A and B with appropriate
1249 cross-referencing from the main text.

1250 Guidelines:

- 1251 • The answer [N/A] means that the paper does not include theoretical results.
- 1252 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
1253 referenced.
- 1254 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 1255 • The proofs can either appear in the main paper or the supplemental material, but if
1256 they appear in the supplemental material, the authors are encouraged to provide a short
1257 proof sketch to provide intuition.
- 1258 • Inversely, any informal proof provided in the core of the paper should be complemented
1259 by formal proofs provided in appendix or supplemental material.
- 1260 • Theorems and Lemmas that the proof relies upon should be properly referenced.

1261 4. Experimental result reproducibility

1262 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
1263 perimental results of the paper to the extent that it affects the main claims and/or conclusions
1264 of the paper (regardless of whether the code and data are provided or not)?

1265 Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including training procedures and implementation details described in the main text and Appendix C. Additionally, we release code with a comprehensive README to support the reproducibility of our results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: The paper specifies all key training and evaluation details in the Section 6 and Appendix C.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: The paper reports standard deviations in the main results tables, and further provides statistical significance via t-test p values in Appendix D.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

1374 Answer: [Yes]

1375 Justification: The paper provides details on compute resources in Appendix C.

1376 Guidelines:

- 1377 • The answer [N/A] means that the paper does not include experiments.
- 1378 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
- 1379 or cloud provider, including relevant memory and storage.
- 1380 • The paper should provide the amount of compute required for each of the individual
- 1381 experimental runs as well as estimate the total compute.
- 1382 • The paper should disclose whether the full research project required more compute
- 1383 than the experiments reported in the paper (e.g., preliminary or failed experiments that
- 1384 didn't make it into the paper).

1385 **9. Code of ethics**

1386 Question: Does the research conducted in the paper conform, in every respect, with the

1387 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

1388 Answer: [Yes]

1389 Justification: This paper does not involve any human subjects and all data has been simu-

1390 lated. This research focuses on numerical methods for physical systems and does not raise

1391 concerns related to safety, security, discrimination, surveillance, deception & harrasment,

1392 environment, human rights, and bias & fairness.

1393 Guidelines:

- 1394 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
- 1395 Ethics.
- 1396 • If the authors answer [No], they should explain the special circumstances that require a
- 1397 deviation from the Code of Ethics.
- 1398 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
- 1399 eration due to laws or regulations in their jurisdiction).

1400 **10. Broader impacts**

1401 Question: Does the paper discuss both potential positive societal impacts and negative

1402 societal impacts of the work performed?

1403 Answer: [Yes]

1404 Justification: The paper discusses broader impacts in the introduction section. No direct

1405 negative societal impacts are anticipated.

1406 Guidelines:

- 1407 • The answer [N/A] means that there is no societal impact of the work performed.
- 1408 • If the authors answer [N/A] or [No], they should explain why their work has no societal
- 1409 impact or why the paper does not address societal impact.
- 1410 • Examples of negative societal impacts include potential malicious or unintended uses
- 1411 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
- 1412 (e.g., deployment of technologies that could make decisions that unfairly impact specific
- 1413 groups), privacy considerations, and security considerations.
- 1414 • The conference expects that many papers will be foundational research and not tied
- 1415 to particular applications, let alone deployments. However, if there is a direct path to
- 1416 any negative applications, the authors should point it out. For example, it is legitimate
- 1417 to point out that an improvement in the quality of generative models could be used to
- 1418 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
- 1419 that a generic algorithm for optimizing neural networks could enable people to train
- 1420 models that generate Deepfakes faster.
- 1421 • The authors should consider possible harms that could arise when the technology is
- 1422 being used as intended and functioning correctly, harms that could arise when the
- 1423 technology is being used as intended but gives incorrect results, and harms following
- 1424 from (intentional or unintentional) misuse of the technology.

- 1425 • If there are negative societal impacts, the authors could also discuss possible mitigation
1426 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1427 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1428 feedback over time, improving the efficiency and accessibility of ML).

1429 11. Safeguards

1430 Question: Does the paper describe safeguards that have been put in place for responsible
1431 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1432 image generators, or scraped datasets)?

1433 Answer: [N/A]

1434 Justification: The paper does not release any models or datasets at all. It focuses on method-
1435 ological contributions for numerical solvers using simulated data. Therefore, safeguards are
1436 not applicable.

1437 Guidelines:

- 1438 • The answer [N/A] means that the paper poses no such risks.
- 1439 • Released models that have a high risk for misuse or dual-use should be released with
1440 necessary safeguards to allow for controlled use of the model, for example by requiring
1441 that users adhere to usage guidelines or restrictions to access the model or implementing
1442 safety filters.
- 1443 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1444 should describe how they avoided releasing unsafe images.
- 1445 • We recognize that providing effective safeguards is challenging, and many papers do
1446 not require this, but we encourage authors to take this into account and make a best
1447 faith effort.

1448 12. Licenses for existing assets

1449 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1450 the paper, properly credited and are the license and terms of use explicitly mentioned and
1451 properly respected?

1452 Answer: [N/A]

1453 Justification: The paper does not use any external datasets or pretrained models. All
1454 experiments are conducted using simulated data and models trained in-house.

1455 Guidelines:

- 1456 • The answer [N/A] means that the paper does not use existing assets.
- 1457 • The authors should cite the original paper that produced the code package or dataset.
- 1458 • The authors should state which version of the asset is used and, if possible, include a
1459 URL.
- 1460 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 1461 • For scraped data from a particular source (e.g., website), the copyright and terms of
1462 service of that source should be provided.
- 1463 • If assets are released, the license, copyright information, and terms of use in the
1464 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1465 has curated licenses for some datasets. Their licensing guide can help determine the
1466 license of a dataset.
- 1467 • For existing datasets that are re-packaged, both the original license and the license of
1468 the derived asset (if it has changed) should be provided.
- 1469 • If this information is not available online, the authors are encouraged to reach out to
1470 the asset's creators.

1471 13. New assets

1472 Question: Are new assets introduced in the paper well documented and is the documentation
1473 provided alongside the assets?

1474 Answer: [Yes]

1475 Justification: We release code for our method which is documented with a detailed
1476 README.

1477 Guidelines:

1478 • The answer [N/A] means that the paper does not release new assets.

1479 • Researchers should communicate the details of the dataset/code/model as part of their

1480 submissions via structured templates. This includes details about training, license,

1481 limitations, etc.

1482 • The paper should discuss whether and how consent was obtained from people whose

1483 asset is used.

1484 • At submission time, remember to anonymize your assets (if applicable). You can either

1485 create an anonymized URL or include an anonymized zip file.

1486 **14. Crowdsourcing and research with human subjects**

1487 Question: For crowdsourcing experiments and research with human subjects, does the paper

1488 include the full text of instructions given to participants and screenshots, if applicable, as

1489 well as details about compensation (if any)?

1490 Answer: [N/A]

1491 Justification: The paper does not involve crowdsourcing or any research with human subjects.

1492 All experiments are conducted using simulated data.

1493 Guidelines:

1494 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1495 with human subjects.

1496 • Including this information in the supplemental material is fine, but if the main contribu-

1497 tion of the paper involves human subjects, then as much detail as possible should be

1498 included in the main paper.

1499 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1500 or other labor should be paid at least the minimum wage in the country of the data

1501 collector.

1502 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1503 **subjects**

1504 Question: Does the paper describe potential risks incurred by study participants, whether

1505 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1506 approvals (or an equivalent approval/review based on the requirements of your country or

1507 institution) were obtained?

1508 Answer: [N/A]

1509 Justification: This paper does not involve human subjects. All experiments are conducted

1510 using simulated data. Therefore, IRB approval is not applicable.

1511 Guidelines:

1512 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1513 with human subjects.

1514 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1515 may be required for any human subjects research. If you obtained IRB approval, you

1516 should clearly state this in the paper.

1517 • We recognize that the procedures for this may vary significantly between institutions

1518 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1519 guidelines for their institution.

1520 • For initial submissions, do not include any information that would break anonymity (if

1521 applicable), such as the institution conducting the review.

1522 **16. Declaration of LLM usage**

1523 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1524 non-standard component of the core methods in this research? Note that if the LLM is used

1525 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1526 scientific rigor, or originality of the research, declaration is not required.

1527 Answer: [N/A]

1528 Justification: LLMs are primarily used for writing and editing. Further details are provided
1529 in Appendix F.
1530 Guidelines:
1531 • The answer [N/A] means that the core method development in this research does not
1532 involve LLMs as any important, original, or non-standard components.
1533 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1534 be described.